The CAD Pillar

SECURITY AUDIT, ARCHITECTURE BLUEPRINT AND DEVELOPER HANDOVER

Master Technical Handbook

The permanent technical reference for founders, developers and security reviewers: architecture, data, access, operations and risk.

ORGANISATION	The CAD Pillar · a brand of Ichizu Private Limited
VERSION	1.0 · generated 13 November 2026
CODE REVISION	main @ `1c08b52` · github.com/simran313/GetCAD
CLASSIFICATION	Internal. Sanitised for repository access; exploit-sensitive detail is in a restricted annex held by the owner.

Contents

1 **Executive technical overview** 8

1.1 What The CAD Pillar is, technically 8

1.2 The design idea that matters most 9

1.3 The application in numbers 9

1.4 Health at a glance 9

1.5 The five things to do next 10

1.6 How to read this handbook 10

2 **Complete application architecture** 11

2.1 Architectural style 11

2.2 Repository layout 12

2.3 Layers and dependency direction 13

2.4 Runtime topology 13

2.5 How a request flows 14

2.6 Where decisions are made 14

2.7 Key architectural decisions (and why) 15

2.8 Cross-cutting concerns 15

3 **Founder’s technical education** 16

3.1 The building blocks, as a shop 16

3.2 How a page reaches someone’s screen 16

3.3 Sign-in: who are you? 17

3.4 Permissions: what may you do? 17

3.5 Money: escrow in software 17

3.6 Files and your intellectual property 17

3.7 The audit log: your black box recorder 18

3.8 How changes reach the live site 18

3.9 Secrets and keys 18

3.10 Backups and recovery 18

3.11 Questions to ask any developer 18

4 **Frontend and backend** 19

4.1 Technology stack 19

4.2 Frontend 19

4.3 Backend 21

5 **Database schema, relationships and row-level security** 23

5.1 Schemas 23

5.2 Table inventory (69 tables) 23

5.3 Keys and identity 24
5.4 Row-level security: how it is applied 24
5.5 Invariants enforced by the database 26
5.6 The order state machine 26
5.7 Triggers you should know 27
5.8 Changing the schema safely 28
5.9 Regenerating the reference appendices 28

6 Authentication, authorisation and the role matrix 29
6.1 Authentication (Clerk) 29
6.2 The identity bridge (Clerk to Supabase) 30
6.3 Authorisation model 30
6.4 Role and capability matrix 31
6.5 Security properties worth knowing 32

7 API and Server Action documentation 33
7.1 Conventions 33
7.2 Route handlers (20) 33
7.3 Server Actions (92 in 31 files) 35

8 File storage and CAD intellectual-property protection 37
8.1 Where files live 37
8.2 The way in 38
8.3 The way out 38
8.4 Proof of delivery and the spec guarantee 39
8.5 Identity documents (designer compliance) 39
8.6 Threats considered 39

9 Security audit and vulnerability register 40
9.1 Summary 40
9.2 Vulnerability register 40
9.3 Controls verified as effective 44
9.4 Handling of sensitive findings 45

10 Infrastructure, GitHub, Vercel and deployment workflows 46
10.1 Infrastructure map 46
10.2 Environments 47
10.3 GitHub 47
10.4 Delivery pipeline 48
10.5 Domains and DNS records (Vercel DNS) 49
10.6 Observability 49

11 Third-party integrations 50

11.1 Summary 50
11.2 Clerk 50
11.3 Supabase 51
11.4 Razorpay 51
11.5 Resend (email) 51
11.6 Web Push 52
11.7 GitHub Actions scheduler 52
11.8 Google 52
11.9 Adding a new integration: checklist 52

12 Feature-by-feature technical reference 54

12.1 Public website and lead capture 54
12.2 Accounts and onboarding 54
12.3 Ordering and the CAD brief 54
12.4 Orders placed on a client's behalf 55
12.5 Categories, quotes and prices 55
12.6 Payments 55
12.7 Confirmation, pod routing and assignment 55
12.8 Designer workspace 56
12.9 Quality control 56
12.10 Client preview, approval, revisions, changes and disputes 56
12.11 Delivery and the spec guarantee 56
12.12 Messaging 57
12.13 Notifications, email and push 57
12.14 SLA and credits 57
12.15 Escrow and payouts 58
12.16 Pod operations 58
12.17 Administration 58
12.18 Calculators 59
12.19 Wallet and plans 59
12.20 Platform features 59

13 Developer onboarding and maintenance 60

13.1 Day 1: read, in this order 60
13.2 Machine setup 60
13.3 Everyday commands 60
13.4 How to… 61
13.5 Code conventions 62
13.6 Pull requests 62
13.7 Routine maintenance 62
13.8 Regenerating this handbook 63

14 Troubleshooting and disaster recovery 64

14.1 First moves for any incident 64

14.2 Troubleshooting by symptom 64

14.3 Disaster recovery 65

15 Technical debt and scalability roadmap 68

15.1 Current scale and headroom 68

15.2 Now (next 30 days) 68

15.3 Next (one to three months) 69

15.4 Later (three to twelve months) 69

15.5 Known debt in the codebase 70

15.6 Principles for future work 70

16 Production readiness checklist 71

16.1 Security 71

16.2 Identity and access 71

16.3 Data and recovery 72

16.4 Infrastructure and operations 72

16.5 Email and domain 72

16.6 Payments 73

16.7 Product walkthrough (after each release, about 15 minutes) 73

17 Architecture and workflow diagrams 74

17.1 System context 74

17.2 Request flow and trust boundaries 74

17.3 Authentication and identity bridge 75

17.4 Order state machine 75

17.5 Money flow and escrow 76

17.6 File pipeline and IP protection 76

17.7 Anonymity wall 77

17.8 Delivery pipeline 77

17.9 Notifications, push and email 78

17.10 Core data model 78

18 Glossary of technical terms 80

19 Audit method, assumptions and missing access 83

19.1 Scope 83

19.2 Method 83

19.3 Verified facts (with evidence) 84

19.4 Assumptions 84

19.5 Not verified (missing infrastructure access) 84

19.6 Repeating this audit 85

Appendix A · Database schema reference 86

Appendix B · Database functions reference 145

Appendix C · Routes reference (pages and API) 168

Appendix D · Server actions reference 174

Appendix E · Configuration variables (names only) 180

Document control

Field	Value
Title	Master Technical Handbook: Security audit, architecture blueprint and developer handover
Version and date	1.0 · 13 November 2026
Audited code revision	`1c08b52` (main)
Source of truth	Markdown in `docs/`, listed in `docs/document-sources/handbook.json`. This PDF and the DOCX are generated: never edit them by hand.
Regenerate	`node docs/document-generation/extract-reference.mjs` (with a local Postgres), then `node docs/document-generation/build-handbook.mjs`. See chapter 13.
Classification	Internal. Sanitised for repository access; exploit-sensitive detail is in a restricted annex held by the owner.
Owner	Ichizu Private Limited (The CAD Pillar). Business decisions: `docs/DECISIONS.md` wins where documents disagree.
Contents	19 chapters and 5 generated appendices (A schema, B functions, C routes, D server actions, E configuration names)

Revision history

Version	Date	Change
1.0	13 November 2026	First edition: full security audit, architecture blueprint and developer handover, generated from the repository after pull request #137.

CHAPTER 1

Executive technical overview

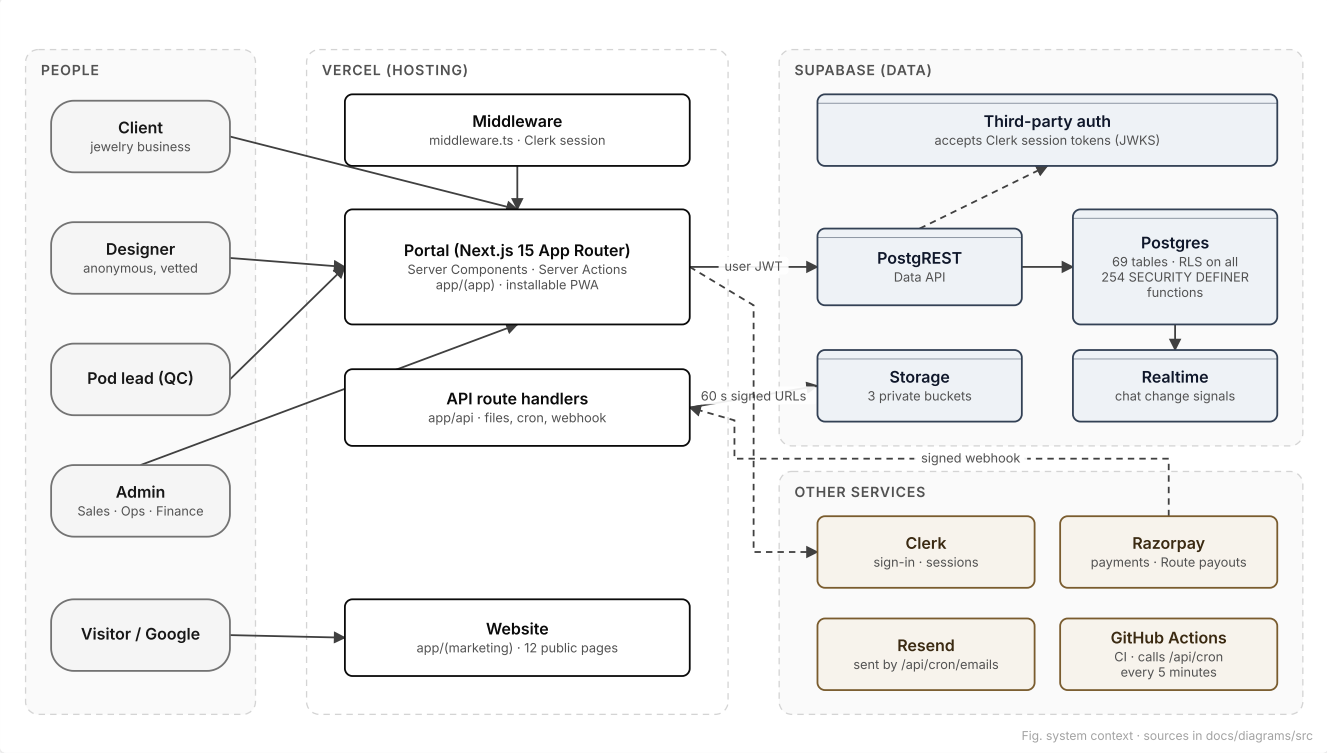
This chapter is the two-page answer to “what have we built, how safe is it, and what needs doing next”. Every number in it comes from the repository at the audited revision (see Audit method and assumptions).

1.1 What The CAD Pillar is, technically

The CAD Pillar (TCP) is a brand of Ichizu Private Limited. It sells jewelry CAD design to the trade: a jewelry business sends a brief, an anonymous vetted designer models the piece, an independent reviewer checks it, the client approves, and only then is money released. The software has three faces, all served from one application:

Face	Who uses it	Where it lives in the code
Public website	Visitors, Google, prospective clients and designers	`app/(marketing)/` (12 pages)
Client portal	Jewelry businesses (clients)	`app/(app)/` routes such as `/home`, `/orders`, `/messages`
Team portal	Designers, pod leads (QC), admins (Sales, Ops, Finance)	`app/(app)/` routes such as `/work`, `/pod`, `/qc`, `/admin`

It is one **Next.js 15** application written in **TypeScript**, hosted on **Vercel**. People sign in with **Clerk**. Data, files and live chat signals live in **Supabase**: Postgres, Storage and Realtime. Payments and payouts go through **Razorpay**. Emails go out through **Resend**, and phone notifications through the Web Push standard. **GitHub** holds the code, runs the automated checks and wakes the scheduled jobs every five minutes.



System context

1.2 The design idea that matters most

The database is the security boundary, not the screens. Every one of the 69 tables has row-level security (RLS) switched on, so a table nobody wrote a rule for is closed. The 254 database functions that do privileged work (SECURITY DEFINER functions) each check who is calling. The web application talks to the database *as the signed-in person*, using that person's Clerk session token. So even a bug in a page cannot show a designer a client's name, and it cannot let a client mark their own order as paid. Four other properties follow from the same idea:

- **Double-blind anonymity.** Clients never learn who designed their piece, and designers never learn who the client is. Identities live in separate tables that only admins can read, and every read is written to the audit log.
- **Money is conserved.** Payments sit in an append-only escrow ledger. A database trigger refuses any release or refund larger than what was collected.
- **History is tamper-evident.** Every state change, money movement and identity read is appended to a hash-chained audit log that cannot be edited or deleted.
- **Files are cleaned and protected.** Uploads are checked by their real file signature, metadata is stripped, and delivered files are fingerprinted (SHA-256). Downloads are decided per request and use links that expire in 60 seconds.

1.3 The application in numbers

Measure	Value
Portal and website pages	73
API route handlers	20
Server actions (form handlers)	92, in 31 files
Database tables	69, all with RLS on
Database policies	43
Database functions	285, of which 254 are SECURITY DEFINER (all with a pinned search path)
SQL migration and policy files	113
Automated tests	1,201 unit and database tests in 132 files, plus 17 browser journey suites
Configuration variables	34 (names in Appendix E)
Real accounts at audit time	26 people moved to production sign-in on 13 November 2026

1.4 Health at a glance

Area	Status	Why
Access control	Strong	RLS on every table, privileged functions check the caller, and anonymity is tested in the browser suite
Payments and money	Strong	Signed webhook with a constant-time check, idempotent handling and a conserved ledger
Files and IP	Strong	Signature checks, metadata stripping, private buckets and 60-second links

Area	Status	Why
Dependencies	**Needs action**	Next.js 15.5.19 and four other packages have published security advisories (SEC-01, SEC-04, SEC-05)
Infrastructure	**Needs action**	The Supabase plan, credential rotation, email domain protection and the Vercel plan (SEC-02, SEC-03, SEC-07, SEC-08)
Code quality and tests	Strong	Strict TypeScript, lint and 1,201 tests run on every pull request; browser journeys run in CI

The full register is in the security audit chapter. **No critical vulnerability was found in The CAD Pillar's own code.** The most urgent items are an upgrade of the Next.js framework, and infrastructure settings that only the owner can change.

1.5 The five things to do next

1. **Upgrade Next.js to the latest 15.5 release (15.5.28 or later), together with `sharp` and `zod`**, in one pull request with the full test suite (SEC-01, SEC-04, SEC-05).
2. **Move the production Supabase project off the Free plan** so it gets daily backups and point-in-time recovery and is never paused. Then rehearse a restore (SEC-02, chapter 14).
3. **Rotate the database password and delete the unused Vercel project** that still holds production database credentials (SEC-03).
4. **Protect the email domain:** publish DMARC, move the Google DKIM key to its proper selector, and verify the transactional sender (SEC-07).
5. **Move Vercel to Pro** (commercial use) and give preview deployments their own non-production database (SEC-08).

1.6 How to read this handbook

- **Founders and managers:** chapters 1, 3, 9, 14 and 16, plus the glossary.
- **New developers:** chapters 2, 4–8, 10–13 and 15, then the appendices.
- **Security reviewers:** chapters 6, 8 and 9, plus Audit method and assumptions.

Everything here is generated from Markdown in `docs/`. The PDF and Word files are rebuilt by one command (chapter 13, "Regenerating this handbook").

CHAPTER 2

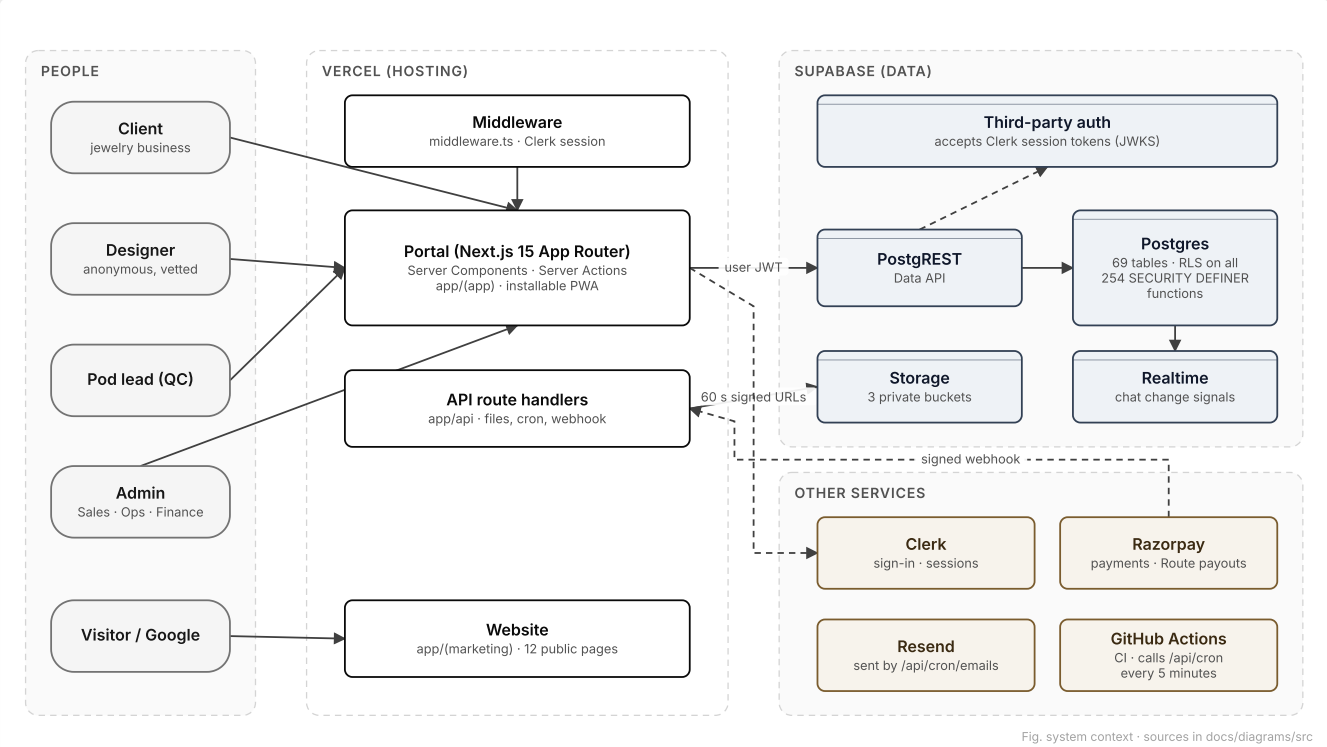
Complete application architecture

This chapter describes how the system is put together: the layers, which way dependencies flow, where every kind of decision is made, and why. Read it before you change anything structural.

2.1 Architectural style

The CAD Pillar is a **modular monolith with a database-centred security model**:

- **One deployable application.** The website, both portals and the API are a single Next.js 15 App Router project deployed to Vercel. There are no microservices to keep in step.
- **A pure domain core.** `core/` holds the business rules: money arithmetic, ids, state names, size charts, weight calculators, notification and email wording, support hours. A lint rule forbids it from importing React or Next.js, and breaking that rule fails CI (`.eslintrc.cjs`, `no-restricted-imports`). Rules there are tested in milliseconds and can never depend on a request.
- **The database enforces the invariants.** Postgres decides who may read and write what (RLS and SECURITY DEFINER functions), which order moves are legal (`public.order_transitions`), whether money is conserved (ledger triggers), and keeps the audit trail. The web layer gathers input and shows results; it is never the only line of defence.
- **Thin adapters around the core.** `lib/` connects to the outside world (Supabase clients, Clerk, Razorpay, Resend, web push, PDF and image generation, file sanitisation helpers). `app/` and `components/` are the delivery layer (pages, server actions, route handlers, UI).

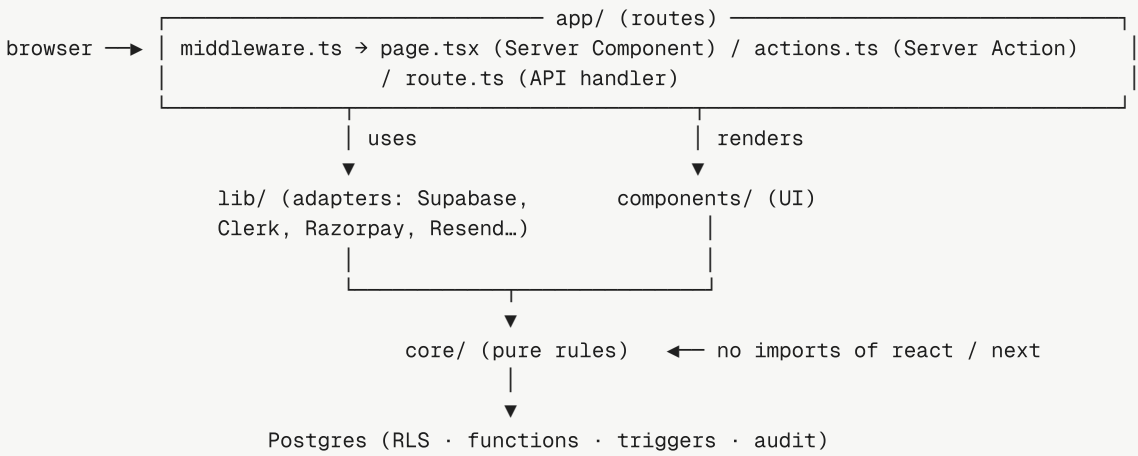


System context

2.2 Repository layout

Folder	Responsibility	Notes
`app/`	Routes: pages, layouts, server actions, API route handlers	Route groups: `(marketing)` website, `(auth)` sign-in, `(app)` portal; `api/` handlers; `p/[token]` public 3D preview links
`components/`	React components	`ui/` design-system primitives, `shell/` portal frame and navigation, `marketing/`, `domain/` (order and money widgets), `work/`, `pwa/`, `preview/` (STL viewer), `seo/`
`core/`	Framework-free business logic	`auth` (route protection), `money`, `orders`, `payments` (webhook signature), `files` (sanitization gate, download gate), `notifications`, `email`, `sizes`, `weights`, `time`, `support`, `ids` and more
`lib/`	Adapters and server utilities	`supabase/` (user client, service-role client, live client), `clerk/`, `razorpay/`, `email/`, `push/`, `files/`, `pdf/`, `certificates/`, `payouts/`, `work/`, `rateLimit.ts`
`config/`	Reading and validating configuration	`supabase.ts` (bucket names, signed URL lifetime), `clerk.ts`, `cron.ts`, `email.ts`, `payments.ts`, `push.ts`, `auth-redirects.ts`
`content/`	Public claims	`content/claims.ts`: the single list of confirmed marketing claims
`db/`	Database source of truth	`migrations/` (67 ordered files), `policies/` (46 RLS and grant files), `ops/` (one-off operational SQL, never applied automatically)
`tests/`	Tests	`db/` (RLS, functions and invariants against a throwaway Postgres), `config/`, `boundary/`, `e2e/` (Playwright journeys), `design-qa/` (screenshots and accessibility)
`scripts/`	Operational scripts	migrations, secret scan, payment and payout verification, email and push senders, OG image, Clerk account copy
`docs/`	Documentation	decisions, specs, runbooks, guides and this handbook
`public/`	Static files	icons, service worker (`sw.js`), Open Graph image

2.3 Layers and dependency direction

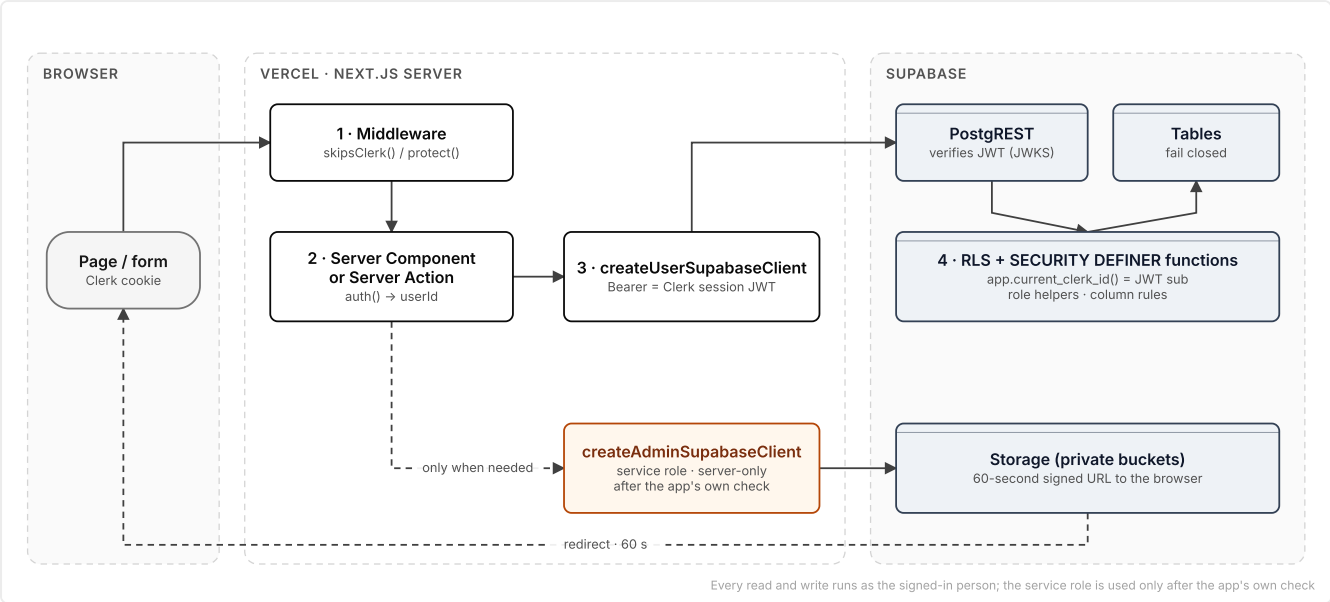


Dependencies point inwards. `core/` imports nothing from the outer layers. `lib/` may use `core/`. `app/` and `components/` may use both. Server-only modules (`lib/supabase/admin.ts`, `lib/work/team.ts` and others) import `server-only`, so the build fails if one is ever pulled into browser code.

2.4 Runtime topology

Runtime	What runs there	Notes
Browser	React client components, the installable PWA shell (`public/sw.js`), the STL 3D viewer (three.js), the Clerk sign-in widgets	Holds only the Clerk session cookie and public keys (`NEXT_PUBLIC_*`)
Vercel edge	`middleware.ts`	Decides whether Clerk's middleware runs (`skipsClerk`), and protects portal paths (`isProtectedPath`)
Vercel serverless functions (Node.js)	Server Components, Server Actions, API route handlers	Region is set next to the database for speed (see `docs/PERFORMANCE.md`)
Supabase	Postgres 15+, PostgREST (Data API), Storage, Realtime, optionally pg_cron	The only stateful component
Third parties	Clerk, Razorpay, Resend, push services, GitHub Actions	See chapter 11

2.5 How a request flows



Request flow and trust boundaries

1. **Middleware** (`middleware.ts`, rules in `core/auth/session.ts`). For a GET or HEAD of a public website page, or of a path that is not an app route at all, Clerk is skipped. That gives search engines a clean 200 or 404 and no redirect. Everything under an app segment (`APP_ROUTE_SEGMENTS`) goes through Clerk, and paths in `PROTECTED_PREFIXES` require a signed-in user. A test walks `app/` and fails if a route sits under a segment the middleware does not know (`tests/config/route-inventory.test.ts`).
2. **Server Component or Server Action.** It reads the user with Clerk's `auth()`. Pages that are only for the team call `requireTeam`, `requireAdmin` or `requireFinance`. These are convenience gates; the real check is in step 4.
3. **The user-scoped Supabase client** (`lib/supabase/server.ts`, `createUserSupabaseClient`) attaches the person's Clerk session token as `Authorization: Bearer …`. It uses the public anon key, never the service role, so every query runs with exactly that person's rights.
4. **Postgres** verifies the token (third-party auth, Clerk's JWKS). `app.current_clerk_id()` returns its `sub`, and RLS policies and SECURITY DEFINER functions allow or refuse each row.
5. **The service role is used only when needed, and only after the caller's access has been proven in step 4.** Typical cases are writing a sanitised file to a private bucket, or minting a 60-second signed download link (`lib/supabase/admin.ts`). Chapter 9 lists every file that does this.

2.6 Where decisions are made

Decision	Made in	Enforced in
Is this person signed in?	Clerk (session)	Middleware and handlers (`auth()`)
What role does this person have?	`users.role`, `user_roles` (database)	`app.current_user_role()`, `app.has_role()`, `app.is_admin()`
Can they see this order, file or message?	RLS policies and visibility functions such as `visible_orders()` and `can_read_thread()`	Postgres
Is this order move legal, and by whom?	`public.order_transitions` (23 rows: from, to, role)	Transition functions

Decision	Made in	Enforced in
Is money conserved?	Ledger rules	`escrow_ledger` triggers (`app.enforce_escrow_conservation`)
Is this file acceptable?	`core/files/sanitizationGate.ts` (type, magic bytes, metadata)	Upload actions
May this file be downloaded now?	`core/files/downloadGate.ts` plus RLS	`lib/files/access.ts`
What does the client pay, and what does the designer earn?	`core/money`, quote functions, admin settings	Money columns are read only through functions that show them by role (policies 0048)

2.7 Key architectural decisions (and why)

Decision	Reason	Trade-off
Authorisation in Postgres (RLS and functions), not in the API layer	One place that every path must pass; a forgotten table fails closed; tests can prove it	Business logic split between SQL and TypeScript; needs SQL skills
Clerk for identity, Supabase for data, joined by third-party auth	Best-in-class sign-in UX with no passwords stored by us; database sees a verified `sub`	Two vendors; Clerk user ids are the primary key of `users` (see the Clerk production switch in `docs/CLERK-PRODUCTION.md`)
Opaque ids and separate identity tables	Makes anonymity structural rather than a matter of discipline	Admin screens must call identity functions, and those reads are logged
Append-only audit log and escrow ledger	Disputes, payouts and compliance need trustworthy history	Corrections are new rows, never edits
Monolith on serverless	Fast iteration for a small team; one deploy	Long jobs need care (timeouts); the scheduler is external
Scheduled work driven by GitHub Actions every 5 minutes	Vercel Hobby allows a cron only once a day	Best-effort timing; moving to Vercel Pro restores `vercel.json` crons
Files stored privately, downloaded via 60-second signed links	IP protection; links cannot be passed around	Each download costs one server round trip

2.8 Cross-cutting concerns

- **Configuration:** read and validated in `config/`. `/api/health` reports which groups are configured, without revealing values.
- **Errors:** user-facing errors are friendly (`core/errors`, `lib/refusal.ts`). Database refusals are translated, never shown raw to clients.
- **Time:** stored in UTC and shown in the viewer's time zone (a device cookie; `core/time`). The live-support hours are computed in US Eastern time with daylight saving (`core/support`).
- **Money:** integers only, cents for USD and paise for INR (`core/money`, `lib/money.ts`).
- **Accessibility and design:** tokens in `app/globals.css`, components in `components/ui`. Axe checks run in the design-QA workflow.
- **Performance budgets:** `docs/PERFORMANCE.md`, measured by `tests/e2e/zz-speed.e2e.ts`. Lighthouse runs in CI.

CHAPTER 3

Founder’s technical education

This chapter explains, in plain language, the ideas you need to understand your own platform, make decisions about it and talk to developers with confidence. No programming knowledge is assumed. Technical words are explained the first time they appear; all of them are also in the glossary.

3.1 The building blocks, as a shop

Think of The CAD Pillar as a jewelry workshop with a shopfront.

Part of the shop	Technical name	What it is	Your provider
Shop window and reception	**Website** (frontend)	The pages people see in a browser	Built in **Next.js**, hosted on **Vercel**
The back office where staff work	**Server** (backend)	Code that runs on computers you rent, not on the visitor’s phone. It does things the visitor must not be able to fake.	Also Next.js on Vercel (“serverless functions”)
The filing cabinet	**Database**	Where every order, message, payment and person is recorded	**Supabase** (a managed **Postgres** database)
The vault for drawings	**Storage**	Where CAD files and pictures are kept	Supabase Storage, in **private buckets**
The doorman who checks ID	**Authentication**	Proving who someone is (sign-in)	**Clerk**
The rule book on who may open which drawer	**Authorisation**	Deciding what a signed-in person may do	Written into the database as **row-level security**
The cashier	**Payments**	Taking card or UPI payments, paying designers	**Razorpay**
The post room	**Email**	Order updates and receipts	**Resend** (sends), plus your Google Workspace mailbox
The workshop’s master copy of the blueprints	**Code repository**	Every line of the software and its history	**GitHub** (`simran313/GetCAD`)

3.2 How a page reaches someone’s screen

1. A client types `thecadpillar.com` or opens the installed app on their phone.
2. **DNS**, the internet’s address book (managed in Vercel, with the domain registered at GoDaddy), tells the phone which computers serve that name.
3. **Vercel** runs the Next.js code and builds the page. If the page needs data, the server asks **Supabase**, sending along proof of who the visitor is.
4. The **database checks its rules** and returns only the rows that person may see.
5. The page arrives in the browser, encrypted end to end (**HTTPS**, the padlock).

3.3 Sign-in: who are you?

When someone signs in, Clerk checks their email code or their Google account. It then gives the browser a **session token**: a short-lived, tamper-proof digital pass (a **JWT**, JSON Web Token) that says "this is user `user_3Hd…`". Every request carries that pass. Supabase checks the pass's signature with Clerk's public key. A forged or expired pass is refused.

Why this matters to you: nobody at TCP ever stores passwords. Clerk does, and it specialises in it. On 13 November 2026 the site moved from Clerk's *development* setup to its *production* setup. That is why every account received a new id (`docs/CLERK-PRODUCTION.md`).

3.4 Permissions: what may you do?

The most important security decision in the platform is **where the rules live**. They are in the database itself, as **row-level security (RLS)**. Every table in the filing cabinet has a lock, and the database checks every single row against a rule before handing it over:

- A client sees only their own orders.
- A designer sees only jobs assigned to them, and never the client's name or the price.
- A pod lead sees their own pod's work and pay, and never what the client paid.
- Only admins see identities and bank details, and each such read is written to a permanent log.

Because the rules are in the database, a mistake in a screen cannot leak data. The screen simply receives nothing it is not allowed to see. If a developer adds a new table and forgets to write a rule, the table is **closed by default**.

3.5 Money: escrow in software

When a client pays, the money is recorded as **held** in an **escrow ledger**: a list of entries that can only be added to, never changed. The ledger has an automatic rule (a **trigger**): no payout or refund may take out more than was paid in. When the order is closed, Finance **releases** it in three parts: the designer's pay (in rupees), the pod's pay, and TCP's commission. Payouts go to verified bank accounts through **Razorpay Route**. If a bank transfer bounces, a **reversal** entry puts the money back in escrow, so it is never lost.

The client cannot mark their own order paid. Only a message from Razorpay that carries Razorpay's secret signature can do that, or an admin recording a manual payment.

3.6 Files and your intellectual property

CAD files are your clients' designs and your designers' work. The platform protects them in five ways:

1. **Checked on the way in:** a file must really be the type it claims to be. A program renamed `.stl` is refused.
2. **Cleaned:** hidden information such as author names, GPS data in photos and software details is removed.
3. **Kept private:** the storage buckets are not public. There is no web address that opens a file.
4. **Released per request:** each time someone downloads a file, the server checks they are allowed *right now*. A client, for example, only gets final files after approving. The server then creates a link that works for **60 seconds**.
5. **Fingerprinted:** each delivered file gets a **SHA-256 fingerprint**, a unique code that changes if even one byte changes. This backs the spec guarantee: if a client edits a file, the fingerprint no longer matches.

3.7 The audit log: your black box recorder

Every important event is appended to the **audit log**: a state change, a payment, a payout, a QC decision, or someone opening a designer's identity documents. Each entry includes a **hash** (a fingerprint) of the previous entry, forming a chain. If anyone altered an old entry, the chain would break and `audit.verify_chain()` would report where. Nobody, not even the platform's own scripts, can edit or delete entries.

3.8 How changes reach the live site

1. A developer (or Claude) writes code on a **branch**: a private copy of the code where changes are made without touching the live site.
2. They open a **pull request (PR)**: a request to merge the branch into `main`, the master copy.
3. **Continuous integration (CI)** runs automatically on GitHub. It type-checks the code, runs 1,201 automated tests against a throwaway database, scans for leaked secrets, and runs browser journeys through every role.
4. Vercel builds a **preview** of the branch at its own temporary address.
5. **You review and merge.** Vercel then deploys `main` to `thecadpillar.com` within minutes.
6. **Database changes (migrations) are applied by you**, separately and deliberately. CI and Vercel never change the production database.

Rolling back a bad release is instant: in Vercel, pick the last good deployment and choose *Promote to Production*. Database changes do not roll back automatically. That is why they are tested against a throwaway database first.

3.9 Secrets and keys

Some settings are **secrets**: the database password, the Supabase service-role key, the Razorpay secrets, the Clerk secret key and the cron secret. They live only in Vercel's environment-variable settings (and in GitHub Actions secrets for the scheduler). Rules to live by:

- Never paste a secret into chat, email, a document or a screenshot.
- Anything starting with `NEXT_PUBLIC_` is **not** secret: it is visible to every visitor by design.
- If a secret may have been exposed, **rotate** it: create a new one, update Vercel, redeploy, then revoke the old one.

3.10 Backups and recovery

Your data lives in one Supabase project. Recovery after a mistake or an outage depends on the plan: paid plans keep daily backups and can offer **point-in-time recovery**, meaning a restore to any second within the retention window. At audit time the organisation showed the **Free** plan. Chapter 14 explains why upgrading is the single most valuable resilience step.

3.11 Questions to ask any developer

- "Which tables does this change touch, and what are their RLS rules?"
- "Does this add a SECURITY DEFINER function? How does it check the caller?"
- "Does it use the service role? What check happens before it?"
- "Is there a migration? Has it been tested on a throwaway database? How do we undo it?"
- "Could this show a client a designer's identity, or a designer a price?"
- "What do the tests prove, and did CI pass?"

CHAPTER 4

Frontend and backend

This chapter explains how the user interface is built and served, and how the server side is organised: what runs in the browser, what runs on Vercel, and the patterns every new screen and action should follow.

4.1 Technology stack

Layer	Technology	Version (package.json)	Used for
Framework	Next.js App Router	15.5.19 (`^15.3.0`)	Routing, Server Components, Server Actions, route handlers, middleware
UI library	React	19.1	Components
Language	TypeScript (strict)	5.7	All application code
Styling	Tailwind CSS with design tokens	3.4	Utility classes; tokens in `app/globals.css`
Primitives	Radix UI (dialog, dropdown, tabs, tooltip, accordion), cmdk	pinned	Accessible menus, dialogs and the command palette
Forms	react-hook-form + zod	7.81 / 4.4	Client-side forms and server-side validation
Motion	motion	14.1	Small transitions
Icons and fonts	lucide-react, Geist (self-hosted woff2 in `app/fonts`)		Brand typography
Sign-in UI	`@clerk/nextjs`	6.39	Sign-in and sign-up pages, session, `auth()`
Data	`@supabase/supabase-js`	2.108	Database (PostgREST), Storage and Realtime
3D	three.js	0.169	STL preview in the browser (`components/preview/stl-viewer.tsx`)
CAD files	rhino3dm	8.32	Reading `.3dm` files and exploding block instances (`lib/files/strip3dm.ts`)
Documents	@react-pdf/renderer, pdf-lib	4.9 / 1.17	Summary sheet, QC certificate, weight sheet, pod report; cleaning uploaded PDFs
Images	sharp	0.34.5	Technical-sheet panels, render composition, PDF images
Push	web-push	3.6	Phone notifications (VAPID)

4.2 Frontend

4.2.1 Route groups and screens

The `app/` folder is the URL map. Folders in brackets, such as `(marketing)` , group routes without changing the address.

Group	Pages	Audience
`(marketing)`	`/`, `/jewelry-cad`, `/how-it-works`, `/pricing`, `/guarantee`, `/quality`, `/faq`, `/contact`, `/apply-designer`, `/privacy`, `/terms`, `/results`	Everyone; no sign-in check on page views
`(auth)`	`/sign-in`, `/sign-up` (Clerk components)	Everyone
`(app)` client	`/home`, `/orders`, `/orders/new`, `/orders/[id]` and its brief, review, pay, claim and sent pages, `/messages`, `/files`, `/wallet`, `/account`	Clients (also used by the team where relevant)
`(app)` designer	`/work`, `/work/[id]`, `/work/[id]/checklist`, `/work/[id]/review`, `/work/earnings`, `/materials`, `/tools`, `/onboarding/designer`, `/settings/payouts`	Designers
`(app)` pod lead (QC)	`/qc`, `/qc/[id]`, `/qc/orders`, `/qc/earnings`, `/pod/board`, `/pod/performance`, `/pod/designers/[id]`	Pod leads
`(app)` admin	`/admin`, plus `access`, `access/pods`, `applications`, `billing`, `clients`, `credits`, `designers`, `designers/[id]`, `emails`, `leads`, `new-order`, `orders`, `orders/export`, `payouts`, `payouts/[batch]`, `settings`, `users`	Admins (Sales, Ops, Finance)
Other	`/p/[token]` (public 3D preview for a client's own customer), `/offline` (PWA fallback), `/design-preview` (component gallery)	

The full list, with guards and files, is generated in Appendix C.

4.2.2 Rendering model

- **Server Components by default.** Pages fetch their data on the server with the user-scoped Supabase client and send finished HTML. No secret, no service-role key and no other person's data ever reaches the browser.
- **Client Components only where interaction needs them**, marked `"use client"`: forms, chat composer, STL viewer, drawing on images, the render composer, the notification bell and the PWA helpers.
- **Mutations are Server Actions.** A form posts to a function marked `"use server"`. That function runs on Vercel, re-checks the caller and calls a database function. Afterwards `revalidatePath` refreshes the affected pages. Next.js also checks the request's Origin header, which protects actions against cross-site request forgery.
- **Freshness.** `next.config.mjs` keeps a page seen in the last 30 seconds in the browser's memory (`staleTimes`), so tabs switch instantly. Any server action clears it. Order and chat pages poll a small "stamp" endpoint (`/api/orders/[id]/stamp`, `/api/inbox/stamp`) to learn about changes. Live chat additionally listens to a private Realtime channel that carries only a "changed" signal, never content (`lib/supabase/live.ts`, migration `0052_chat_live.sql`).

4.2.3 Design system

- Tokens (colours, surfaces, radii, the `--chrome-top` and `--chrome-bottom` safe-area offsets) are in `app/globals.css`. The brand is black and white, with near-black marks for live and selected states (DECISIONS §1).
- Components live in `components/ui` (buttons, inputs, cards, badges, tables, dialogs) and `components/shell` (portal header, navigation per role, notification centre). Every component is shown in `/design-preview`.

- The page-by-page record is `docs/redesign/BEFORE-AFTER.md`. Screenshots of every route are captured by the design-QA workflow.

4.2.4 Progressive Web App

The portal installs to a phone's home screen (`app/manifest.ts`, `public/sw.js`, `components/pwa/*`). The service worker offers an offline page and handles push notifications. Fixed bars respect the iPhone status bar, Dynamic Island and home indicator through `--chrome-top` and `--chrome-bottom`; `tests/config/mobile-chrome.test.ts` forbids hard-coded pixel offsets.

4.3 Backend

4.3.1 Building blocks

Kind	Where	Count	Pattern
Server Components (pages)	`app/**/page.tsx`	73	`auth()` → user client → read through RLS or visibility functions → render
Server Actions	`app/**/actions.ts`, `lib/client/standingActions.ts`	92 in 31 files	`auth()` → user client → `supabase.rpc("…")` → `revalidatePath` → return a friendly result or `Refusal`
Route handlers	`app/api/**/route.ts`	20	File downloads, PDFs, stamps, health, cron, webhook (see chapter 7)
Middleware	`middleware.ts`	1	Clerk on app routes; skipped for public page views
Database functions	`db/migrations`	285	Business operations and visibility rules; 254 SECURITY DEFINER

4.3.2 The standard server action

```ts
"use server";
import { auth } from "@clerk/nextjs/server";
import { revalidatePath } from "next/cache";
import { createUserSupabaseClient } from "@/lib/supabase/server";
import { Refusal } from "@/lib/refusal";

export async function confirmOrderAction(orderId: string) {
  const { userId } = await auth();
  if (!userId) throw new Refusal("Sign in first.");          // convenience check
  const supabase = await createUserSupabaseClient();          // runs as this person
  const { error } = await supabase.rpc("confirm_order", { p_order_id: orderId });
  if (error) throw new Refusal(error.message);                // the database decided
  revalidatePath(`/orders/${orderId}`);
}
```

Illustrative. It mirrors the shape of the real actions (for example `app/(app)/orders/actions.ts`); function names differ per feature.

The important property: **the action does not decide whether the person may confirm the order. The database function does.** The action only translates the outcome for the screen.

4.3.3 When the service role is used

`createAdminSupabaseClient()` (`lib/supabase/admin.ts`) bypasses RLS. It is imported only on the server (`server-only`) and used for four things:

1. Writing sanitised bytes to a private bucket after the upload has been authorised.
2. Minting short-lived signed upload or download URLs after the user-scoped client has proven access.
3. Trusted system work with no user: the Razorpay webhook, the cron senders, payout execution and public statistics.
4. Reading a public preview by a hashed token (`/p/[token]`).

Chapter 9 lists the 36 files that use it, and how each is guarded.

4.3.4 Generated documents

Document	Built by	Served at
Summary sheet (order, images, stones, weights, timeline)	`lib/pdf/summarySheet.tsx`	`/api/orders/[id]/summary`
QC certificate and spec sheet	`lib/pdf/certificate.tsx`, `lib/certificates/issue.ts`	`/api/orders/[id]/certificate`
Weight sheet (10K, 14K, 18K, Pt950, Ag925 grams)	`lib/pdf/weightSheet.tsx`	`/api/orders/[id]/weight-sheet`
Pod performance report	`lib/pdf/podReport.tsx`	`/api/pod/report`
Technical sheet panel	`lib/files/techSheet.tsx` (sharp)	joined under the uploaded technical image

4.3.5 Validation

- **Browser:** react-hook-form with zod schemas give instant feedback.
- **Server:** actions validate again (zod and explicit checks). Database functions validate a third time (enums, CHECK constraints, function guards). The database is authoritative.
- **Files:** `core/files/sanitizationGate.ts` (extension, size, magic bytes, metadata) plus format-specific cleaners: `lib/files/cleanPdf.ts` and `lib/files/strip3dm.ts`.

CHAPTER 5

Database schema, relationships and row-level security

The database is the heart of the platform and its security boundary. This chapter explains how it is organised, how the tables relate, how row-level security (RLS) and privileged functions work together, and how to change the schema safely. The exhaustive, machine-generated listing of every table, column, constraint, policy, trigger and function is in Appendix A and Appendix B.

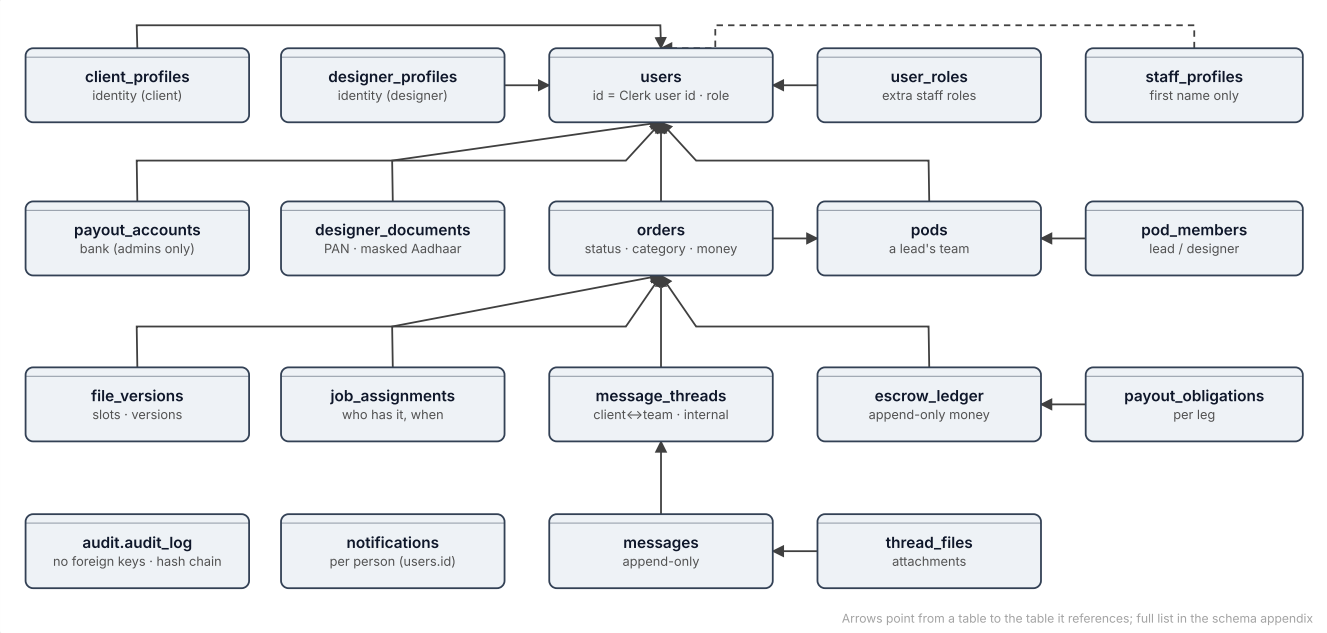
5.1 Schemas

Schema	Purpose	Reachable by signed-in users?
`public`	Business tables and the functions the application calls (through PostgREST, the Data API)	Yes, subject to RLS and grants
`app`	Internal helpers: identity (`current_clerk_id`, `current_user_role`, `has_role`, `is_admin`, `is_staff`), visibility (`can_read_thread`, `order_in_my_pods`, `is_assigned_designer`), triggers, notification and email queueing	Used *inside* policies and functions. It must not be exposed through the Data API (see SEC-06).
`audit`	The hash-chained audit log, and the functions that append to it and verify it	No direct access. `anon` and `authenticated` have no usage on the schema.

5.2 Table inventory (69 tables)

Domain	Tables
Identity and access	`users`, `user_roles`, `client_profiles`, `designer_profiles`, `staff_profiles`, `client_accounts`, `client_managers`, `pods`, `pod_members`, `designer_rates`
Designer onboarding and compliance	`designer_applications`, `agreement_documents`, `agreement_acceptances`, `designer_documents`, `payout_accounts`
Orders and briefs	`orders`, `order_transitions`, `order_specs`, `order_spec_accents`, `order_reference_images`, `order_reference_pins`, `order_screenshots`, `order_stone_certificates`, `client_master_instructions`, `job_categories`, `price_bands`
Work and QC	`job_assignments`, `job_pauses`, `job_alerts`, `job_escalations`, `job_volumes`, `job_weights`, `checklist_items`, `order_checks`, `qc_measurements`, `revision_requests`, `pod_materials`
Files and deliverables	`file_versions`, `deliverables`, `deliverable_slots`, `deliverable_files`, `certificates`, `preview_links`, `spec_claims`, `spec_waivers`
Messaging	`message_threads`, `thread_participants`, `messages`, `thread_files`, `thread_reads`
Money	`payment_intents`, `escrow_ledger`, `payout_obligations`, `payouts`, `client_credits`, `disputes`, `sla_events`

Domain	Tables
Notifications and email	`notifications`, `push_subscriptions`, `email_outbox`, `email_preferences`
Marketing and website	`marketing_leads`, `marketing_events`, `marketing_attribution`, `public_stats`, `rate_limit_events`
Configuration	`app_settings`, `feature_flags`
Audit	`audit.audit_log`



Core data model

5.3 Keys and identity

- **`users.id` is the Clerk user id** (for example `user_3Hd…`). It is the `sub` claim of the session token, so `app.current_clerk_id()` can match rows directly. That is why the move from Clerk development to production required re-keying every reference (`db/ops/clerk-id-remap.sql`, `docs/CLERK-PRODUCTION.md`).
- **Business ids are opaque.** Orders and most rows use random ids generated by nanoid (`core/ids`) or `gen_random_uuid()`, never sequences, so ids cannot be guessed or counted. Human-facing references (job codes such as `RN-C-1001`, client reference codes) are separate columns.
- **Identity is separated from activity.** Names, emails, companies, bank details and documents sit only in `client_profiles`, `designer_profiles`, `payout_accounts` and `designer_documents`. Orders, threads and files carry only ids and role labels.
- **Roles.** `users.role` holds the primary role. Staff can hold extra roles in `user_roles`. Enum `role`: `CLIENT`, `DESIGNER`, `QC`, `SALES`, `OPS`, `FINANCE`, `ADMIN`, `SUPER_ADMIN`. An “admin” in the business sense holds Sales, Ops and Finance (`app.is_admin()`). A pod lead is a `QC` user who leads a pod (`pod_members.member_role = 'POD_LEAD'`).

5.4 Row-level security: how it is applied

Every one of the 69 tables has RLS enabled. The model has three layers.

1. **Deny by default.** With RLS on and no policy for a role, the table returns nothing and accepts nothing. Many sensitive tables deliberately have *no* policy for `anon` or `authenticated`. They can only be reached through SECURITY DEFINER functions that check the caller (Appendix A shows "Policies: none").
2. **Policies for simple, direct reads and writes.** Forty-three policies cover cases such as "a person reads their own notifications" and "a client reads their own orders". Policies call helpers in `app` (for example `app.current_clerk_id()`, `app.can_read_thread(id)`, `app.order_in_my_pods(id)`).
3. **SECURITY DEFINER functions for everything else.** There are 254. Each runs with its owner's rights, checks the caller explicitly (raising `not authenticated`, `only the client who owns an order may …`, and so on) and pins `search_path` (all 254 do), so a caller cannot trick it with a look-alike table. Examples: `visible_orders()` (an order with the money columns shown by role), `confirm_order()`, `assign_designer()`, `qc_decide()`, `designer_document_open()` (admins only, and writes an audit row), `payout_account_full()` (admins only, logged).

Column-level rules. Some columns are hidden from roles that may see the row. The order's price and payments are read only through functions that show them by who is asking, so pod leads and designers never see what the client paid (policies `0048_money_visibility_rls.sql`, migration `0065_money_visibility_rates.sql`).

5.4.1 Policy examples

The precise text of every policy is in Appendix A. Typical shapes:

```
-- A person reads only their own notifications. (db/policies)
CREATE POLICY notifications_read_own ON public.notifications
  FOR SELECT TO authenticated
  USING (user_id = app.current_clerk_id());

-- Thread messages are readable only by people the thread admits.
CREATE POLICY messages_read ON public.messages
  FOR SELECT TO authenticated
  USING (app.can_read_thread(thread_id));
```

These two are the exact policies in production (names and expressions as read from the database built from `db/policies/`). The `orders` table shows how several narrow policies combine; a row is visible if *any* of them allows it:

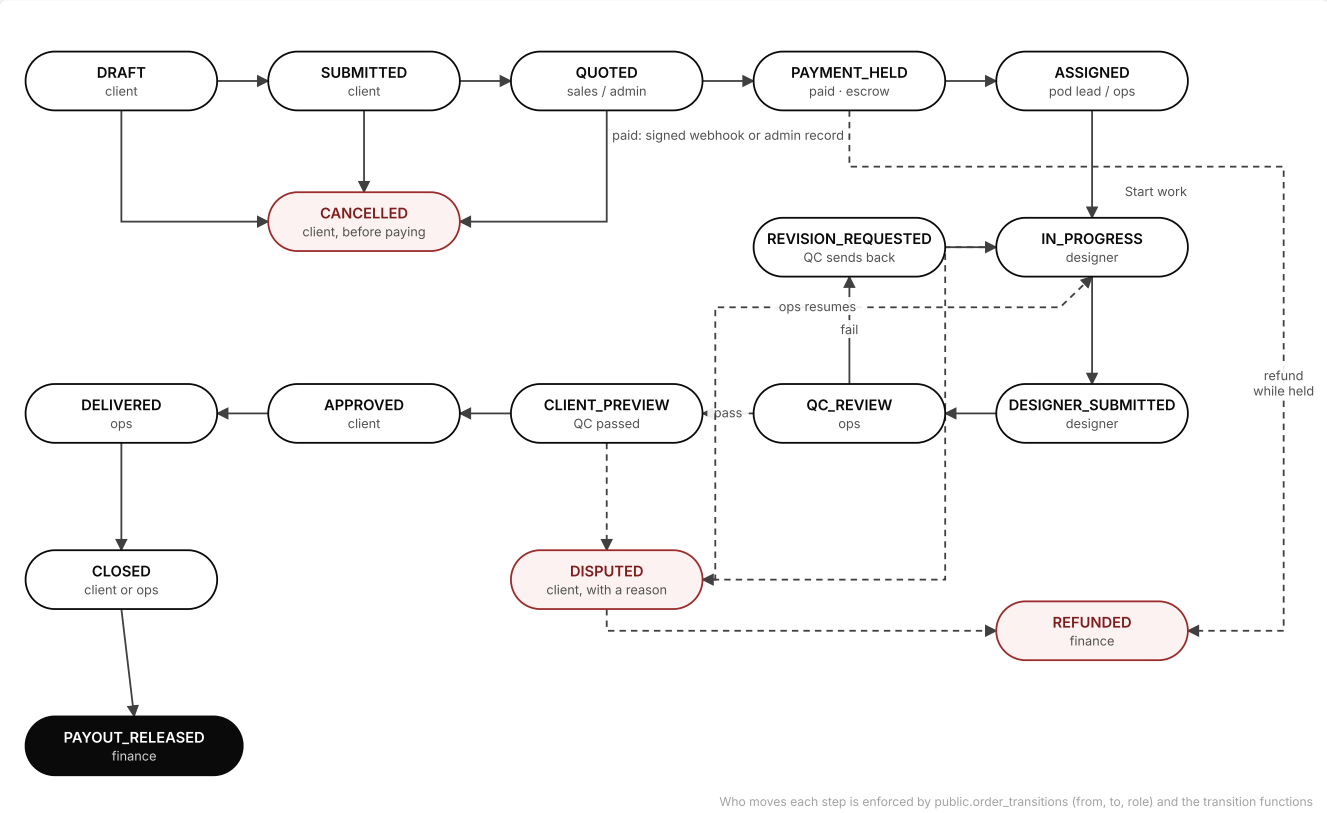
Policy on `orders`	Who it admits
`orders_client_select`	The client who owns the order (`client_id = app.current_clerk_id()`)
`orders_pod_lead_select`	A pod lead, for orders in their pods (`pod_id = ANY (app.current_pod_ids())`)
`orders_qc_reviewer_select`	The QC reviewer recorded on the order
`orders_staff_select`	Staff whose role may act on the order's current status (`order_transitions`, staff scope, excluding QC)
`orders_admin_select`	Admins (`app.is_admin()`)

Designers read their jobs through functions such as `my_work()`, never through `orders` directly, so a designer cannot see the client's columns.

5.5 Invariants enforced by the database

Invariant	Mechanism	Source
Only legal order moves, by the right role	`order_transitions` table (23 rows) checked by the transition functions	`0035_order_model.sql` and later
A client cannot fund their own order	Funding only through `confirm_payment` (the signed webhook path) or an admin recording a manual payment	Payment migrations, `app/api/webhooks/razorpay`
Quote parts sum to the total	CHECK and function validation	Quote migrations (`0040_quote_sla.sql`)
Money is conserved per order	Trigger `app.enforce_escrow_conservation` on `escrow_ledger`	Escrow migrations
Ledger, messages, agreement acceptances and certificates are immutable	`BEFORE UPDATE OR DELETE` triggers that raise	e.g. `messages_no_update`, `escrow_ledger_no_delete`, `certificates_immutable`
Audit history cannot be rewritten	Append-only triggers plus a hash chain (`audit.verify_chain()`)	`0006_audit_log.sql`
A reviewer cannot pass their own work	Check inside `qc_decide`	QC migrations
Work only goes to vetted designers	`app.designer_is_assignable()` (active, with the current agreement signed)	`0009_designer_onboarding_gate.sql`
A payout is paid once	Unique constraint per obligation, plus idempotency keys	Payout migrations

5.6 The order state machine



Order state machine

`public.order_transitions` lists every legal move with the role that may make it:

From	To	Actor role
DRAFT	SUBMITTED, CANCELLED	CLIENT
SUBMITTED	QUOTED	SALES
SUBMITTED	CANCELLED	CLIENT
QUOTED	PAYMENT_HELD, CANCELLED	CLIENT (payment is confirmed by the signed webhook or an admin)
PAYMENT_HELD	ASSIGNED	OPS
PAYMENT_HELD	REFUNDED	FINANCE
ASSIGNED	IN_PROGRESS	DESIGNER
IN_PROGRESS	DESIGNER_SUBMITTED	DESIGNER
IN_PROGRESS	DISPUTED	CLIENT
DESIGNER_SUBMITTED	QC_REVIEW	OPS
QC_REVIEW	REVISION_REQUESTED, CLIENT_PREVIEW	QC
REVISION_REQUESTED	IN_PROGRESS	DESIGNER
CLIENT_PREVIEW	APPROVED, DISPUTED	CLIENT
APPROVED	DELIVERED	OPS
DELIVERED	CLOSED	CLIENT or OPS
CLOSED	PAYOUT_RELEASED	FINANCE
DISPUTED	IN_PROGRESS	OPS
DISPUTED	REFUNDED	FINANCE

Later decisions layered business steps on top of these states without adding states: - the admin confirms the order and chooses its pod (DECISIONS §11), and the pod lead assigns; - reassignment and withdrawal of a job (`reassign_job`, migration `0066_one_designer_withdraw.sql`); - the SLA clock, which starts when an order is both paid and confirmed.

5.7 Triggers you should know

Trigger family	What it does
`orders_*` (11 triggers)	Assign references, route to the only pod, create threads, start the first-look clock, gate on specs and weights, reprice on assignment, close pauses, queue client emails and marketing events, open payouts on close
`audit_log_chain`, `audit_log_fanout`	Compute each entry's hash; fan entries out to notifications
`*_no_update`, `*_no_delete`, `certificates_immutable`	Make history tables append-only
`escrow_ledger_conservation`, `escrow_ledger_hold_paid`	Keep money conserved and mark orders paid
`messages_signal_thread`, `thread_files_signal_thread`, `thread_reads_signal_thread`	Send the "thread changed" Realtime signal

5.8 Changing the schema safely

1. **Write a new file.** Never edit an applied migration. Name it with the next number, for example `db/migrations/0067_short_name.sql`. Put RLS policies and grants for new tables in `db/policies/` (also numbered).
2. **New table checklist:** `ALTER TABLE … ENABLE ROW LEVEL SECURITY`; policies for exactly the roles that need direct access, or none if access is only through functions; no grant to `anon` unless the table is public by design; opaque ids; integer money; an audit write for every meaningful change.
3. **New SECURITY DEFINER function checklist:** `SET search_path = ''` and schema-qualified names; check `app.current_clerk_id()` and the role first; refuse with a clear message; `REVOKE EXECUTE … FROM PUBLIC, anon` and `GRANT` only to the roles that call it; append to the audit log.
4. **Test it** with `npm run test:local` (throwaway Postgres on `127.0.0.1:5433`). Add a test in `tests/db/` that proves both the allowed and the refused paths.
5. **Apply in production deliberately.** The owner runs `npm run db:apply` with `DATABASE_URL` pointing at production, or pastes the file into the Supabase SQL editor. `scripts/apply-migrations.mjs` records applied files in `public.schema_migrations`, and `--status` shows what is pending. CI and Vercel never touch the production database. The test harness refuses any non-local database (`tests/helpers/db.ts`, `assertDestroyable`), because a production database was once dropped by a test run in a shell where `DATABASE_URL` was still set.
6. **One-off operational SQL** (data fixes, account moves) goes in `db/ops/` with a dry-run mode. It is never applied automatically.

5.9 Regenerating the reference appendices

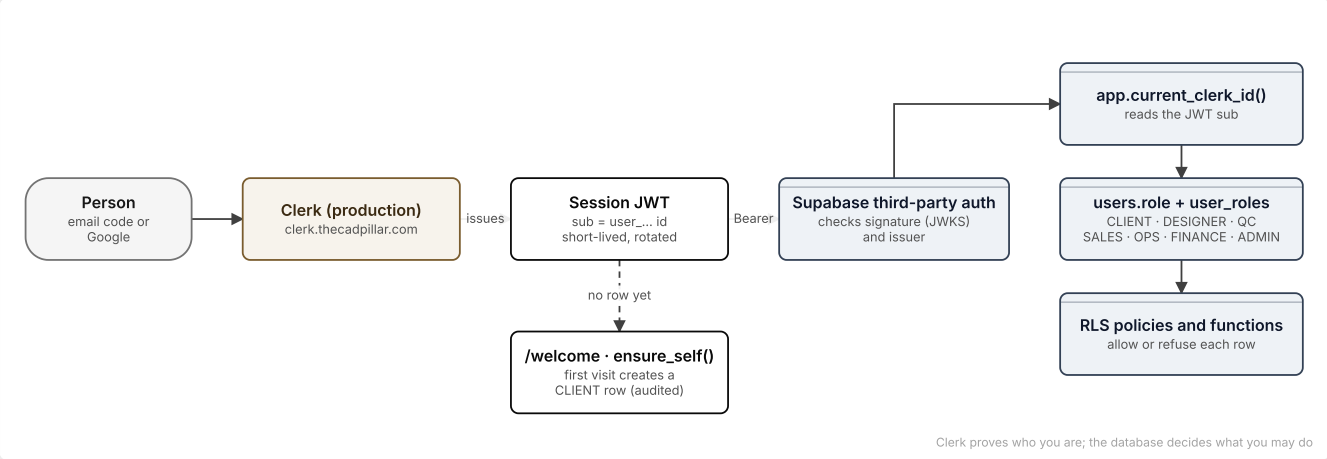
```
npm run test:db:up        # or any local Postgres on 127.0.0.1:5433
DATABASE_URL=postgres://postgres@127.0.0.1:5433/postgres \
  node docs/document-generation/extract-reference.mjs
```

The script builds a scratch database from `db/migrations` and `db/policies`, reads the catalogue, writes Appendices A to E, and drops the scratch database. It refuses any host that is not local.

CHAPTER 6

Authentication, authorisation and the role matrix

Authentication answers “who are you?” and is handled by Clerk. **Authorisation** answers “what may you do?” and is handled by the database. This chapter covers both, the bridge between them, how roles are granted, and the full capability matrix.



Authentication and identity bridge

6.1 Authentication (Clerk)

Item	Setting (13 November 2026)
Instance	**Production**, on `clerk.thecadpillar.com` (DNS CNAMEs in Vercel DNS). The development instance remains for CI and browser tests only.
Sign-in methods	Email verification code; Google (custom OAuth credentials owned by the company); password via “Forgot password” for accounts that set one
Session	Clerk session cookie on `thecadpillar.com`; short-lived session token (JWT) refreshed by Clerk
Keys	`NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY` (public), `CLERK_SECRET_KEY` (server only), `CLERK_JWT_ISSUER = https://clerk.thecadpillar.com`. Preview deployments use the development keys.
Pages	`/sign-in`, `/sign-up` (`app/(auth)`), wrapped in `components/auth`
Passwords stored by TCP	None

6.1.1 Middleware

`middleware.ts` wraps Clerk’s middleware with three rules from `core/auth/session.ts`:

1. `skipsClerk(method, path)`: a GET or HEAD of a public website page (`PUBLIC_WEBSITE_PAGES`), or of a path outside every app segment (`APP_ROUTE_SEGMENTS`), skips Clerk entirely. Search engines get a clean 200 or 404, and the 404 page renders. Form posts always go through Clerk.

2. `isProtectedPath(path)`: paths under `PROTECTED_PREFIXES` (`/home`, `/welcome`, `/messages`, `/wallet`, `/files`, `/dashboard`, `/orders`, `/account`, `/admin`, `/settings`, `/onboarding`, `/work`, `/qc`, `/pod`) call `auth.protect()`. A signed-out browser is redirected to `/sign-in`.
3. A first-touch marketing cookie (HTTP-only, SameSite=Lax, Secure in production) records where a visitor first came from.

`/api/webhooks/razorpay` is deliberately not protected: Razorpay carries no session, so its security is the HMAC signature. The middleware is the first layer only. Every page, action and handler checks again.

6.2 The identity bridge (Clerk to Supabase)

1. The server asks Clerk for the current session token (`auth().getToken()`).
2. `createUserSupabaseClient()` sends it as `Authorization: Bearer …`, alongside the public anon key.
3. Supabase's **third-party auth** integration (configured with Clerk's domain) verifies the signature against Clerk's published keys (JWKS) and checks the issuer.
4. In SQL, `app.current_clerk_id()` returns the token's `sub`. Every policy and function uses it.
5. On a first visit, `/welcome` calls `ensure_self()`. That creates a `users` row with role `CLIENT` and writes an audit entry. Staff and designers are created by admins instead (see below).

If the Supabase third-party auth domain does not match the Clerk instance that issued the token, every signed-in read returns nothing. This is the first thing to check after any Clerk change (`docs/LAUNCH.md`, `docs/CLERK-PRODUCTION.md`).

6.3 Authorisation model

- **Roles** (`role` enum): `CLIENT`, `DESIGNER`, `QC` (pod lead), `SALES`, `OPS`, `FINANCE`, `ADMIN`, `SUPER_ADMIN`. `users.role` holds one primary role; `user_roles` adds staff roles.
- **Admin** in the business sense means Sales + Ops + Finance (`app.is_admin()`). Some admin-only actions check all three explicitly (for example `sendTestEmailsAction`).
- **Scope.** A pod lead's reach is their pods (`app.current_pod_ids()`, `app.order_in_my_pods()`). A designer's reach is jobs assigned to them (`app.is_assigned_designer()`, `my_work()`). A client's reach is their own orders and their client account.
- **Enforcement.** Row-level security plus SECURITY DEFINER functions (chapter 5). UI gates such as `requireTeam`, `requireAdmin`, `requireFinance` and role-based navigation are conveniences, not controls.

6.3.1 How roles are granted

Who	How	Function
Client	Self sign-up (`/welcome` → `ensure_self`), or an admin or pod lead registers them while placing an order on their behalf	`ensure_self`, `register_client`
Designer	An admin adds them by "email - Name" on Admin → Designers. They sign in with an emailed code, apply in-app, sign the agreement, and become `ACTIVE`.	`register_designer`, `apply_as_designer`, `accept_designer_agreement`
Pod lead	An admin sets the role to pod lead on Admin → Access. That creates the lead's pod ("Masud's pod"). Designers are placed in pods.	`set_user_role`, `create_pod`, `set_pod_member`, `remove_pod_member`

Who	How	Function
Admin, Sales, Ops, Finance	An admin grants the role on Admin → Access	`set_user_role`
Suspension	An admin sets the user's status (typing `suspend` to confirm)	`set_user_status`

Every grant, status change and identity lookup is written to the audit log (`record_identity_lookup` for admin reads of names).

6.4 Role and capability matrix

Legend: ✔ allowed · ◖ limited (see note) · ✖ refused by the database.

Capability	Client	Designer	Pod lead (QC)	Admin (Sales/Ops/Finance)
Create and submit an order, write the brief	✔ own	✖	◖ on behalf of a client	✔ on behalf of a client
See the order's price and payments	✔ own	✖	✖ (1.25)	✔
Quote an order	✖	✖	✖	✔ (Sales)
Pay (card or UPI)	✔ own	✖	✖	◖ records manual payments
Confirm the order and choose its pod	✖	✖	◖ confirm in own pod	✔
Assign or move a job to a designer	✖	✖	✔ own pod (reason once started)	◖ folded exception
See the brief and references	✔ own	✔ assigned jobs (no client identity)	✔ own pod	✔
Upload deliverables (3DM, STL, render, technical sheet)	✖	✔ assigned job	◖	◖
Designer self-check (9 lines)	✖	✔	✖	✖
QC decision (pass or send back)	✖	✖ (never on own work)	✔ own pod	✔
Approve, request a revision or dispute	✔ own	✖	✖	✖
Download final files	◖ after approval	✔ own versions	✔ own pod	✔
Message the client thread	✔ ("Your team")	✖ never	✔	✔
Message the internal thread	✖	✔	✔	✔
Forward a message between client and designer	✖	✖	✔	✔
See own pay per job	✖	✔	✔ designer and pod pay	✔
See or change a designer's rate	✖	✖	◖ see only	✔

Capability	Client	Designer	Pod lead (QC)	Admin (Sales/Ops/Finance)
See identities (names, emails, companies)	✖	✖	◖ first names of staff	✔ (logged)
See or edit bank details and identity documents	✖	◖ own last four digits	✖	✔ (every view logged)
Release payouts and refunds	✖	✖	✖	✔ (Finance)
Decide SLA credits	✖	✖	✖	✔
Settings, feature flags, emails	✖	✖	✖	✔
Create preview links for the client's customer	✔ own	✖	✖	✔
Pod materials	✖	✔ read own pod	✔ add and remove in own pod	✔
Calculators (`/tools`)	✖	✔	✔	✔

The matrix follows DECISIONS §2, §5, §10, §11 and §14. Where a cell says ✖, the refusal is enforced in the database.

6.5 Security properties worth knowing

- **No identity leaves the wall through the UI or the API.** The browser test `tests/e2e/anonymity.e2e.ts` crawls every client and designer screen and fails on any counterparty name or email.
- **Sessions are not trusted for roles.** The role is read from the database on every request, so removing a role takes effect on the next request; nobody has to sign out.
- **Server actions** are protected by Next.js's Origin check. `NEXT_SERVER_ACTION_ALLOWED_ORIGINS` lists any extra allowed origins.
- **Sign-out** reloads the page so nothing is kept in the browser's router cache for the next person on the device (`components/auth/sign-out`).

CHAPTER 7

API and Server Action documentation

The CAD Pillar has no public, third-party API. Its server interface has three parts:

- **Server Actions**, which the portal's forms call;
- **Route handlers** under `/api`, for downloads, documents, polling, health, scheduled jobs and the payment webhook;
- **Database functions**, which the server calls through Supabase's Data API (PostgREST) on behalf of the signed-in person.

This chapter documents each part, its contract and its security. The generated inventories are Appendix C (routes) and Appendix D (actions).

7.1 Conventions

Convention	Rule
Authentication	Every non-public handler and action starts with Clerk `auth()`. With no user it returns `401 unauthenticated` (handlers) or throws `Refusal` (actions).
Authorisation	Decided by the database: a SECURITY DEFINER function or an RLS-filtered read through the user-scoped client. Handlers answer `404 not found` for both "doesn't exist" and "not yours", so ids cannot be probed.
Caching	Personal responses send `Cache-Control: no-store`.
Downloads	Never stream from a public URL. Authorise, then redirect to a Supabase signed URL valid for **60 seconds** (`SIGNED_URL_TTL_SECONDS`). The STL stream endpoint proxies bytes through the same gate.
Errors	Friendly text for people (`lib/refusal.ts`, `core/errors`). Database messages are mapped, never shown raw to clients.
Idempotency	Payment and payout operations carry idempotency keys (`razorpay:payment.captured:<id>`, payout keys), so retries are safe.
Cross-site requests	Server Actions get Next.js's Origin check. `POST /api/threads/[id]/read` refuses `sec-fetch-site: cross-site`.

7.2 Route handlers (20)

Method and path	Purpose	Authentication and authorisation	Response
`GET /api/health`	Which configuration groups are set (auth, database, storage, payments, email, push, cron, SEO), without values	Public	`200 {status, configured}`, or `503` naming the missing required group
`GET /api/notifications`	The signed-in person's notifications and unread count	`auth()`; RLS (`notifications_read_own`)	JSON, `no-store`

Method and path	Purpose	Authentication and authorisation	Response
`GET /api/inbox/stamp`	A change stamp for the message list (polling)	`auth()`; `my_conversations()`	`{stamp}`
`GET /api/orders/[id]/stamp`	A change stamp for one order page	`auth()`; `order_activity_stamp()` refuses orders you cannot see	`{stamp}` or 404
`POST /api/threads/[id]/read`	Mark a thread read (receipts and unread counts)	Same-site only; `auth()`; `mark_thread_read()`	204, 401 or 403
`GET /api/files/[versionId]`	Download a file version	`auth()`; `grantedObjectKey()`: RLS read of `file_versions`, `visible_orders()` relation, `fileGrantFor()` rule by state and role	302 to a 60 s signed URL, or 404
`GET /api/files/[versionId]/stream`	Stream an STL to the 3D viewer when storage refuses the browser's read	As above	Bytes, or 404
`GET /api/reference-images/[imageId]`	A brief reference image	`auth()`; RLS on `order_reference_images`	302 to a signed URL, or 404
`GET /api/thread-files/[fileId]`	A chat attachment	`auth()`; `thread_file_for_download()`	302 to a signed URL, or 404
`GET /api/materials/[id]`	A pod material (instructions, charts)	`auth()`; `pod_material_file()` (own pod or admin)	302 to a signed URL, or 404
`GET /api/designer-documents/[id]`	A designer's identity document	`auth()`; `designer_document_open()` (admins only, writes an audit row)	302, or 404 with `no-store`
`GET /api/orders/[id]/summary`	Summary sheet PDF	`auth()`; `summary_sheet_data()` (role- and state-aware)	PDF, or 404
`GET /api/orders/[id]/certificate`	QC certificate and spec sheet PDF	`auth()`; `certificate_for_download()`	302 to a signed URL, or 404
`GET /api/orders/[id]/weight-sheet`	Weight sheet PDF (grams per metal)	`auth()`; `weight_sheet_data()`	PDF, or 404
`GET /api/pod/report`	Pod performance export (Excel, CSV or PDF)	`auth()`; `requireTeam`; the data functions scope to own pod or admin	File, or 404
`GET /api/cron/push`	Send queued push notifications	`Authorization: Bearer <CRON_SECRET>`, compared in constant time; 404 otherwise	Counts
`GET /api/cron/emails`	Send queued emails and run the email sweep	As above	Counts
`POST /api/webhooks/razorpay`	Payment captured or failed; transfer processed or reversed	HMAC-SHA256 of the raw body with `RAZORPAY_WEBHOOK_SECRET`, verified in constant time **before parsing**	200 handled or ignored; 401 invalid signature; 500 to ask Razorpay to retry

Other public endpoints: `/robots.txt`, `/sitemap.xml`, `/manifest.webmanifest` (generated by `app/robots.ts`, `app/sitemap.ts` and `app/manifest.ts`), and `/p/[token]`, the public preview page. That page hashes the token and looks it up with `preview_by_token()`, which checks expiry and revocation and counts views; tokens are 192-bit random.

7.2.1 The Razorpay webhook, step by step

1. Read the **raw body** before parsing anything.
2. Verify `x-razorpay-signature` with HMAC-SHA256 and a constant-time comparison (`core/payments/razorpaySignature.ts`). If it fails, return 401 with no detail.
3. Parse the event and switch on its type.
   - `payment.captured` → `confirm_payment()` with the idempotency key `razorpay:payment.captured:<payment id>`. The ledger gets a HOLD and the order becomes paid.
   - `payment.failed` → `fail_payment_intent()`.
   - Transfer events → `record_payout_result()` with the payout key. A reversal credits escrow back.
4. On a database error, return 500 so Razorpay retries. Idempotency makes the retry harmless.
5. `/api/health` reports `payments: false` if the webhook secret is weak, is a URL, or equals the key secret.

7.2.2 Scheduled endpoints

Vercel Hobby allows a cron only once a day, so `.github/workflows/push-dispatch.yml` calls `/api/cron/push` and `/api/cron/emails` every five minutes with the `CRON_SECRET` bearer token. On Vercel Pro, move the schedule back to `vercel.json` (the instructions are at the top of that workflow file). The SLA sweep (`sla_sweep()`) is separate: it runs every minute inside Postgres through `pg_cron` **if the extension is enabled** in Supabase. That is not verifiable from the code (see the assumptions chapter).

7.3 Server Actions (92 in 31 files)

Server Actions are the write API of the portal. Each is a function in a `"use server"` file, called by a form or button. They are grouped by area below. Appendix D lists every action with the database functions it calls.

Area (files)	Main actions	Database functions
Orders (`app/(app)/orders/actions.ts`)	create order, transition, confirm, assign, QC decide, request revision, dispute and resolve, change orders, checks, measurements, waivers, record a manual payment	`create_order`, `transition_order`, `confirm_order`, `assign_designer`, `qc_decide`, `request_revision`, `raise_dispute`, `resolve_dispute`, `create_change_order`, `mark_check`, `record_qc_measurements`, `add_spec_waiver`, `mark_payment_received`, `set_job_volume`, `post_message`
Brief (`orders/[id]/brief/*`)	write the brief and stones, upload reference images, pins, certificates	`set_order_accents`, `add_reference_image`, `set_reference_pins`, `set_primary_reference`
Files (`orders/fileActions.ts`)	signed upload URL per slot, store the version, technical-sheet source	`job_slots`, `add_slot_file`, `add_file_version`, `set_tech_source`
Payments and payouts (`paymentActions.ts`, `payoutActions.ts`)	open a Razorpay payment intent; open and execute payouts (Finance)	`open_payment_intent`, `open_payouts_for_order`, `claim_payouts`, `record_payout_result`

Area (files)	Main actions	Database functions
Chat (`orders/chat/actions.ts` , `messages/actions.ts`)	post messages and files, forward between client and designer, drawings on images, voice notes	thread and message functions
Preview links (`previewActions.ts`)	create and revoke a client's preview link (192-bit token, hashed at rest)	`create_preview_link`
Work (`work/actions.ts`)	start, pause and resume, designer checks, weights, final review, submit	`my_work` and the work functions
Pod (`pod/actions.ts`)	reassign, withdraw, escalate	`reassign_job` and the escalation functions
Materials (`materials/actions.ts`)	add and remove pod materials	material functions
Admin (`admin/**/actions.ts`)	access and roles, designers (register, rates, bank details, documents), applications, leads, settings, credits, emails, new order on behalf of a client	`set_user_role` , `set_user_status` , `create_pod` , `set_pod_member` , `register_designer` , `set_designer_rate` , `set_payout_account_for` , `add_designer_document` , `remove_designer_document` , `register_client` , `create_order_on_behalf` , `submit_order_on_behalf`
Account (`account/*` , `lib/client/standingActions.ts`)	time zone, email preferences, push subscription, master instructions, logo	`my_email_preferences` , `my_master_instructions` , push functions
Onboarding (`onboarding/designer/actions.ts` , `settings/payouts/actions.ts`)	apply in-app, accept the agreement, payout details	`apply_as_designer` , `accept_designer_agreement` , `upsert_payout_account`
Website (`(marketing)/*/actions.ts`)	contact form, designer application, brief lead	`submit_marketing_lead` , `submit_designer_application` , `start_brief_lead` , `check_rate_limit` (per network address, stored as a salted one-way hash)

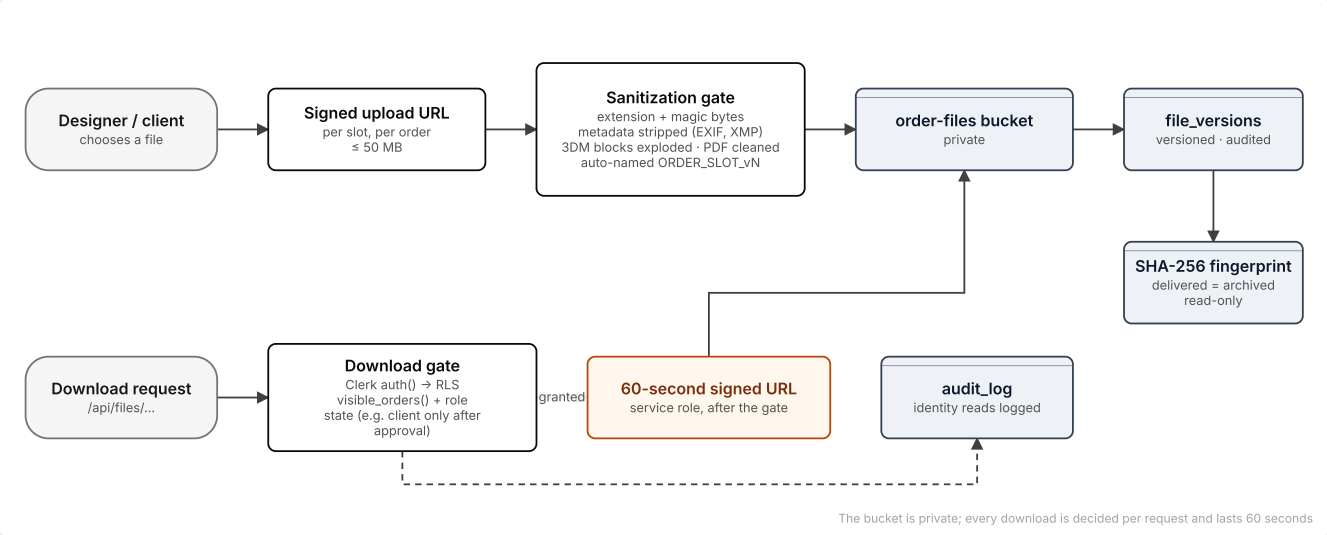
7.3.1 Writing a new action: checklist

- [ ] `"use server"` at the top of the file. Export only async functions.
- [ ] `const { userId } = await auth()` , and refuse if empty.
- [ ] Use `createUserSupabaseClient()` . Use the service role only after an authorised read has proven access, and say why in a comment.
- [ ] Do the permission check in a database function: SECURITY DEFINER, `search_path` pinned, caller checked, audited.
- [ ] Validate input (zod) and cap sizes; reject unexpected fields.
- [ ] `revalidatePath` for the pages whose data changed.
- [ ] Return friendly messages; never leak database errors or identities.
- [ ] Add a test in `tests/db/` for allowed and refused callers, and an e2e step if it is a user journey.

CHAPTER 8

File storage and CAD intellectual-property protection

CAD files are the product. They are the client's confidential designs and the designer's work, and the escrow model depends on nobody receiving final files before approval. This chapter describes where files live, how they enter, how they are cleaned, who may take them out and when, and how deliveries are proven unchanged.



File pipeline and IP protection

8.1 Where files live

Bucket (Supabase Storage)	Visibility	Contents	Configured in
`order-files`	Private	Deliverables (3DM, STL, render, technical sheet), reference images, stone certificates, screenshots, chat attachments, certificates, client logos, pod materials	`config/supabase.ts` (`ORDER_FILES_BUCKET`)
`designer-application-files`	Private	Portfolio files sent with public designer applications	`DESIGNER_APPLICATION_FILES_BUCKET`
`designer-documents`	Private	PAN card, masked Aadhaar, bank proof (admins only)	`DESIGNER_DOCUMENTS_BUCKET`, created by `0061_designers_by_team.sql`

No bucket is public. There is no stable web address for any file. The browser never receives the service-role key. Object keys are opaque. Delivered files are renamed `{ORDER}_{SLOT}_v{n}.{ext}` (DECISIONS §9), so original filenames, which often name the studio, never travel.

8.2 The way in

1. **Choose a slot.** Required deliverable slots: `3DM`, `STL`, `RENDER` and `TECH` (DECISIONS §9; older ten-view slots stay readable on old versions). Accepted types per slot are listed in `core/files/slotTypes.ts`. Images are PNG or JPEG only; PDF is accepted only where a document is expected, such as stone certificates.
2. **Signed upload URL.** The server checks the caller may upload to this order and slot (`my_work`, `job_slots`). It then mints a single-use signed upload URL (`createSignedUploadUrl`), so large files go straight to storage without passing through a serverless function. Deliverables may be up to **50 MB** (`MAX_DELIVERABLE_BYTES`); other uploads go through the general gate at **25 MiB** (`MAX_UPLOAD_BYTES`).
3. **Sanitization gate** (`core/files/sanitizationGate.ts`), a pure function covered by unit tests:
   - the extension must be allowed for the slot;
   - the size must be within the limit;
   - **the magic bytes must match the declared type**: PNG `89 50 4E 47`, JPEG `FF D8 FF`, PDF `%PDF`, STEP `ISO-10303-21`, 3DM header; STL is recognised by its exact binary length or its ASCII form;
   - the object is renamed to an opaque key.
4. **Metadata stripping** (`core/files/metadataStripper.ts`). The container is rebuilt from its structural parts, keeping an **allow-list** of what is needed to render or manufacture. Anything unknown is dropped by default. That covers JPEG EXIF (camera, GPS, author), PNG text and time chunks (`tEXt`, `iTXt`, `zTXt`, `eXIf`, `tIME`), and STEP header author and organisation fields.
5. **Format-specific cleaners:**
   - **3DM** (`lib/files/strip3dm.ts`, rhino3dm): the model is *rebuilt*. A new model receives only units, tolerances, layers (name and colour) and geometry. Creator, last editor, application and licence, notes, comments, object names, user text and plug-in data are left behind by construction. **Blocks are exploded** into geometry, so block names cannot leak.
   - **PDF** (`lib/files/cleanPdf.ts`, pdf-lib): only the pages are kept. Document info, XMP, comments, attachments, scripts and earlier revisions are removed (DECISIONS 1.14).
   - **Images** passed to `sharp` (render composition, technical-sheet panels) have already passed the signature check, so only verified PNG and JPEG reach the image decoder.
6. **Record the version.** `add_slot_file` / `add_file_version` writes a `file_versions` row (versioned, audited, uploader party recorded by trigger `app.set_uploader_party`).

8.3 The way out

Every download is decided per request (`lib/files/access.ts`):

1. Clerk `auth()`: no session gives 401.
2. Read the `file_versions` row with the **user-scoped client**. If RLS hides it, the response is 404.
3. Read the order through `visible_orders()`. It returns nothing for orders the caller cannot see, plus the caller's *relation* to the order (client, designer, staff).
4. `fileGrantFor()` (`core/files/downloadGate.ts`) decides **which artefact** this relation may receive in this order state. The rule the escrow model rests on: **before approval, the client sees the four render views, the gold weight chart and the diamond details only. Approval releases the STL, the 3DM and the summary sheet.** There is no "degraded model" before approval, because there is no model at all.
5. Only then does the server mint a **60-second** signed URL with the service role, and redirect.

Other file families follow the same pattern through dedicated functions: `thread_file_for_download`, `pod_material_file`, `designer_document_open` (admins only, audited), `certificate_for_download` and RLS on `order_reference_images`.

8.4 Proof of delivery and the spec guarantee

- **Fingerprints.** Delivered files are hashed with SHA-256 and archived read-only (DECISIONS §9). The QC certificate (`certificates`, immutable by trigger) records the fingerprints. It doubles as the signed spec sheet: minimum wall, prong and stone-seat sizes, ring size, mesh tolerance, no open edges.
- **Spec claims.** If a delivered file breaks its own spec sheet, TCP fixes it free and refunds the CAD fee. A claim on a file whose fingerprint no longer matches (an edited file) is excluded, as are casting failures where the archived file meets spec, recorded client waivers, and claims after 30 days (DECISIONS §7; tables `spec_claims`, `spec_waivers`).
- **Preview links** (`/p/[token]`). A client can send their own customer a branded 3D view built from the STL, with renders alongside. The token is 192 bits from `randomBytes(24)` and is stored only as a hash. Links expire, can be revoked and count views. The page never shows the designer, TCP's client, the price or any download.

8.5 Identity documents (designer compliance)

- Entered and seen by **admins only**, never pod leads or other designers (DECISIONS §14).
- Aadhaar is kept as the **masked** copy (last four digits).
- Full bank details are shown only on request (`payout_account_full`), and **every full view and every document open is written to the audit log**.
- Removing a document deletes its stored file.

8.6 Threats considered

Threat	Control
Client downloads final files before approval or payment	Download gate by artefact and state; RLS; 60-second links
Designer learns the client's identity from a file	Metadata stripping, 3DM rebuild, opaque names, chat role labels
Client learns the designer's studio from a file	As above; STEP and PDF author fields removed
Malicious file disguised as an image or model	Magic-byte checks; type allow-lists per slot; size caps; images re-encoded
Leaked link reused later	Signed URLs live 60 seconds; preview tokens expire and can be revoked
Bulk scraping of the bucket	Private buckets; no listing for clients; keys are opaque
Edited file blamed on TCP	SHA-256 fingerprints on the certificate; fingerprint mismatch excludes the claim
Insider over-reading identity documents	Admins only; every view audited in a hash-chained log

Residual risks (dependency advisories in the image libraries, upload abuse by self-registered accounts) are in the security register, chapter 9.

CHAPTER 9

Security audit and vulnerability register

Audit date: 13 November 2026. **Code revision:** the commit named on the cover (`main` after pull request #137). **Classification:** this is the *sanitised* register, suitable for the repository. Precise evidence, verification queries and exploit preconditions for the items marked † are in a **restricted report** held by the owner outside the repository (see "Handling of sensitive findings" below).

9.1 Summary

No critical vulnerability was found in The CAD Pillar's own code. The application's security design is strong and consistent: the database is the authorisation boundary, every table has RLS, every privileged function checks its caller and pins its search path, money and history are append-only, and files are cleaned and released per request.

The open risks are in three places: - **third-party dependencies** with published advisories, including the Next.js framework; - **infrastructure configuration** outside the code: database plan and backups, credential hygiene, email domain protection and hosting plan; - a few **defence-in-depth gaps** worth closing.

Severity	Count	IDs
Critical	0	(in TCP's code; one dependency advisory rated Critical upstream is assessed High in context, SEC-01)
High	3	SEC-01, SEC-02, SEC-03†
Medium	5	SEC-04, SEC-05, SEC-06†, SEC-07, SEC-08
Low	5	SEC-09, SEC-10, SEC-11, SEC-12, SEC-13
Informational	2	SEC-14, SEC-15

Severity uses impact × likelihood in TCP's context (the deployment on Vercel, which features are used, and who can reach the code path). The upstream rating is given where it differs.

9.2 Vulnerability register

9.2.1 SEC-01 HIGH Next.js 15.5.19 has published security advisories

- **Area:** framework dependency (`next`).
- **Finding:** `npm audit` (13 November 2026) lists advisories affecting the installed version:
  - remote code execution in the Image Optimization API with crafted SVG or AVIF images (upstream **Critical**; fixed in 15.5.24 and 15.5.28);
  - denial of service through Server Actions (High);
  - server-side request forgery in rewrites and in Server Actions on custom servers (High);
  - cache poisoning and confusion (Moderate);
  - disclosure of internal Server Function endpoints (Moderate).
- **Context:** the application does not use `next/image`, sets no remote image patterns, and runs on Vercel on Linux, so the image-optimiser remote code execution path is unlikely to be reachable (hence High, not Critical). The Server Actions denial of service *does* apply: the portal is built on Server Actions.

- **Recommendation:** upgrade `next` (and `eslint-config-next`) to the latest 15.5.x (≥ 15.5.28) in its own pull request. Run `npm run ci` and the e2e suite, check the preview deployment, then merge. Add a weekly dependency check (Dependabot or `npm audit` in CI).
- **Effort:** small. **Owner:** developer.

9.2.2 SEC-02 HIGH Production database resilience (plan, backups, recovery)

- **Area:** infrastructure (Supabase).
- **Finding:** the Supabase organisation hosting the production project showed the **Free** plan at audit time. Free projects have limited or no restorable backups, no point-in-time recovery, and are paused after a period of inactivity. Payments, escrow records, designer identity records and all order history live in this one project.
- **Recommendation:** move to a paid plan with daily backups. Enable point-in-time recovery when revenue justifies it. Rehearse a restore into a scratch project (chapter 14). Confirm the project region and that `pg_cron` is enabled (SEC-14).
- **Effort:** small (billing change plus one restore drill). **Owner:** founder.

9.2.3 SEC-03† HIGH Credential rotation pending; an unused hosting project retains production database credentials

- **Area:** secrets management.
- **Finding:** the owner's own checklist calls for a database password reset that has not yet been done. Separately, an older, unused Vercel project still stores production database connection strings. A copy of a production credential outside the live system widens who could obtain it (anyone with access to that project, its logs or its future deployments).
- **Recommendation:** reset the database password in Supabase. Update only the places that need it (local `.env.local` for `db:apply`; the live app does not use `DATABASE_URL`). Delete the unused Vercel project, or at least its variables. Review who has access to the Vercel team and the GitHub repository.
- **Effort:** small. **Owner:** founder.

9.2.4 SEC-04 MEDIUM Image library `sharp` 0.34.5 bundles vulnerable decoders

- **Area:** dependency (`sharp`, libvips, librsvg, libheif).
- **Finding:** advisories rated High upstream cover the bundled libvips, librsvg and libheif (fixed in `sharp` ≥ 0.35.5). The server runs `sharp` on user-supplied images (technical-sheet panels, render composition, PDF images).
- **Context:** uploads are signature-checked as PNG or JPEG before decoding, so the SVG (librsvg) and HEIF (libheif) decoders should not be reached. Rated Medium.
- **Recommendation:** upgrade `sharp` to ≥ 0.35.5 together with SEC-01. Keep `limitInputPixels` set wherever `sharp` reads untrusted input.
- **Effort:** small.

9.2.5 SEC-05 MEDIUM Validation library `zod` 4.4.3: memory exhaustion on oversized collections

- **Area:** dependency (`zod`).
- **Finding:** advisory (High upstream): validating very large constrained arrays or records can exhaust memory (fixed in 4.6.6). Server Actions validate user input with zod.
- **Recommendation:** upgrade `zod` to ≥ 4.6.6. Keep explicit length caps before `.parse` on any user-supplied collection.
- **Effort:** small.

9.2.6 SEC-06† MEDIUM · CONDITIONAL Write-capable internal helper functions are executable by every database role

- **Area:** database grants.
- **Finding:** PostgreSQL grants `EXECUTE` on new functions to `PUBLIC` by default. Several internal `app` schema helpers were never revoked, including two that **write** (queue an email, create a notification). The `app` schema must be usable by `anon` and `authenticated` because RLS policies call its helpers. So if the `app` schema were ever added to Supabase's exposed Data API schemas, these helpers would become directly callable with the public anon key. Today they are reachable only from inside other database functions and policies, provided the `app` schema is **not** exposed. That could not be verified from the code.
- **Recommendation:** in a migration, `REVOKE EXECUTE … FROM PUBLIC, anon, authenticated` on every `app` function that writes or reveals another person's state. Callers are SECURITY DEFINER functions running as the owner, so nothing breaks. Then confirm in Supabase → Project Settings → Data API that the exposed schemas are only `public` and `graphql_public`. Add a test in `tests/db/` that fails if any non-read-only `app` function is executable by `anon` or `authenticated`.
- **Effort:** small (one migration and one test). A remediation migration has been drafted and tested: with it applied, all 58 database test files (607 tests) pass. It is in the restricted annex.

9.2.7 SEC-07 MEDIUM Email domain authentication is incomplete

- **Area:** DNS and email (Google Workspace, transactional email).
- **Finding (public DNS, 13 November 2026):**
  - **no DMARC record** at `_dmarc.thecadpillar.com`;
  - a DKIM public key is published at the domain apex instead of under its selector (for example `google._domainkey`), so receivers cannot use it;
  - SPF authorises Google only (`include:_spf.google.com ~all`);
  - no DNS records were found for the transactional email provider on the domain.

  Anyone can send email that claims to be from `@thecadpillar.com`, and legitimate mail is more likely to be filed as spam.
- **Recommendation:**
  1. In Google Admin, generate DKIM and publish it at `google._domainkey`; remove the apex copy.
  2. Verify the transactional sender domain with the email provider; add its SPF and DKIM records, ideally on a subdomain such as `send.thecadpillar.com`.
  3. Publish DMARC: start with `p=none` and reporting, then move to `quarantine`, then `reject`.
- **Effort:** small (DNS changes in Vercel DNS). **Owner:** founder.

9.2.8 SEC-08 MEDIUM Hosting plan and preview-environment isolation

- **Area:** infrastructure (Vercel).
- **Finding:** the team is on Vercel **Hobby**, which Vercel's terms limit to non-commercial use, and which lacks some production features, for example frequent crons, so a GitHub Actions workaround is in use. Preview deployments are built for every branch. Whether they receive the **production** Supabase URL and service-role key could not be verified. If they do, any branch's code runs with full access to production data.
- **Recommendation:** move to Vercel Pro. Give the Preview environment its own Supabase project (seeded test data), and keep production secrets scoped to the Production environment only. Restore crons to `vercel.json`. Turn on Deployment Protection for previews only, never for Production, because the payment webhook must reach it.
- **Effort:** medium. **Owner:** founder plus developer.

9.2.9 SEC-09 LOW Content-Security-Policy covers framing only

- **Finding:** the response headers are good: HSTS (two years), `X-Frame-Options: SAMEORIGIN`, `X-Content-Type-Options: nosniff`, `Referrer-Policy: strict-origin-when-cross-origin`, and a `Permissions-Policy`. But the CSP contains only `frame-ancestors 'self'`. There is no `script-src` or `connect-src` policy to contain a future cross-site scripting bug. React escapes output by default, and the only raw HTML is escaped JSON-LD (`components/seo/json-ld.tsx`), so the risk is low.
- **Recommendation:** add a CSP in report-only mode first: `script-src 'self'` with Clerk and Vercel allowances or nonces; `connect-src` for Supabase, Clerk and Razorpay; `img-src` for Supabase signed URLs. Review the reports, then enforce.

9.2.10 SEC-10 LOW No per-account rate limits or storage quotas for signed-in users

- **Finding:** public forms are rate-limited per network address (salted hash, `check_rate_limit`). Signed-in actions are not rate-limited. Anyone can sign up (an automatic CLIENT account) and upload reference images and attachments within the size caps. Abuse could raise storage and bandwidth costs or fill queues.
- **Recommendation:** per-account limits on uploads and orders (count and bytes per day) in the database functions; alerts on storage growth. Optionally require email verification before first upload (Clerk setting).

9.2.11 SEC-11 LOW Some public functions are executable by `anon`, but refuse anonymous callers

- **Finding:** a handful of `public` brief and reference functions (for example `add_reference_image`, `set_order_accents`) and website-form functions are executable by `anon`. The brief functions check the caller and raise `not authenticated`, so there is no exposure. This is defence in depth only.
- **Recommendation:** revoke `anon` execution on functions that need a signed-in caller. Keep it only on the deliberate public entry points (`submit_marketing_lead`, `submit_designer_application`, `start_brief_lead`, `record_marketing_event`, `check_rate_limit`, `public_stats_latest`).

9.2.12 SEC-12 LOW Build-time and development dependency advisories

- **Finding:** `postcss` 8.4.31 (inside Next.js), `nanoid` 3.3.15 and `source-map-js` 1.2.1 have advisories that affect build tooling rather than runtime requests. A further 12 moderate advisories are in development-only dependencies.
- **Recommendation:** most are resolved by the Next.js upgrade (SEC-01). Re-run `npm audit` afterwards. Track the rest in a dependency pull request.

9.2.13 SEC-13 LOW The development sign-in instance still holds copies of real accounts

- **Finding:** before the production switch, the development Clerk instance held the real staff, designer and client accounts. They remain there (emails and names), next to the test accounts that CI uses.
- **Recommendation:** delete non-test users from the development instance (keep only the `+clerk_test` accounts used by CI). That is data minimisation, in line with the Privacy page.

9.2.14 SEC-14 INFORMATIONAL SLA sweep depends on `pg_cron` being enabled

- **Finding:** `sla_sweep()` flags overdue first looks and creates pending credits every minute **if** `pg_cron` is enabled; migration `0040_quote_sla.sql` schedules it only when the extension exists. Whether it is enabled in production could not be verified from the repository.
- **Recommendation:** in Supabase → Database → Extensions, confirm `pg_cron` is on and `cron.job` lists the sweep. Otherwise enable it and run the `cron.schedule` statement from the migration.

9.2.15 SEC-15 INFORMATIONAL Scheduled jobs rely on best-effort GitHub Actions timing

- **Finding:** push notifications and emails are sent by a GitHub Actions workflow every five minutes. GitHub can delay schedules under load and pauses scheduled workflows on inactive repositories.

- **Recommendation:** resolved by SEC-08 (Vercel Pro crons). Until then, watch the workflow's run history weekly.

9.3 Controls verified as effective

Control	Evidence
RLS on every table	69 of 69 tables have `relrowsecurity = true` (Appendix A)
Privileged functions are hardened	254 SECURITY DEFINER functions, all with a pinned `search_path`; caller checks sampled in brief, file, money and admin functions
No direct anonymous access to the audit log	`anon` and `authenticated` have no `USAGE` on schema `audit`
Service role only on the server, after authorisation	`lib/supabase/admin.ts` imports `server-only`; every route handler that signs a URL first authorises through the user-scoped client or a definer function (table below)
Payment webhook integrity	HMAC-SHA256 over the raw body, constant-time comparison, verified before parsing, idempotency keys; a weak secret is reported by `/api/health`
Scheduled-endpoint authentication	Bearer secret compared in constant time over fixed-width buffers; 404 when wrong
Short-lived downloads	`SIGNED_URL_TTL_SECONDS = 60`
File sanitisation	Magic-byte checks, metadata allow-lists, 3DM rebuild, PDF cleaning (unit-tested)
Preview tokens	192-bit random, stored hashed, expiring, revocable
Anonymity	E2E crawl `tests/e2e/anonymity.e2e.ts`; identity tables reachable only through logged functions
Transport and headers	HSTS (`max-age=63072000`), frame, MIME-type and referrer headers (verified live)
No secrets in the repository or its history	`npm run secret-scan` clean; a full-history pattern scan for Clerk, Razorpay, Supabase JWT, Postgres URL, Resend and private-key formats found nothing; only `.env.example` is tracked
Test isolation from production	The test harness refuses non-local databases (`assertDestroyable`)

9.3.1 Service-role usage inventory

Location	Purpose	Guard before use
`app/api/files/[versionId]` and `/stream`	Sign and stream deliverables	`auth()` + `grantedObjectKey()` (RLS + `visible_orders` + download gate)
`app/api/reference-images/[imageId]`	Sign a reference image	`auth()` + RLS read
`app/api/thread-files/[fileId]`	Sign a chat attachment	`auth()` + `thread_file_for_download()`
`app/api/materials/[id]`	Sign a pod material	`auth()` + `pod_material_file()`
`app/api/designer-documents/[id]`	Sign an identity document	`auth()` + `designer_document_open()` (admins, audited)
`app/api/orders/[id]/certificate` and `/summary`	Sign or build documents	`auth()` + `certificate_for_download()` / `summary_sheet_data()`
`app/api/webhooks/razorpay`	Record payments and payout results	HMAC signature
`app/api/cron/emails` and `/push`	Send queued emails and pushes	Bearer `CRON_SECRET`
`app/p/[token]`	Read a preview by hashed token	Token hash, expiry, revocation

Location	Purpose	Guard before use
`app/(marketing)/results`	Public statistics	Read-only public function
`app/(marketing)/apply-designer/actions.ts`	Store portfolio files	Rate limit + submission function
`app/(app)/orders/fileActions.ts`, `brief/*`, `TeamClaims.tsx`, `work/[id]/review`, `orders/[id]/page.tsx`	Upload and sign order files	`auth()` + `my_work` / `job_slots` / RLS reads
`app/(app)/orders/paymentActions.ts`	Open a payment intent	`auth()` + `visible_orders()`
`app/(app)/orders/payoutActions.ts`, `lib/payouts/execute.ts`	Execute payouts	`requireFinance()`
`app/(app)/admin/*` (designers, emails, new-order, applications)	Documents, test emails, screenshots, portfolios	`requireAdmin` / `requireTeam` / admin-only functions
`app/(app)/account/page.tsx`, `lib/client/standingActions.ts`, `lib/work/*`, `lib/certificates/issue.ts`, `lib/email/dispatch.ts`	Logos, standing instructions, references, certificates, email dispatch	`auth()` + RLS reads, or system-only paths

9.4 Handling of sensitive findings

- The restricted report contains exact evidence: the function names and grants behind SEC-06, the names of the infrastructure items behind SEC-03, the verification queries and the remediation SQL. It is delivered to the owner directly and **must not be committed**. `.gitignore` excludes `docs/security/restricted/` as a safety net.
- Fixes are **not** made in the documentation pull request. Each should be its own pull request: dependency upgrades (SEC-01, 04, 05, 12) together; the grant hardening migration (SEC-06, SEC-11) with its test; infrastructure changes by the owner (SEC-02, 03, 07, 08, 13, 14).
- Re-run this audit after those changes, and at least quarterly. The method is in Audit method and assumptions.

CHAPTER 10

Infrastructure, GitHub, Vercel and deployment workflows

This chapter maps every piece of infrastructure, who owns it, how code travels from a branch to production, and how the database is changed. Settings marked *(verify)* could not be confirmed from the repository; they are listed in the assumptions chapter.

10.1 Infrastructure map

Component	Provider	Identifier	Notes
Domain registration	GoDaddy	`thecadpillar.com`	Nameservers point to Vercel
DNS	Vercel DNS	`ns1.vercel-dns.com`, `ns2.vercel-dns.com`	Records live under Vercel → Team → Domains → `thecadpillar.com` → DNS Records
Web hosting	Vercel	Team project **`tcp`** (`tcp-gamma.vercel.app`); `www` → 308 → apex	Plan: Hobby *(see SEC-08)*. An older, unused project also exists *(see SEC-03)*.
Function region	Vercel	Should be next to the database: Cleveland `cle1` for a Supabase project in Ohio `us-east-2` (`docs/DEPLOY.md` §2b) *(verify)*	Each database round trip costs about 2 ms when co-located, about 200 ms when not
Database, storage, realtime	Supabase	Production project in the organisation “The CAD Pillar”	Plan: Free at audit *(see SEC-02)*; `pg_cron` *(verify, SEC-14)*
Identity	Clerk	Production instance on `clerk.thecadpillar.com`; development instance for CI	Google sign-in with company OAuth credentials (Google Cloud project “The CAD Pillar”)
Payments	Razorpay	Checkout + Route (payouts)	Webhook → `/api/webhooks/razorpay`. Payments are on hold until approved.
Email (transactional)	Resend	`RESEND_API_KEY`, `EMAIL_FROM`	Delivery gated by `EMAIL_DELIVERY` *(see SEC-07)*
Email (people)	Google Workspace	MX `smtp.google.com`	`simran@thecadpillar.com`
Push	Web Push (VAPID)	`NEXT_PUBLIC_VAPID_PUBLIC_KEY`, `VAPID_PRIVATE_KEY`, `VAPID_SUBJECT`	Browser push services
Source code and CI	GitHub	`simran313/GetCAD`	Actions: `ci.yml`, `design-qa.yml`, `push-dispatch.yml`
Search	Google Search Console	Property for `thecadpillar.com`	`NEXT_PUBLIC_GOOGLE_SITE_VERIFICATION`

10.2 Environments

Environment	Where	Clerk	Database	Who uses it
Local development	A developer machine or Codespace	Development keys	A local Postgres for tests; the developer's chosen Supabase project for the running app	Developers
CI	GitHub Actions	Development keys (test users with `+clerk_test`)	Throwaway Postgres service container, plus PostgREST for e2e	Automated
Preview	Vercel, per branch	Development keys	*(verify: should not be production; SEC-08)*	Reviewers
Production	Vercel `tcp` on `thecadpillar.com`	Production keys	Supabase production	Everyone

Configuration variables are listed by name in Appendix E. Values live only in Vercel (by environment), GitHub Actions secrets (`APP_URL` and `CRON_SECRET` for the dispatcher; Clerk development keys for e2e) and developers' untracked `.env.local`.

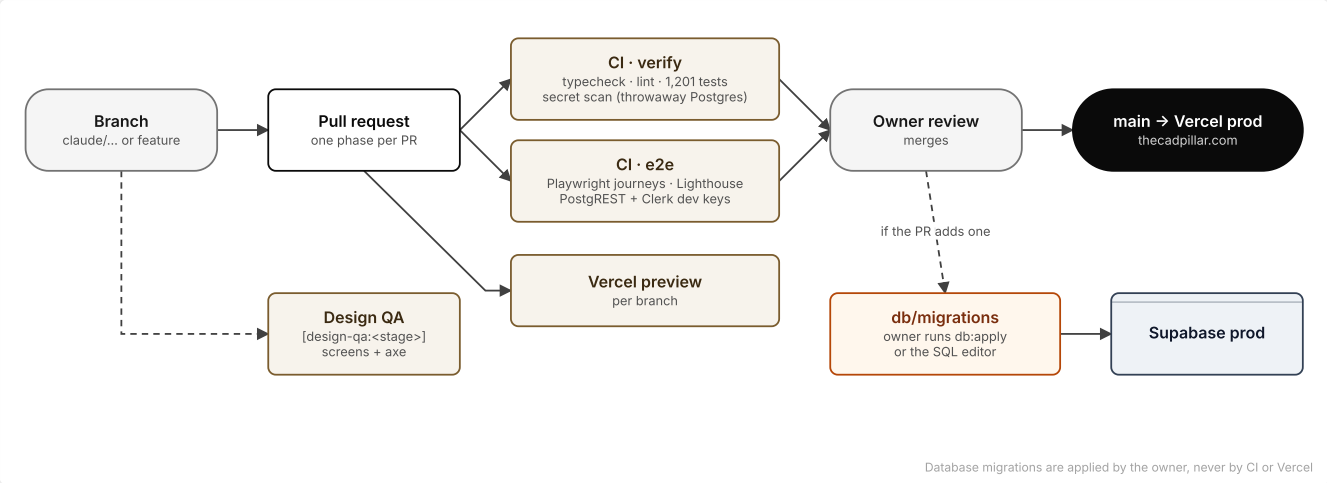
10.3 GitHub

- **Default branch:** `main`, deployed automatically to production by Vercel.
- **Working branches:** `claude/…` for AI-assisted work, plus feature branches. One phase per pull request (`CLAUDE.md`).
- **Pull requests:** CI must be green; the owner reviews and merges. Branch protection rules *(verify)*: recommended are required status checks (`verify`, `e2e`), no force pushes to `main`, and at least one approval.
- **Secrets in the repository:** none (verified). `.env.example` documents names only. `npm run secret-scan` runs in CI.

10.3.1 Workflows

Workflow	Trigger	What it does
`.github/workflows/ci.yml` → job `verify`	Every pull request; pushes to `main`	`npm ci`, type-check (strict), lint (including the `core/` boundary), vitest (1,201 tests against a Postgres 16 service container), secret scan. Target: under 5 minutes.
`ci.yml` → job `e2e`	Same	Prepares a database and Clerk test users, starts PostgREST, builds the app against the test stack and runs the Playwright journeys (17 suites, including the anonymity crawl, money, files, live chat, iPhone layout and speed). Then Lighthouse on home and pricing (mobile), uploading the report.
`.github/workflows/design-qa.yml`	A push to `design/redesign` or `claude/gracious-hopper-amum18` whose commit message has `[design-qa:<stage>]`; manual dispatch	Screenshots every route at several sizes and runs axe. `[design-qa:after]` fails on any serious accessibility finding. Results go to the `design-qa` branch.
`.github/workflows/push-dispatch.yml`	Every 5 minutes (cron) and manual	Calls `/api/cron/push` and `/api/cron/emails` with the `CRON_SECRET` bearer token. It exists because Vercel Hobby crons run at most once a day.

10.4 Delivery pipeline



Delivery pipeline

1. **Branch** from `main`. Make the change, with tests.
2. **Run locally:** `npm run ci` (type-check, lint, tests, secret scan), or `npm run test:local` to start the throwaway Postgres first.
3. **Open a pull request.** CI (`verify`, `e2e`) and a Vercel preview start automatically.
4. **Review.** The owner reviews the diff, the preview and CI, then merges.
5. **Deploy.** Vercel builds `main` and promotes it to `thecadpillar.com` (zero downtime; the previous deployment stays available).
6. **Database migration**, if the pull request adds one. The owner applies it deliberately, before or after the deploy as the pull request states:

   ```
   # from a trusted machine, with DATABASE_URL pointing at production
   npm run db:status          # what is pending (changes nothing)
   npm run db:apply           # applies pending files in order, recording each
   ```

   Or paste the file into the Supabase SQL editor. Never point `npm test` or `npm run ci` at production: the harness drops schemas and refuses non-local hosts for that reason.
7. **Verify:** `/api/health` shows every group true; a quick role walkthrough (chapter 16); Vercel → Logs filtered to errors.

10.4.1 Rollback

What	How	Time
Application	Vercel → Deployments → last good deployment → **Promote to Production**	Seconds
Configuration change	Revert the variable in Vercel, then redeploy	Minutes
Database migration	Forward-only: write a new migration that undoes the change, tested locally. For data loss, restore from backup (chapter 14).	Depends

What	How	Time
Clerk switch	Documented in `docs/CLERK-PRODUCTION.md` ("If something is wrong")	Minutes

10.5 Domains and DNS records (Vercel DNS)

Name	Type	Purpose
`@`	A (Vercel)	Website
`www`	CNAME / redirect	308 to the apex
`@`	MX `smtp.google.com`	Google Workspace mail
`@`	TXT `v=spf1 include:_spf.google.com ~all`	SPF (Google only)
`@`	TXT `google-site-verification=…`	Search Console
`@`	TXT `v=DKIM1 …`	**Misplaced** DKIM key; should be at `google._domainkey` (SEC-07)
`clerk`	CNAME `frontend-api.clerk.services`	Clerk Frontend API
`accounts`	CNAME `accounts.clerk.services`	Clerk account portal
`clkmail`	CNAME `mail.<id>.clerk.services`	Clerk email sending
`clk._domainkey`, `clk2._domainkey`	CNAME `dkim1/2.<id>.clerk.services`	Clerk email DKIM
`_dmarc`	*(missing)*	Add a DMARC policy (SEC-07)

10.6 Observability

Signal	Where
Application errors and logs	Vercel → Project `tcp` → Logs (filter Level: Error); Runtime Logs per deployment
Configuration health	`GET https://thecadpillar.com/api/health`
Database health, slow queries, storage use	Supabase dashboard (Reports, Logs, Storage)
Scheduled jobs	GitHub → Actions → "push-dispatch" run history; the `/api/cron/*` responses carry counts
Payments and payouts	Razorpay dashboard (webhook delivery logs); `npm run verify:payment`, `verify:payout`, `verify:delivery`
Email delivery	Resend dashboard; `email_outbox` rows (status, attempts)
Audit integrity	`SELECT audit.verify_chain();` in the SQL editor
Performance	Lighthouse report artefact in CI; `docs/PERFORMANCE.md`; Vercel Speed Insights *(if enabled)*

There is no external uptime monitor or error-tracking service at audit time. Adding one is in the roadmap (chapter 15).

CHAPTER 11

Third-party integrations

Every outside service the platform depends on: what it does for TCP, how it is connected, which settings it uses, how it fails, and what to do when it does.

11.1 Summary

Service	Role	Direction	Data shared	If it is down
Clerk	Sign-in, sessions, user accounts	App ↔ Clerk	Email, name, sign-in methods	Nobody can sign in; signed-in pages fail. The website still works.
Supabase	Database, file storage, realtime, scheduled SQL	App ↔ Supabase	Everything in the platform	The portal is down; the website mostly works (the `/results` page and forms fail)
Razorpay	Checkout (card, UPI, international); Route payouts	App ↔ Razorpay, webhook in	Amount, order reference, payment ids; payee bank linkage for Route	New payments wait; orders stay unpaid; webhooks retry later
Resend	Transactional email	App → Resend	Recipient email, subject, body	Emails queue in `email_outbox` and send later
Web Push services	Phone and browser notifications	App → push service	Short notification text, identity-free	Pushes delayed or dropped; in-app bell unaffected
GitHub Actions	CI; 5-minute scheduler	GitHub → app	`CRON_SECRET` bearer	CI blocked; push and email delayed until it runs
Vercel	Hosting, DNS, build		Full application runtime	Site down; roll back or wait
Google (OAuth)	“Continue with Google”	Clerk ↔ Google	Email, name, Google id	Use email code instead
Google Workspace	People’s mailboxes			Staff email only
WhatsApp	Support one-tap link (`wa.me`)	Link only, no API	Nothing until the person sends	Phone support

11.2 Clerk

- **Package:** `@clerk/nextjs` (server and client), `@clerk/backend` and `@clerk/testing` (CI, scripts).
- **Settings:** `NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY`, `CLERK_SECRET_KEY`, `CLERK_JWT_ISSUER`, optional `CLERK_JWKS_URL`, `NEXT_PUBLIC_CLERK_SIGN_IN_URL` and `NEXT_PUBLIC_CLERK_SIGN_UP_URL`.
- **Server use:** `auth()` and `getToken()` in pages, actions and handlers. `lib/clerk/*` looks up accounts by email (adding designers and clients) and resolves names for admins (`lib/clerk/identities.ts`, logged via `record_identity_lookup`).
- **Production instance:** created 13 November 2026. The full runbook and record are in `docs/CLERK-PRODUCTION.md`, including the account-copy script (`scripts/clerk-copy-users.mjs`), the id remap (`db/ops/clerk-id-remap.sql`) and the health check (`db/ops/clerk-switch-health.sql`).

- **Supabase link:** Supabase → Authentication → Third-Party Auth → Clerk, set to the production Frontend API domain. Clerk → Integrations → Supabase must be enabled on the production instance.
- **Failure modes:** a wrong issuer or domain makes signed-in reads return empty; a development key on production shows a "Development mode" badge and caps accounts at 100; Google sign-in without custom credentials gives "Access blocked: missing client_id".

11.3 Supabase

- **Package:** `@supabase/supabase-js`.
- **Clients:**
  - `createUserSupabaseClient()`: the anon key plus the person's Clerk token (`lib/supabase/server.ts`);
  - `createAdminSupabaseClient()`: the service role, server only (`lib/supabase/admin.ts`);
  - a live client for Realtime in the browser, which fetches a fresh Clerk token per connection (`lib/supabase/live.ts`).
- **Settings:** `NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY`, `SUPABASE_SERVICE_ROLE_KEY`; `DATABASE_URL` for migrations only (never in Vercel).
- **Storage:** three private buckets (chapter 8). Signed URLs last 60 seconds.
- **Realtime:** private channels per thread; a "changed" signal only, sent by database triggers; channel access governed by policies on `realtime.messages` (migrations `0052_chat_live.sql`, `0053_chat_receipts.sql`); switched on by `NEXT_PUBLIC_CHAT_REALTIME=1`.
- **Scheduled SQL:** `sla_sweep()` every minute through `pg_cron`, if enabled (SEC-14).
- **Failure modes:** project paused (Free plan), wrong third-party auth domain, an exposed schema list that includes `app` (SEC-06), or region far from the Vercel functions (slow pages).

11.4 Razorpay

- **Settings:** `RAZORPAY_KEY_ID`, `RAZORPAY_KEY_SECRET`, `RAZORPAY_WEBHOOK_SECRET` (different from the key secret; checked by `/api/health`).
- **Checkout:** `open_payment_intent()` records an intent, the server creates the Razorpay order, and the client pays in Razorpay's own widget. TCP never sees or stores card numbers.
- **Webhook:** `POST /api/webhooks/razorpay`, HMAC-verified, with these events:
  - `payment.captured` → `confirm_payment`;
  - `payment.failed` → `fail_payment_intent`;
  - transfer processed or reversed → `record_payout_result`.
- **Payouts (Route):** Finance releases a closed order. `open_payouts_for_order` creates obligations. `lib/payouts/execute.ts` claims them (`claim_payouts`) and sends transfers with our idempotency key in the transfer notes. Results come back by webhook, or by asking Razorpay for stale payouts (`stale_payouts`). A bounced transfer becomes a ledger REVERSAL.
- **Manual payments:** an admin records Wise, PayPal or bank transfers (`mark_payment_received`).
- **Verification scripts:** `npm run verify:payment`, `verify:payout`, `verify:delivery` (see `docs/LAUNCH.md`).
- **Status:** live payments are on hold until the owner approves. Razorpay Route activation is required for payouts.

11.5 Resend (email)

- **Settings:**

  - `RESEND_API_KEY`;
  - `EMAIL_FROM`, `EMAIL_REPLY_TO`;
  - `EMAIL_DELIVERY` (`live` sends to real recipients; anything else sends to `EMAIL_TEST_TO` with a "[Test → recipient]" prefix);
  - `EMAIL_BATCH`.
- **Flow:** events insert into `email_outbox` in the same transaction, with an idempotency key. `/api/cron/emails` (called every 5 minutes) renders templates (`core/email/templates.ts`) and sends through Resend (`lib/email/dispatch.ts`), recording attempts.
- **Templates (25):** WELCOME, ORDER_RECEIVED, ORDER_PLACED_FOR_YOU, QUOTE_READY, QUOTE_REMINDER, PAYMENT_RECEIVED, ORDER_CONFIRMED, ORDER_DELAYED, FIRST_LOOK_READY, PREVIEW_REMINDER, REVISION_RECEIVED, REVISED_DESIGN_READY, DESIGN_APPROVED, FILES_DELIVERED, FEEDBACK_REQUEST, NEW_MESSAGE, PROBLEM_RECEIVED, CREDIT_DECIDED, ORDER_CANCELLED, ORDER_REFUNDED, PAYOUT_SENT, CONTACT_RECEIVED, DESIGNER_APPLICATION_RECEIVED, DESIGNER_APPLICATION_ACCEPTED, DESIGNER_APPLICATION_REJECTED.
- **Preferences:** per person on `/account` (`email_preferences`).
- **Admin test:** Admin → Emails sends every template to the test inbox (admins only).
- **Domain:** sender domain verification and DMARC are pending (SEC-07).

11.6 Web Push

- **Settings:** `NEXT_PUBLIC_VAPID_PUBLIC_KEY`, `VAPID_PRIVATE_KEY`, `VAPID_SUBJECT`, `PUSH_BATCH`.
- **Flow:** people opt in on the device (`components/pwa/push-opt-in.tsx`), which stores a subscription in `push_subscriptions`. `/api/cron/push` sends pending notifications (`lib/push`, `web-push`), and the service worker (`public/sw.js`) shows them. Subscriptions that have expired are removed.

11.7 GitHub Actions scheduler

`push-dispatch.yml` runs every 5 minutes. It calls `${APP_URL}/api/cron/push` and `/api/cron/emails` with `Authorization: Bearer ${CRON_SECRET}`. A 404 means the secret in GitHub does not match Vercel's. To move to Vercel Pro crons, delete the workflow and add `{"crons":[{"path":"/api/cron/push","schedule":"* * * * *"}]}` (plus the email path) to `vercel.json`.

11.8 Google

- **OAuth (sign-in):** Google Cloud project "The CAD Pillar"; OAuth client "Clerk production"; redirect URI `https://clerk.thecadpillar.com/v1/oauth_callback`; consent screen published (External). No logo, to avoid brand review.
- **Search Console:** site verification by TXT record and `NEXT_PUBLIC_GOOGLE_SITE_VERIFICATION`. SEO is described in `docs/SEO.md`.

11.9 Adding a new integration: checklist

☐ Server-only secrets; never `NEXT_PUBLIC_` for a secret. Add the name to `.env.example` and to `/api/health` if it is required.

☐ Verify inbound webhooks by signature, in constant time, before parsing, and handle them idempotently.

- [ ] Queue outbound side effects in the database (an outbox) inside the business transaction; send from a worker.
- [ ] Share the minimum data. Never send counterparty identities across the anonymity wall.
- [ ] Document it here and in the privacy page’s list of service providers.

CHAPTER 12

Feature-by-feature technical reference

Each feature below is described the same way: **what it does**, **who uses it**, **where it lives** (routes, code, tables, database functions), and **the rules the system enforces**. Business rules come from `docs/DECISIONS.md` (version 1.28), which wins over any other document. Section numbers such as §11 refer to it.

12.1 Public website and lead capture

- **What:** the marketing site (home, jewelry CAD services, how it works, pricing "from $35", guarantee, quality, FAQ, contact, apply as a designer, privacy, terms, results), with structured data for search engines and a social sharing image.
- **Where:** `app/(marketing)/*`, `components/marketing/*`, `content/claims.ts` (confirmed claims only), `lib/seo.ts`, `app/sitemap.ts`, `app/robots.ts`; `docs/SEO.md`.
- **Data:** `marketing_leads` (contact and brief leads), `marketing_events`, `marketing_attribution` (first-touch cookie), `designer_applications`, `public_stats` (`/results`, hidden until 30 days of data, §12).
- **Rules:** forms are rate-limited per network address, stored only as a salted hash (`check_rate_limit`, `RATE_LIMIT_SALT`). Claims must be in `content/claims.ts`. A route-inventory test keeps the public URL list fixed.

12.2 Accounts and onboarding

- **Clients:** self sign-up → `/welcome` → `ensure_self()` creates a CLIENT row (audited). Or the team registers a client while placing an order on their behalf (`register_client`).
- **Designers:** apply publicly (lead, no account) → reviewed at `/admin/applications` → added by an admin on Admin → Designers (`register_designer`) → sign in with an emailed code → apply in-app (`apply_as_designer`) → sign the agreement (`accept_designer_agreement`; stores the SHA-256 of the exact text shown; a new version re-gates everyone) → status ACTIVE → payout details at `/settings/payouts` (`upsert_payout_account`).
- **Staff and pod leads:** an admin grants roles on Admin → Access (`set_user_role`). Making someone a pod lead creates their pod (`create_pod`).
- **Tables:** `users`, `user_roles`, `client_profiles`, `designer_profiles`, `staff_profiles`, `agreement_documents`, `agreement_acceptances` (immutable), `payout_accounts`, `designer_documents`.

12.3 Ordering and the CAD brief

- **What:** a client creates an order and writes a three-step brief (§11).
  1. **The piece:** name, type and at least one photo.
  2. **Stones:** centre stone yes or no; if yes, shape, size and setting; side stones optional.
  3. **Size and metal:** country and ring size (about 40 countries), metal and karat, notes.
- **Extras:** reference images with numbered pins and drawings; drafts and "start from a past order"; master instructions (a client's standing factory rules, shown on every order to the designer, pod lead and admins); certified centre-stone certificates (up to 5; PDFs cleaned to pages only).
- **Ring sizes:** inside diameter and circumference come from the chosen country's chart (`core/sizes`). The client may enter their own measurement instead; designers and pod leads see both and where the measurement came from (1.26).

- **Where:** `/orders/new`, `/orders/[id]/brief`, `/orders/[id]/brief/review`; `app/(app)/orders/[id]/brief/*`; `lib/client/standingActions.ts`.
- **Data:** `orders`, `order_specs`, `order_spec_accents`, `order_reference_images`, `order_reference_pins`, `order_stone_certificates`, `client_master_instructions`.
- **Rules:** no step moves on without its required answers (1.22). The brief is fixed once quoted (functions refuse with "the brief is fixed once the order has been quoted"). Only the owning client may write it.

12.4 Orders placed on a client's behalf

- **What:** the team turns a WhatsApp, email or phone request into an order, with at least one screenshot (§11).
- **Where:** `/admin/new-order`; `create_order_on_behalf`, `add_order_screenshot`, `submit_order_on_behalf`; the client receives the ORDER_PLACED_FOR_YOU email; table `order_screenshots`.

12.5 Categories, quotes and prices

- **Categories A–L** (minutes 15 to 480; Standard A–F, Complex G–L; `job_categories`).
- **Client bands** (internal defaults, `price_bands`, `app.band_price`): Standard A–C $35; Complex D–F $85; Sculpted G–L from $120, quoted; the first design for a new account is $35. The website shows one price, "from $35" (§4).
- **Quote:** an admin sets the price. The system splits it into designer pay, pod/QC pay and platform commission. The parts must sum exactly, and the system warns above a 40% payout share (§4).
- **Designer pay (1.25):** category minutes × the designer's rate (standard ₹8 a minute; an admin may set one designer's own rate in `designer_rates`). Repriced to the assigned designer's rate on assignment (trigger `orders_reprice_on_assign`).
- **Pod pay:** 50% of the designer's pay, at least ₹300 (admin settings).
- **Visibility:** pod leads and designers never see the client's price or payments. Money columns are read through role-aware functions (policies 0048).

12.6 Payments

- **Card and UPI:** `/orders/[id]/pay` → `open_payment_intent` → the Razorpay widget → the signed webhook → `confirm_payment` → ledger HOLD → order paid (`payment_intents`, `escrow_ledger`).
- **Manual:** an admin records a Wise, PayPal or bank payment (`mark_payment_received`).
- **Rule:** a client can never mark their own order paid (§7.1).

12.7 Confirmation, pod routing and assignment

- **Confirm (§6, §11):** an admin or the pod lead presses "Confirm order" once the brief is complete (`confirm_order`). **The SLA clock starts when the order is both paid and confirmed**, whichever happens last.
- **Pod routing:** if exactly one pod has a lead, a paid or confirmed order joins it automatically (`orders_auto_pod`). Otherwise the admin chooses.
- **Assignment (1.28):** the pod lead assigns one designer from their pod (`assign_designer`). After that the job is moved (`reassign_job`, with a reason once work has started) or taken back, never assigned again on top. An admin assigning directly is a folded exception.
- **Capacity:** 480 category-minutes per designer per shift; a warning in v1, blocking behind the `capacity_blocking` flag.
- **Data:** `job_assignments`, `pods`, `pod_members`; trigger `orders_assign_ref`.

12.8 Designer workspace

- **Where:** `/work`, `/work/[id]`, `/work/[id]/checklist`, `/work/[id]/review`, `/work/earnings`; `app/(app)/work/*`, `components/work/*`.
- **Clock:** "Start work" starts the category clock. Pause and resume are recorded (`job_pauses`); the designer's own clock stops while paused, but the due time to QC never moves (1.6). Alerts fire an hour before due and once overdue (`job_alerts`).
- **Uploads:** slots 3DM, STL, RENDER and TECH; direct signed upload up to 50 MB; 3DM blocks exploded; metadata stripped (chapter 8).
- **Render composer:** four equal cells; paste or drag images, swap by dragging, × to remove; composed server-side into one RENDER image.
- **Weights:** the designer saves the 18K cast weight (with allowances) and the diamonds (`job_weights`, `job_volumes`). The server joins a panel under the technical image (`lib/files/techSheet.tsx`). A CAD version cannot be submitted without the gold weight (`weights_gate`).
- **Designer check:** 9 lines, all must pass (`checklist_items`, `order_checks`, `mark_check`).
- **Final review:** full screen with the references and brief, the model (STL in 3D), and the checks; "Send to QC" unlocks when the last line passes.

12.9 Quality control

- **Two stages (§10):** the designer's 9-line check, then the QC manager's 5-line check (the pod lead, never the designer). Each line is Pass or Fail, with a finding and remark.
- **Decision:** `qc_decide` passes the work (CLIENT_PREVIEW) or sends it back (REVISION_REQUESTED). Sending back needs a failed line or a note, and the failed lines become the revision note. Measurements are recorded for CAD (`qc_measurements`, `record_qc_measurements`).
- **Rule:** the reviewer is recorded at decision time and cannot be the producer or the order's owner (§7.3).
- **Where:** `/qc`, `/qc/[id]`, `/qc/orders`, `/qc/earnings`.

12.10 Client preview, approval, revisions, changes and disputes

- **Preview:** the client sees the four render views, the gold weight chart and the diamond details, never the model before approval (chapter 8).
- **Approve:** `transition_order` → APPROVED. That releases the STL, the 3DM and the summary sheet.
- **Revisions:** 2 rounds included within 14 days of first look (`request_revision`, `revision_requests`). QC send-backs do not count.
- **Change orders:** changes after approval are a new job at their own category (`create_change_order`).
- **Disputes:** during work or at preview, with a written reason (`raise_dispute`); resolved by Ops or Finance (`resolve_dispute`; `disputes`).

12.11 Delivery and the spec guarantee

- **Deliverables (§9):** 3DM and STL; one render image with four views; one technical image (measurements, diamonds, gold weight); a summary sheet PDF made by the platform; a QC certificate (which doubles as the spec sheet); a weight sheet (grams in 10K, 14K, 18K, Pt950 and Ag925).
- **Fingerprints:** delivered files are SHA-256 fingerprinted and archived read-only (`certificates`, immutable).
- **Spec claims:** `/orders/[id]/claim` (`spec_claims`, `spec_waivers`). Exclusions: an edited file (fingerprint mismatch), a recorded waiver, claims after 30 days. Liability is capped at the CAD fee (§7).

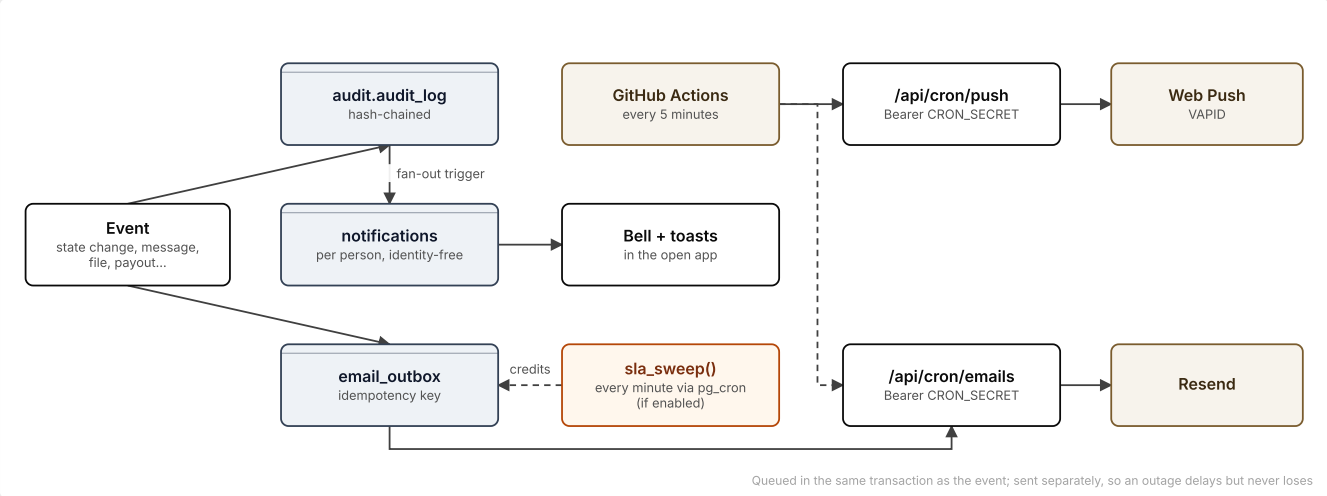
- **Preview link for the client's customer:** `/p/[token]`, a branded 3D view (`preview_links`, `create_preview_link`). The token is hashed at rest, expires and can be revoked.

12.12 Messaging

- **Threads:** client ↔ team (to the client everyone is "Your team"), and designer ↔ pod lead or admin (internal). Designers can never post in client threads (RLS) (§11).
- **The brief is the first message** of every thread, with reference pictures; the designer's copy never names the client (1.27).
- **Markup:** any picture can be marked up with a circle, a pen or numbered pins, and sent.
- **Forwarding:** the team forwards between client and designer. To the designer it appears as "Forwarded from the client"; files are cleaned again on the way.
- **Live:** a Realtime "thread changed" signal on a private channel (no content), read receipts and unread counts (`thread_reads`, `/api/threads/[id]/read`).
- **Data:** `message_threads`, `thread_participants`, `messages` (immutable), `thread_files`.

12.13 Notifications, email and push

- **In-app:** audit entries fan out into identity-free `notifications` for the right people (never the person who caused the event). The bell shows unread counts, and toasts appear within 20 seconds while the app is open (1.18).
- **Email:** `email_outbox` → `/api/cron/emails` → Resend (25 templates; chapter 11).
- **Push:** `push_subscriptions` → `/api/cron/push` → web push.

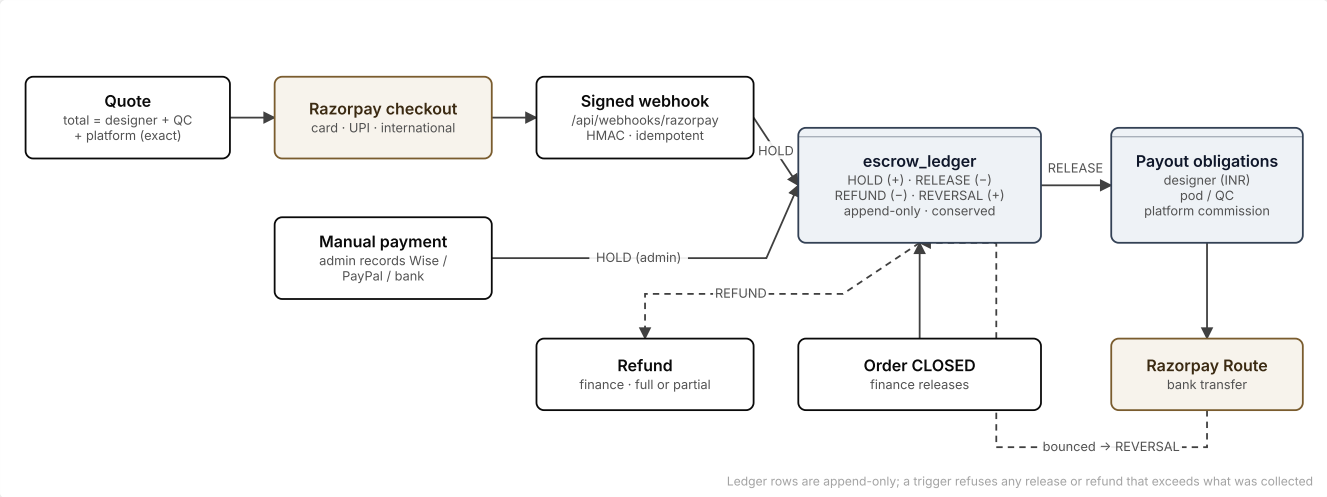


Notifications, push and email

12.14 SLA and credits

- **Standard tier:** first look within 6 hours of confirmation. **Complex tier:** a deadline set by the admin with the price (§6).
- **Sweep:** `sla_sweep()` every minute (pg_cron) flags each overdue order once. A missed standard order creates one **pending** credit for the full amount, decided by an admin (apply to the next order, refund, partial credit or reject, with a reason) (`client_credits`, `sla_events`, `/admin/credits`). Automatic credit sits behind the `auto_sla_credit` flag.

12.15 Escrow and payouts



Money flow and escrow

- **Ledger:** `escrow_ledger` rows HOLD, RELEASE, REFUND and REVERSAL. Append-only and conserved per order.
- **Release:** Finance releases a CLOSED order (`open_payouts_for_order`), creating `payout_obligations` per leg. `lib/payouts/execute.ts` sends Razorpay Route transfers (`payouts`). Results arrive by webhook. Release is refused if a payee has no verified bank account (§7).
- **Batches and exports:** `/admin/payouts` , `/admin/payouts/[batch]` (CSV export for bank transfer, §5).

12.16 Pod operations

- **Board, performance and designer pages:** `/pod/board` , `/pod/performance` , `/pod/designers/[id]` . Output, working time, turnaround, on-time rate, first-pass QC rate, QC returns and client changes, all computed from recorded events. Exports in Excel, CSV and PDF (`/api/pod/report`).
- **Move or take back a job:** `reassign_job` (reason once started) and withdrawal (`0066_one_designer_withdraw.sql`). **Escalate to admins:** `job_escalations` .
- **Pod materials:** `/materials` ; `pod_materials` (PDF, image, ZIP, STEP). Only the lead or an admin adds and removes.

12.17 Administration

Screen	Purpose	Key functions and tables
`/admin`	Pipeline and queues	Visibility functions
`/admin/access` , `/admin/access/pods`	Roles, status, pods and members	`set_user_role` , `set_user_status` , `create_pod` , `set_pod_member`
`/admin/designers` , `/admin/designers/[id]`	Add designers, rates, bank details, identity documents (logged)	`register_designer` , `set_designer_rate` , `set_payout_account_for` , `add_designer_document` , `payout_account_full`
`/admin/applications`	Review designer applications (portfolios through short-lived links)	`list_designer_applications`
`/admin/leads`	Website leads	`marketing_leads`
`/admin/clients` , `/admin/users`	Clients, account managers, all users	`client_managers` , identity lookups (logged)

Screen	Purpose	Key functions and tables
`/admin/orders`, `/admin/orders/export`	All orders, CSV export	
`/admin/credits`	Decide SLA credits	`client_credits`
`/admin/payouts`	Payout batches	`payout_obligations`, `payouts`
`/admin/billing`	Billing overview	
`/admin/emails`	Send every template to the test inbox	`email_outbox`
`/admin/settings`	Settings and feature flags	`app_settings`, `feature_flags`
`/admin/new-order`	Order on a client's behalf	see above

12.18 Calculators

`/tools` (designers, pod leads, admins). Gold casting weight (18K cast weight + 0.1 g, + 1 g for "with post"; other metals by density) and diamond weight (pieces × carat per stone from the size charts; grams = ct × 0.2). Nothing is saved (§10; `core/weights`, `core/tools`).

12.19 Wallet and plans

`/wallet`, `/wallet/invoices`, `/wallet/transactions`: credit, payments on open orders, history. Plans (volume credits, monthly partner, dedicated designer) and wallet top-ups are behind feature flags and sold manually until enabled (§4, §12).

12.20 Platform features

- **Installable app (PWA):** manifest, service worker, offline page, install hint, push opt-in; iPhone safe areas.
- **Time zones:** every time is shown in the viewer's time zone (device cookie). Support hours are shown in US Eastern time with daylight saving (`core/support/hours`), with an online indicator and one-tap WhatsApp.
- **Timeline:** a client-safe window onto the audit log on each order (`docs/BLACKBOX.md` AH–AJ).
- **Design preview:** `/design-preview` shows every component.

CHAPTER 13

Developer onboarding and maintenance

Everything a new developer needs to become productive and safe on The CAD Pillar in the first week, plus the routine maintenance that keeps the platform healthy.

13.1 Day 1: read, in this order

1. `CLAUDE.md` : the house rules. `docs/DECISIONS.md` wins over every other document; one phase per pull request; no pull request without passing tests; never run migrations against production.
2. `docs/DECISIONS.md` : what the business has decided (roles, prices, payouts, SLA, deliverables, QC).
3. This handbook, chapters 2, 4, 5, 6 and 8.
4. `docs/SYSTEM.md` and `docs/OPERATIONS.md` : the order life cycle and every action per role.
5. `docs/redesign/REVAMP-NOV-2026.md` and `/design-preview` : the design system you build with.

13.2 Machine setup

Need	Version	Why
Node.js	≥ 20 (CI uses the version in `ci.yml`)	Next.js 15, scripts
npm	bundled	`npm ci` from `package-lock.json`
Docker	recent	The throwaway test Postgres (`docker compose`), and PostgREST for e2e
Git	recent	
Playwright Chromium	`npx playwright install chromium`	e2e and design QA
pandoc ≥ 3.1, LibreOffice (optional)		Only to regenerate this handbook's DOCX and check it

```
git clone https://github.com/simran313/GetCAD && cd GetCAD
npm ci
cp .env.example .env.local      # fill with DEVELOPMENT values only (Clerk dev keys, a dev Supabase project)
npm run test:db:up              # throwaway Postgres on 127.0.0.1:5433 (trust auth, tmpfs)
npm run ci                      # typecheck, lint, all tests, secret scan; must pass before you start
npm run dev                     # http://localhost:3000
```

Never put production keys in `.env.local` on a development machine. The only production credential a developer may need is `DATABASE_URL` , for applying a migration the owner has approved, from a trusted machine.

13.3 Everyday commands

Command	What it does
`npm run dev`	Development server
`npm run typecheck` / `npm run lint`	Strict TypeScript; ESLint, including the `core/` boundary rule
`npm test`	All vitest suites (needs the test database)

Command	What it does
`npm run test:local`	Starts the test database, then runs every test
`npm run ci`	Exactly what CI's `verify` job runs
`npm run e2e`	Browser journeys (needs Docker and the Clerk **development** keys; see `tests/e2e/README.md`)
`npm run db:status` / `npm run db:apply`	Show or apply pending migrations to `$DATABASE_URL` (local or staging; production only by the owner)
`npm run secret-scan`	Scans the working tree for secrets
`npm run verify:payment` / `verify:payout` / `verify:delivery`	Razorpay checks (see `docs/LAUNCH.md`)

13.4 How to…

13.4.1 Add a page

1. Create `app/(app)/<segment>/page.tsx`. If it is a new top-level segment, add it to `APP_ROUTE_SEGMENTS`, and to `PROTECTED_PREFIXES` if it needs sign-in (`core/auth/session.ts`). The route-inventory test fails otherwise.
2. Read data with `createUserSupabaseClient()`, never the service role, through a visibility function or an RLS-protected table.
3. Use `components/ui` and `components/shell`, tokens only, no hard-coded colours or pixel offsets for fixed bars (`--chrome-top`, `--chrome-bottom`).
4. Add it to role navigation if needed (`core/nav`).
5. Commit with `[design-qa:after]` in the message when the change is visual, so screenshots and axe run.

13.4.2 Add a server action

Follow the checklist in chapter 7: `"use server"`, `auth()`, the user client, a database function that checks the caller, validation, `revalidatePath`, friendly errors, tests.

13.4.3 Change the database

Follow chapter 5, "Changing the schema safely": a new numbered migration, RLS on, a policy or function per access path, `search_path` pinned, explicit grants, audit writes, and a `tests/db/*.test.ts` that proves allowed and refused callers. **Never** edit an applied migration. **Never** run tests against a remote database; the harness refuses.

13.4.4 Add a test

Kind	Where	Runs in
Pure logic	next to the module (`core/**/x.test.ts`)	vitest
Database (RLS, functions, invariants)	`tests/db/*.test.ts`, using `connectFreshDb()` from `tests/helpers/db.ts`	vitest + the throwaway Postgres
Config and route rules	`tests/config/*.test.ts`	vitest
User journeys	`tests/e2e/*.e2e.ts`	Playwright in CI's `e2e` job
Visual and accessibility	`tests/design-qa/*.qa.ts`	The design-qa workflow

13.4.5 Handle money

Integers only (cents, paise), using `core/money` helpers. Never compute a client's price where a pod lead or designer can read it. Ledger rows are append-only: correct with a new row.

13.4.6 Handle identities

Never join identity tables into a query a client or designer can run. Admin identity reads go through `record_identity_lookup` / `lib/clerk/identities.ts` and are logged. Run the anonymity e2e suite for any change near orders, threads or files.

13.5 Code conventions

- TypeScript strict; no `any` without a comment explaining why.
- `core/` is pure: no React, no Next.js, no I/O.
- Server-only modules import `server-only`.
- File and component names are kebab-case for components and camelCase for helpers, matching the folder you are in.
- Copy is plain, short and specific (see the redesign docs). No claims that are not in `content/claims.ts`.
- Comments explain *why*, especially for security decisions.

13.6 Pull requests

- One phase or feature per pull request. Title in plain English.
- A description that covers what changed, why, how it was tested, any migration and its rollback, and screenshots for UI.
- CI green (`verify` and `e2e`). Design QA for visual changes.
- Do not merge your own pull request into production without the owner's review.

13.7 Routine maintenance

Cadence	Task	How
Weekly	Dependency advisories	`npm audit --omit=dev`; open one upgrade pull request for anything High or Critical
Weekly	Scheduled-job health	GitHub → Actions → push-dispatch: recent runs green
Weekly	Error review	Vercel → Logs → Level: Error
Monthly	Audit chain	`SELECT audit.verify_chain();` → `{"valid": true}`
Monthly	Email queue	`email_outbox` rows stuck or failing
Monthly	Storage growth	Supabase → Storage usage; orphaned files
Quarterly	Security review	Re-run chapter 9's method; update the register
Quarterly	Restore drill	Restore a backup into a scratch project (chapter 14)
Quarterly	Access review	Vercel, GitHub, Supabase, Clerk, Razorpay members; remove leavers

Cadence	Task	How
On every release	Health check	`/api/health`; quick role walkthrough (chapter 16)

13.8 Regenerating this handbook

The Markdown in `docs/` is the source of truth. The PDF and Word files are generated.

```
# 1. Refresh the code-derived appendices (needs a local Postgres on 127.0.0.1:5433)
npm run test:db:up
DATABASE_URL=postgres://postgres@127.0.0.1:5433/postgres \
  node docs/document-generation/extract-reference.mjs

# 2. Re-render diagrams, then build the PDF and DOCX into docs/master-documents/
node docs/document-generation/build-handbook.mjs
```

Requirements: Node ≥ 20 with the repository's dependencies installed (Playwright Chromium, pdfjs-dist), and **pandoc ≥ 3.1** on the PATH. LibreOffice is optional; with it, the build also renders the DOCX to PDF to check pagination. The chapter order and metadata are in `docs/document-sources/handbook.json`. Diagram sources are in `docs/diagrams/src/*.mjs`. Edit those, never the SVG or PNG outputs.

CHAPTER 14

Troubleshooting and disaster recovery

The first part is a symptom-first troubleshooting guide. The second covers disaster recovery: what can be lost, how it is restored, and who does what. Commands that touch production are marked **(owner)**.

14.1 First moves for any incident

1. **Is it the whole site or one feature?** Open `https://thecadpillar.com/api/health` (configuration) and a public page (hosting).
2. **What changed?** Vercel → Deployments (the latest deploy and its time); GitHub → merged pull requests; any migration applied today; any provider setting changed (Clerk, Supabase, Razorpay, DNS).
3. **Roll back first, investigate second.** If a deploy is the likely cause, Vercel → the previous good deployment → **Promote to Production**. That takes seconds and is safe.
4. **Look at the logs:** Vercel → Logs → Level: Error, around the time reported; Supabase → Logs (API, Postgres).
5. **Write down what you saw and did**, in the pull request or incident note, for the post-incident review.

14.2 Troubleshooting by symptom

Symptom	Likely cause	Check	Fix
Signed-in pages load but show no data	Supabase third-party auth domain does not match the Clerk instance; or `CLERK_JWT_ISSUER` is wrong	Supabase → Authentication → Third-Party Auth; Vercel variables	Set the domain to `https://clerk.thecadpillar.com`; redeploy (`docs/CLERK-PRODUCTION.md`)
Everyone lands on onboarding or `/welcome` as a new client	Their Clerk id has no `users` row (for example after an instance change)	`db/ops/clerk-switch-health.sql` (read-only)	Run the remap or clean-up from `db/ops` with a dry run first **(owner)**
"Development mode" badge, or a 100-user cap	Production deploy uses the development Clerk key	Page source contains `pk_test_`	Set the production keys on the **Production** environment; redeploy without the build cache
"Access blocked: missing client_id" on Google sign-in	Production Clerk has no custom Google credentials	Clerk → SSO → Google	Add the Google OAuth client ID and secret (chapter 11)
A sign-in code never arrives	Spam filtering; an Apple private-relay address; Clerk mail DNS	Clerk → Domains (all verified); the user's spam folder	Add the person's real email in Clerk; fix email DNS (SEC-07)
`/api/health` returns 503	A required configuration group is missing	The response names the group	Add the variable in Vercel (Production); redeploy
Payment taken but the order still unpaid	Webhook failing: wrong secret, Deployment Protection on, or the URL changed	Razorpay → Webhooks → delivery logs; `npm run verify:delivery`	Fix the secret or URL; turn off protection for Production; replay the webhook (idempotent)
Webhook returns 401	Signature mismatch	Is `RAZORPAY_WEBHOOK_SECRET` identical in Razorpay and Vercel?	Make them match exactly; redeploy
Payout stuck "in flight"	Lost transfer response	`stale_payouts`	The payout worker asks Razorpay and records the result; check the Razorpay dashboard

Symptom	Likely cause	Check	Fix
Push and emails not sending	Scheduler not running, or `CRON_SECRET` mismatch (404)	GitHub → Actions → push-dispatch logs	Re-sync `CRON_SECRET` in GitHub secrets and Vercel; re-run the workflow
Emails go to the test inbox	`EMAIL_DELIVERY` is not `live`	Vercel variables	Set `EMAIL_DELIVERY=live` when approved
SLA misses never flagged	`pg_cron` disabled	Supabase → Extensions; `select * from cron.job`	Enable `pg_cron`; run the schedule statement from `0040_quote_sla.sql` **(owner)**
Chat does not update live	Realtime off, or a channel policy missing	`NEXT_PUBLIC_CHAT_REALTIME`; Supabase Realtime settings	Set the flag; apply migration 0052 policies; the page still polls the stamp endpoints as a fallback
A file download gives 404	Not permitted in this state (for example a client before approval), or the object is missing	The order status and the caller's role; Storage object key	Expected behaviour, or re-upload the version
Upload refused "type does not match"	The file's real signature is not the declared type	Open the file locally	Export the correct format (PNG/JPEG, real STL or 3DM)
Pages slow (over 1 s)	Function region far from the database	Vercel → Functions region vs Supabase region	Co-locate (`docs/DEPLOY.md` §2b)
Search Console "Redirect error"	Unknown paths redirected by middleware	`curl -I` an unknown path should give 404	Already fixed by `skipsClerk`; validate the fix in Search Console
CI database tests fail locally	Test Postgres not running	`docker ps`, port 5433	`npm run test:db:up`
"REFUSING TO RUN … not a local database"	`DATABASE_URL` exported in your shell	`echo $DATABASE_URL`	`unset DATABASE_URL`; this guard protects production
Audit chain invalid	Tampering or manual edits	`SELECT audit.verify_chain();` shows `broken_at`	Treat as a security incident (below)

14.3 Disaster recovery

14.3.1 What we must be able to recover

Asset	Where	Recovery source	Target
Application code	GitHub	Any clone; Vercel keeps every deployment	Minutes
Configuration and secrets	Vercel, GitHub secrets, provider dashboards	The owner's password manager (keep an encrypted, up-to-date list of variable names and where each value comes from)	Hours
Database (orders, money, identities, audit)	Supabase Postgres	Supabase backups / point-in-time recovery (**plan-dependent, SEC-02**); exports	RPO ≤ 24 h (daily backup) or minutes (PITR); RTO ≤ 4 h
Files (deliverables, documents)	Supabase Storage	Storage is **not** covered by database backups. Periodic copy to an external bucket (roadmap).	RPO ≤ 7 days today; target 24 h
Sign-in accounts	Clerk	Clerk keeps them. Ids are mirrored in `users`.	Provider SLA

Asset	Where	Recovery source	Target
Payment records	Razorpay plus the ledger	Razorpay dashboard and exports reconcile the ledger	Hours

RPO is the recovery point objective: how much data may be lost. RTO is the recovery time objective: how long recovery may take.

14.3.2 Scenario playbooks

A. A bad deploy breaks the site. Promote the previous deployment (seconds). Open a fix pull request. No data risk.

B. A bad migration damaged data (owner). 1. Stop writes if the damage is spreading: put the site in maintenance mode by promoting a deployment that shows the offline page, or pause the relevant feature flag. 2. Identify the time just before the migration (`schema_migrations.applied_at`). 3. With PITR: restore into a **new** project at that time, export the affected tables, compare, and repair production with a reviewed migration. Without PITR: the same, from the latest daily backup. Changes since then must be reconciled from the audit log and Razorpay. 4. Verify: `audit.verify_chain()`, the money conservation queries, the role walkthrough.

C. The Supabase project is lost or unavailable for a long time (owner). 1. Create a new project in the same region; restore the latest backup. 2. Re-apply grants and settings: third-party auth (Clerk domain), exposed schemas (`public, graphql_public`), Realtime, `pg_cron`, Storage buckets (`order-files`, `designer-application-files`, `designer-documents`, all private). 3. Restore Storage objects from the external copy. 4. Update `NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY` and `SUPABASE_SERVICE_ROLE_KEY` in Vercel; redeploy. 5. Run `npm run db:status` against the new database to confirm every migration is recorded.

D. A secret is leaked (owner).

Secret	Rotate at	Then
Supabase service-role key or JWT secret	Supabase → API settings	Update Vercel; redeploy. The old key stops working at once.
Database password	Supabase → Database	Update local `.env.local` only
`CLERK_SECRET_KEY`	Clerk → API keys	Update Vercel; redeploy
Razorpay key secret or webhook secret	Razorpay dashboard	Update Vercel (and the webhook config); redeploy
`CRON_SECRET`	Generate new (`openssl rand -hex 32`)	Update Vercel **and** the GitHub secret
VAPID keys	Generate new	Update Vercel; existing push subscriptions must re-subscribe
`RESEND_API_KEY`	Resend	Update Vercel

After rotating, review the logs for use of the old secret. Record the incident.

E. Suspected data breach or anonymity breach. 1. Contain: revoke the involved accounts (`set_user_status`); rotate the secrets involved. 2. Preserve evidence: export the relevant audit log range and Vercel and Supabase logs. Do not edit rows. 3. Assess what identities or files were exposed, using the audit log (identity reads are logged). 4. Notify affected clients and designers as required by contract and law (India's DPDP Act 2023; clients' jurisdictions). Get legal advice. 5. Fix the root cause in a reviewed pull request with a test that would have caught it.

F. Clerk outage. Sign-in is unavailable; nothing is lost. Communicate through WhatsApp support. Do not switch instances during an outage.

G. Domain or DNS problem. Check Vercel → Domains (valid configuration) and GoDaddy (nameservers `ns1/ns2.vercel-dns.com`, domain not expired; keep auto-renew on).

14.3.3 Restore drill (quarterly)

1. Create a scratch Supabase project.
2. Restore yesterday's backup into it, or `pg_dump` and restore.
3. Point a local copy of the app at it with development Clerk keys, and confirm orders, files and the audit chain read correctly.
4. Record the times taken (actual RTO) and any gaps. Delete the scratch project.

14.3.4 Recommended backup improvements

- A paid Supabase plan with daily backups; PITR when justified (SEC-02).
- A weekly encrypted `pg_dump` to storage the company controls, in a separate account.
- A nightly copy of Storage buckets to an external bucket.
- An inventory of every configuration variable (names, the provider each value comes from, who can rotate it) kept with the owner.

CHAPTER 15

Technical debt and scalability roadmap

An honest list of what should be improved, why, and in what order. Items are grouped by horizon. Each has an effort estimate: **S** is under a day, **M** a few days, **L** a week or more.

15.1 Current scale and headroom

Dimension	Today	Comfortable limit of the current design	First bottleneck
Accounts	26 real (plus tests)	Thousands	Admin screens that list everyone without pagination
Orders	Tens per month	Tens of thousands per month	Visibility functions that compute per row; missing indexes on hot filters
Files	MB to GB	Hundreds of GB on a paid Supabase plan	Storage cost; no lifecycle rules
Concurrent users	Low	Hundreds	Database connections from serverless functions (use the pooler); Realtime channel count
Scheduled work	Every 5 minutes (GitHub Actions)	Fine for notifications	SLA precision depends on `pg_cron`

The architecture (stateless serverless functions plus Postgres) scales well vertically. The practical limits are database plan size, connection pooling and query cost, not the code structure.

15.2 Now (next 30 days)

#	Item	Why	Effort
1	Upgrade `next`, `sharp` and `zod` (SEC-01, 04, 05, 12)	Published advisories	S
2	Grant-hardening migration and test (SEC-06, SEC-11)	Defence in depth; closed-by-default functions	S
3	Supabase paid plan; restore drill (SEC-02)	Recoverability	S
4	Rotate the database password; remove the unused Vercel project (SEC-03)	Credential hygiene	S
5	DMARC, DKIM and the transactional sender domain (SEC-07)	Deliverability; anti-spoofing	S
6	Vercel Pro; separate preview database; crons back in `vercel.json` (SEC-08, SEC-15)	Terms, isolation, reliable schedules	M
7	Confirm `pg_cron` and the SLA sweep schedule (SEC-14)	SLA promise depends on it	S

#	Item	Why	Effort
8	Automated dependency updates (Dependabot or Renovate) and `npm audit` in CI	Catch advisories early	S

15.3 Next (one to three months)

#	Item	Why	Effort
9	Error tracking (for example Sentry) and uptime monitoring with alerts	Today errors are found by reading Vercel logs	S
10	Content-Security-Policy, report-only then enforced (SEC-09)	XSS containment	M
11	Per-account rate limits and storage quotas (SEC-10)	Abuse and cost control	M
12	Nightly copy of Storage buckets to external storage	Files are not in database backups	M
13	Pagination and server-side filtering on admin lists (users, orders, leads)	Growth	M
14	Index review using `pg_stat_statements` on production; add indexes for hot filters (status, pod_id, client_id, created_at)	Query cost at scale	M
15	Connection pooling check (Supabase pooler for serverless)	Connection exhaustion under load	S
16	Branch protection on `main` (required checks, no force push)	Process safety	S
17	Remove real accounts from the development Clerk instance (SEC-13)	Data minimisation	S

15.4 Later (three to twelve months)

#	Item	Why	Effort
18	Move business rules that live in both TypeScript and SQL to one place, with generated types (for example from the database schema)	Fewer drift bugs; tests already guard several pairs	L
19	Generated Supabase types for every RPC (`supabase gen types`)	Type safety at the boundary	M
20	Storage lifecycle: archive old versions to cold storage	Cost	M
21	Wallet, plans and in-app payouts out from behind flags (§12)	Revenue features	L

#	Item	Why	Effort
22	PayPal and Wise integrations (§13, last build phase)	Payment options	L
23	Read replicas or materialised summaries for pod performance reports	Report speed at scale	M
24	Multi-region or a status page	Customer trust at scale	M
25	Dental brand (separate product, January 2027 per DECISIONS)	Out of scope for this codebase	—

15.5 Known debt in the codebase

Debt	Where	Notes
Superseded documents	`docs/SYSTEM.md`, `docs/TCP-BUILD-SPEC.md`	Marked “superseded where they disagree with DECISIONS”. Keep them historical; this handbook and DECISIONS are current.
Two sources for some rules	`core/` and SQL (for example payout maths, slot kinds)	Pairs are cross-checked by tests; prefer one source when touching them
Scheduler workaround	`.github/workflows/push-dispatch.yml`	Remove on Vercel Pro
Legacy deliverable slots	Ten view slots before 1 November 2026	Still readable on old versions; no new uploads
Large page components	Some order and work pages	Split when next touched; keep Server Components thin
Manual production migrations	`npm run db:apply` by the owner	Deliberate for safety; consider a gated “apply migrations” workflow with manual approval later

15.6 Principles for future work

- Keep the database as the security boundary. New features add functions and policies first, UI second.
- Prefer append-only records for anything with money, identity or legal weight.
- Every new external dependency needs an owner, a failure mode and a line in chapter 11.
- Measure before optimising (`tests/e2e/zz-speed.e2e.ts`, `docs/PERFORMANCE.md`).

CHAPTER 16

Production readiness checklist

Use this before launch campaigns, after major releases, and quarterly. Each item has a status as of the audit (13 November 2026): ✅ done and verified, ⚠️ open or needs action, ❔ not verifiable from the repository (the owner must confirm).

16.1 Security

#	Item	Status	Reference
1	RLS enabled on every table	✅ 69/69	Appendix A
2	Every SECURITY DEFINER function pins `search_path` and checks its caller	✅ 254/254 pinned; caller checks sampled	Appendix B, ch. 9
3	Internal write helpers not executable by `anon` / `authenticated`	⚠️	SEC-06
4	Exposed Data API schemas are only `public, graphql_public`	❔	SEC-06
5	No secrets in the repository or its history	✅	ch. 9
6	Dependencies free of High/Critical advisories	⚠️	SEC-01, 04, 05
7	HSTS and security headers	✅	ch. 9
8	Content-Security-Policy beyond framing	⚠️	SEC-09
9	Webhook signature verification and idempotency	✅	ch. 7
10	Cron endpoints secret-protected	✅	ch. 7
11	Downloads authorised per request, links ≤ 60 s	✅	ch. 8
12	Credentials rotated; no stray copies	⚠️	SEC-03
13	Anonymity e2e crawl green	✅ (CI)	`tests/e2e/anonymity.e2e.ts`

16.2 Identity and access

#	Item	Status	Reference
14	Clerk production instance with own domain	✅	`docs/CLERK-PRODUCTION.md`
15	Google sign-in with company-owned credentials	✅	ch. 11
16	Supabase third-party auth points at production Clerk	✅	Health check after the switch

#	Item	Status	Reference
17	Development Clerk instance holds test users only	⚠️	SEC-13
18	Team access reviewed (Vercel, GitHub, Supabase, Clerk, Razorpay)	❔	ch. 13

16.3 Data and recovery

#	Item	Status	Reference
19	Paid Supabase plan with daily backups	⚠️ (Free at audit)	SEC-02
20	Restore drill done in the last quarter	⚠️	ch. 14
21	Storage buckets copied off-site	⚠️	ch. 14
22	Audit chain valid (`audit.verify_chain()`)	✅ (checked after the Clerk switch)	ch. 5
23	`pg_cron` enabled with the SLA sweep scheduled	❔	SEC-14

16.4 Infrastructure and operations

#	Item	Status	Reference
24	Vercel plan appropriate for commercial use	⚠️ (Hobby)	SEC-08
25	Preview deployments isolated from production data	❔	SEC-08
26	Function region co-located with the database	❔	ch. 10
27	Scheduler running (push and email every 5 min)	✅ (workflow present)	ch. 10
28	Error tracking and uptime alerts	⚠️ (none)	ch. 15
29	Branch protection on `main`	❔	ch. 10
30	`/api/health` all groups true	✅ (13 Nov 2026)	ch. 7

16.5 Email and domain

#	Item	Status	Reference
31	SPF covers every sender	⚠️ (Google only)	SEC-07
32	DKIM at the correct selector	⚠️ (misplaced at apex)	SEC-07
33	DMARC published	⚠️ (missing)	SEC-07
34	`EMAIL_DELIVERY=live` (when approved)	❔	ch. 11

16.6 Payments

#	Item	Status	Reference
35	Razorpay live keys and webhook secret distinct and strong	❔ (payments on hold)	`docs/LAUNCH.md`
36	Razorpay Route activated for payouts	❔	ch. 11
37	`verify:payment`, `verify:payout`, `verify:delivery` pass against production	❔	`docs/LAUNCH.md`

16.7 Product walkthrough (after each release, about 15 minutes)

- [ ] Website: home, pricing, an unknown URL gives 404, `/sitemap.xml`.
- [ ] Sign in as admin (email code and Google): the admin home shows the pipeline.
- [ ] Client: create a draft brief with a reference image; cancel it.
- [ ] Designer: the work list shows assigned jobs; the job page opens; the STL viewer loads.
- [ ] Pod lead: the board and performance pages load; an export downloads.
- [ ] Chat: a message from one side appears on the other without refresh.
- [ ] Downloads: a permitted file downloads; a forbidden one gives 404.
- [ ] `/api/health`: every group true.
- [ ] Vercel logs: no new errors.

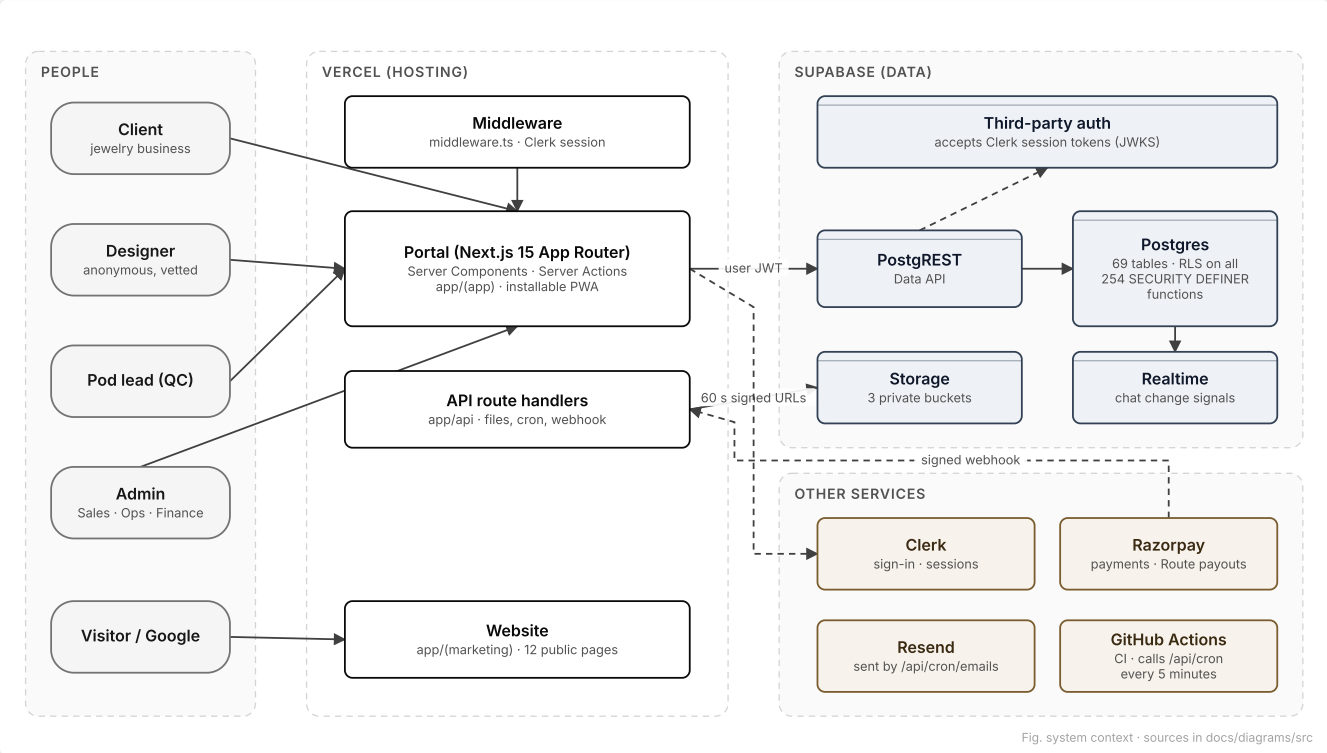
CHAPTER 17

Architecture and workflow diagrams

All diagrams in one place, for printing or presenting. Each is generated from an editable source in `docs/diagrams/src/` by `node docs/document-generation/diagrams.mjs`. The source is data (boxes, groups and arrows with coordinates), so a diagram can be changed with a text editor and re-rendered. SVG files are used by the Markdown and the PDF; 2× PNG files (`docs/diagrams/png/`) are used by the Word document.

17.1 System context

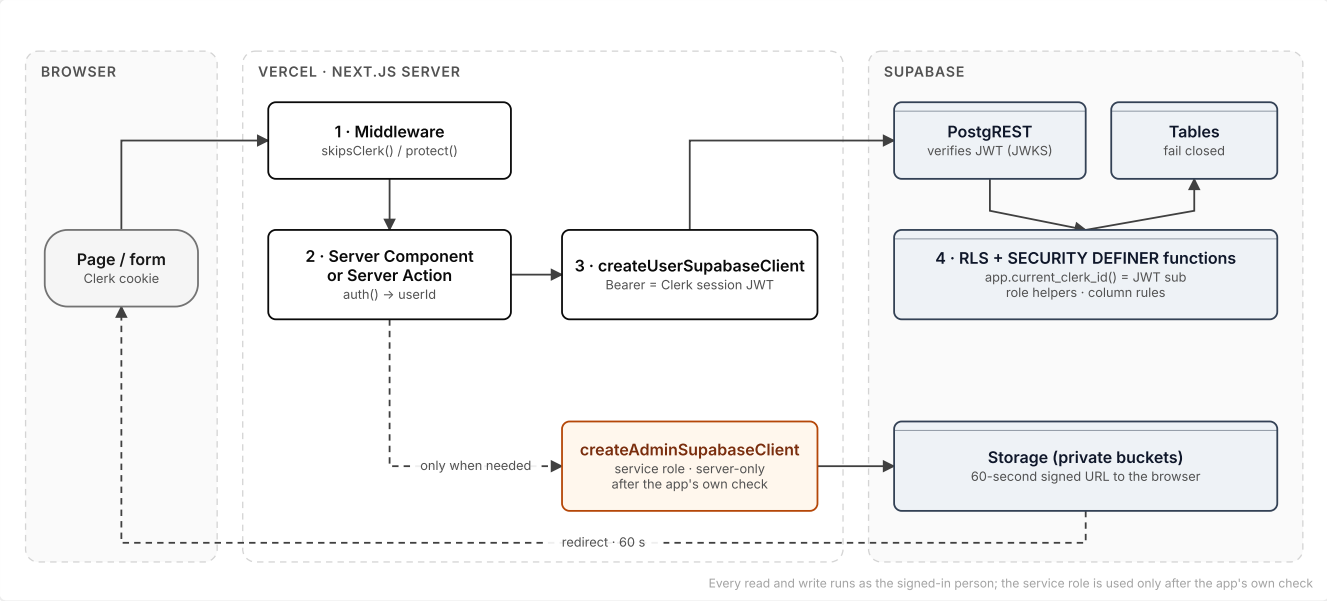
Who uses the platform and which services it depends on.



System context

17.2 Request flow and trust boundaries

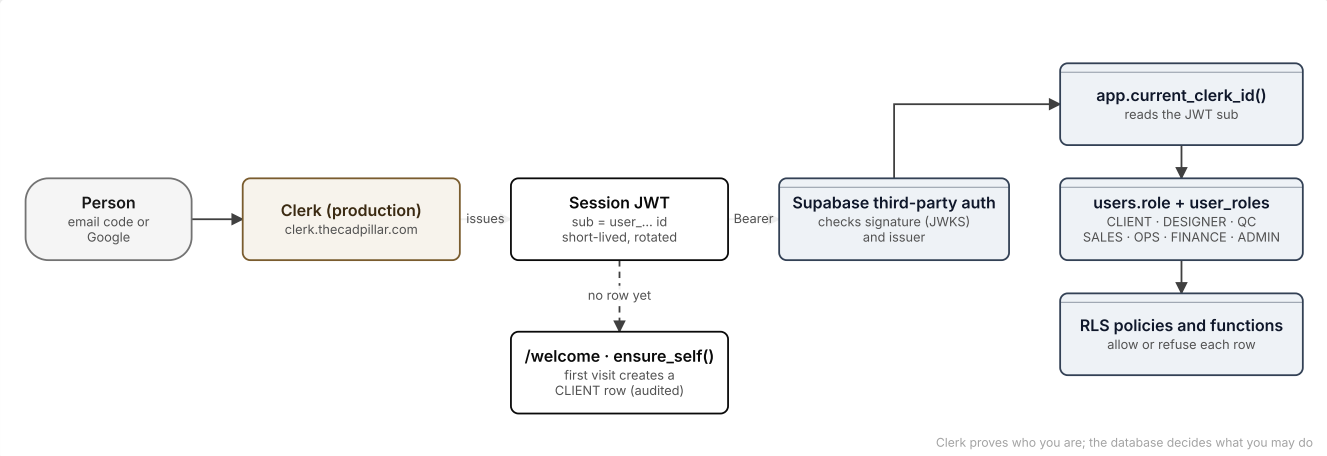
How a signed-in request reaches the data, and where each check happens.



Request flow and trust boundaries

17.3 Authentication and identity bridge

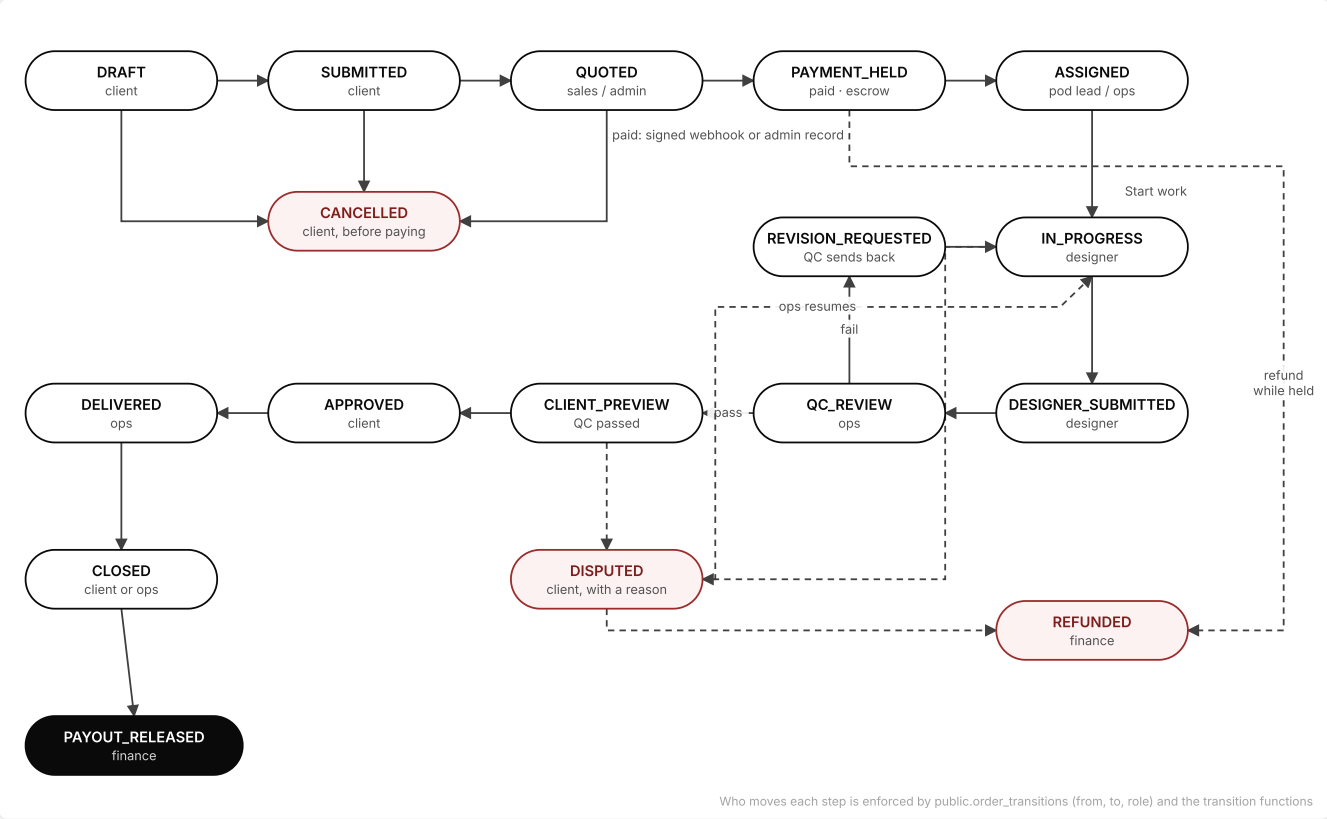
From sign-in to a database row.



Authentication and identity bridge

17.4 Order state machine

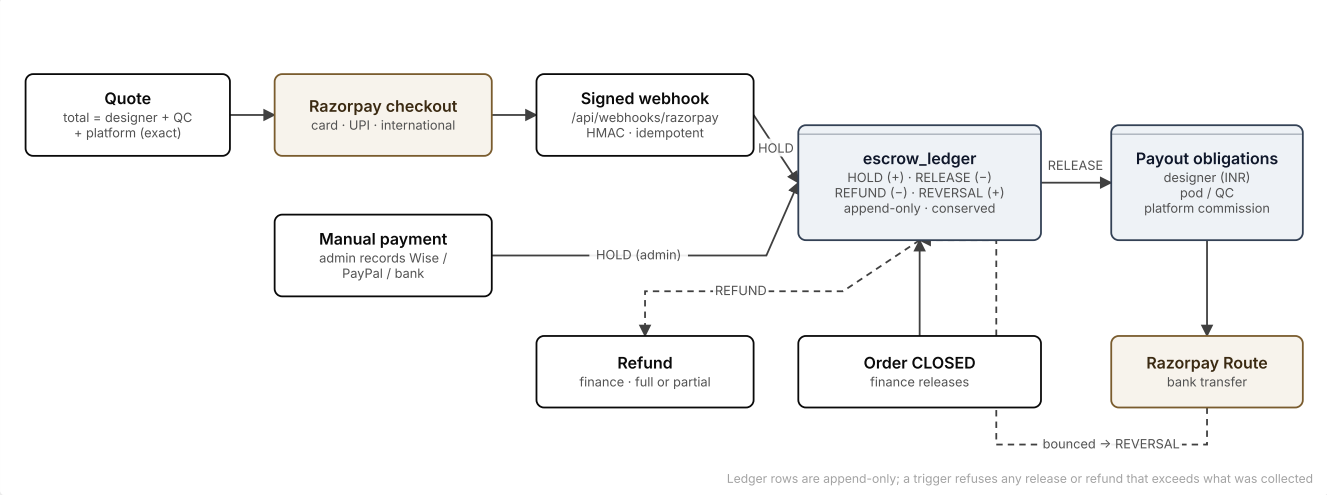
The 17 states and the legal moves between them.



Order state machine

17.5 Money flow and escrow

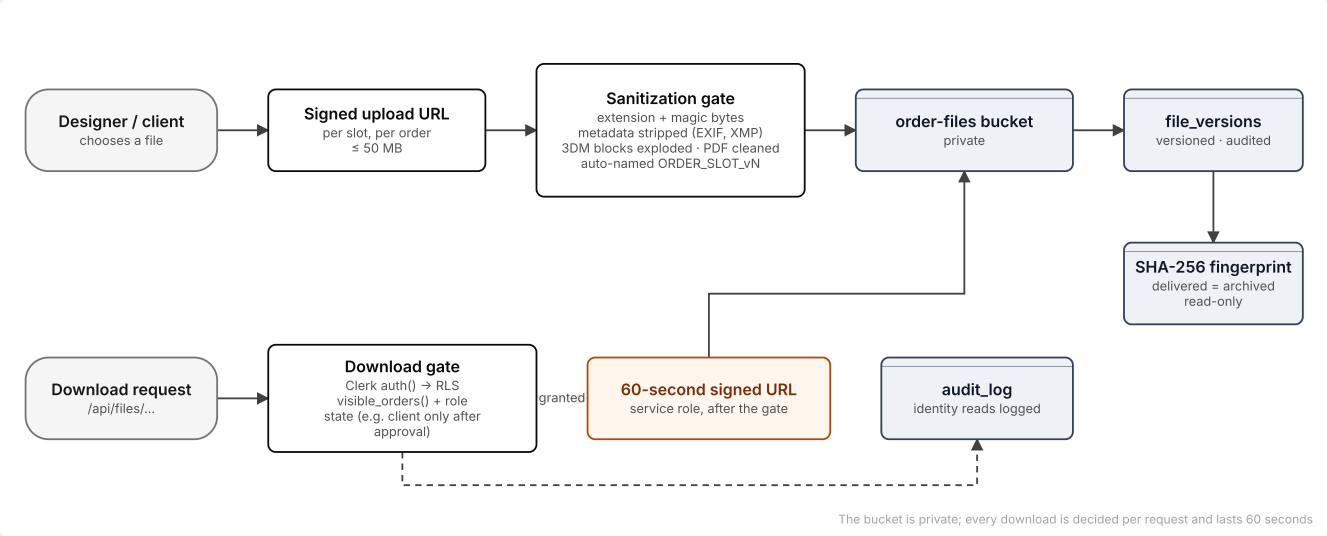
Where the money is at every step.



Money flow and escrow

17.6 File pipeline and IP protection

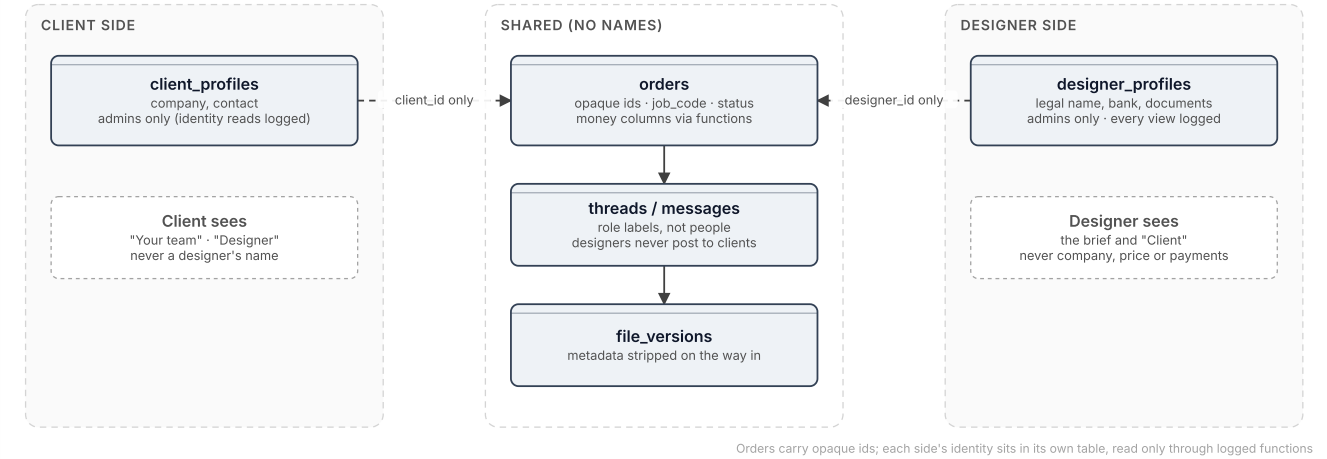
How CAD files are cleaned, stored and released.



File pipeline and IP protection

17.7 Anonymity wall

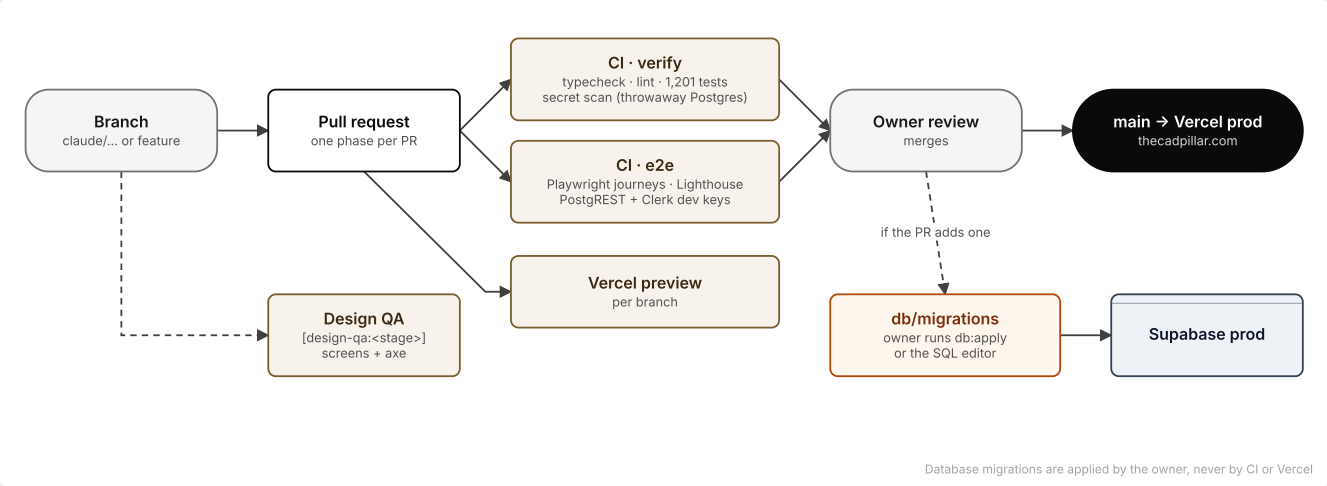
How client and designer identities are kept apart.



Anonymity wall (double blind)

17.8 Delivery pipeline

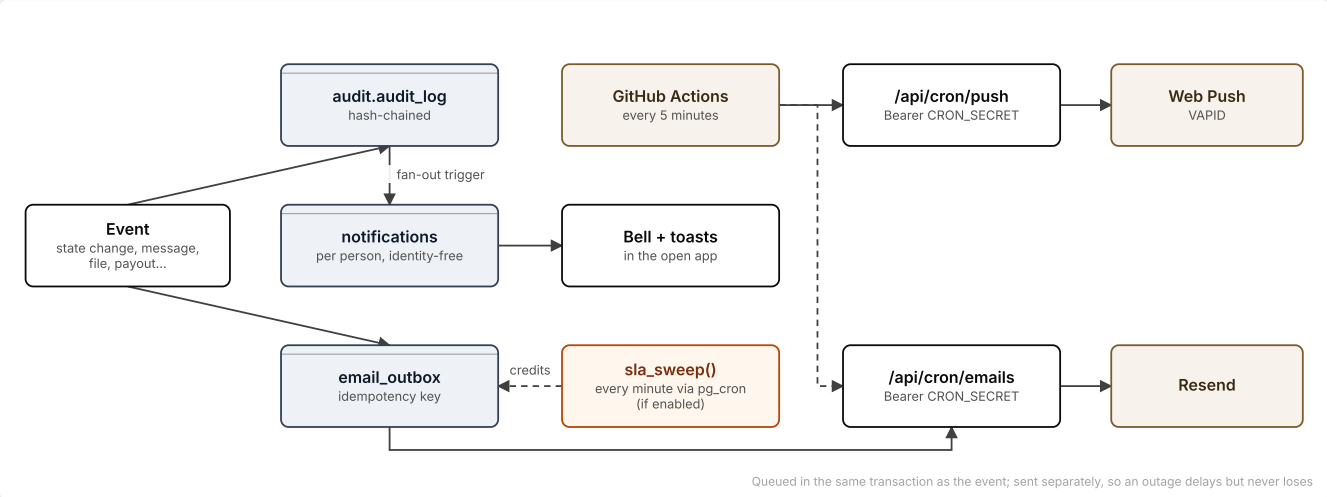
From a code change to production.



Delivery pipeline

17.9 Notifications, push and email

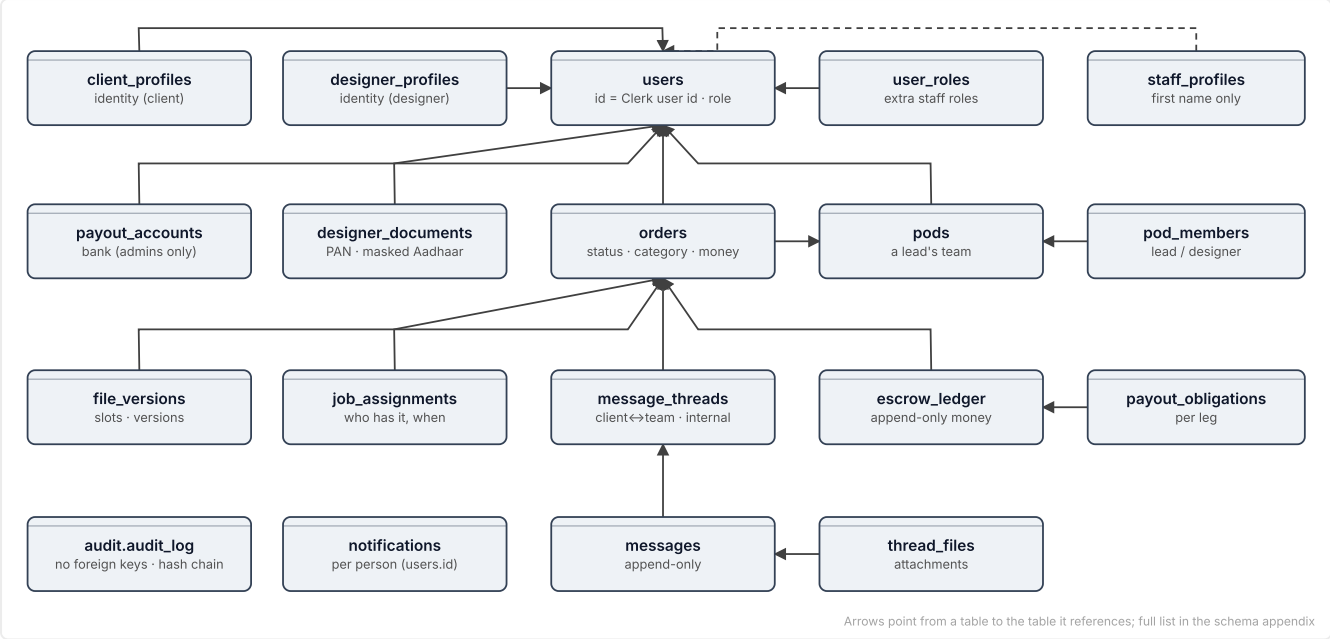
What runs on its own.



Notifications, push and email

17.10 Core data model

The central tables and their relationships.



Core data model

CHAPTER 18

Glossary of technical terms

Plain-language definitions of the terms used in this handbook and in the codebase.

Term	Meaning
3DM	Rhino's native CAD file format. Delivered with the STL. The platform rebuilds it to remove who made it, and explodes blocks.
Anon key	Supabase's public key, used by every visitor's browser. Not secret. It grants nothing on its own: row-level security decides.
API	Application Programming Interface: a way for programs to talk to each other. TCP's internal API is its server actions and `/api` routes.
App Router	The Next.js way of mapping folders in `app/` to web addresses, with server-rendered pages.
Append-only	A table that only ever gains rows; existing rows cannot be changed or deleted (enforced by triggers). Used for the audit log, the ledger, messages and certificates.
Audit log	`audit.audit_log` : the permanent record of important events, hash-chained so tampering is detectable.
Authentication	Proving who someone is (sign-in). Done by Clerk.
Authorisation	Deciding what a signed-in person may do. Done by the database.
Branch	A separate line of code changes in Git, merged into `main` when approved.
Bucket	A container for files in Supabase Storage. All of TCP's are private.
CI (continuous integration)	Automatic checks (type-checking, lint, tests, secret scan, browser journeys) run by GitHub on every pull request.
Clerk	The sign-in provider. Holds accounts and passwords; issues session tokens.
CNAME	A DNS record that points one name at another (for example `clerk.thecadpillar.com` → Clerk's servers).
Cron	A schedule for running a job repeatedly (for example every 5 minutes).
CSP (Content-Security-Policy)	A browser header that limits where scripts and data may load from; a defence against injected scripts.
CSRF	Cross-site request forgery: tricking a signed-in browser into sending a request. Server Actions check the request's origin to prevent it.
Database migration	A numbered SQL file that changes the database structure. Applied once, in order, never edited afterwards.
Definer function	See *SECURITY DEFINER*.
Deployment	One built version of the app on Vercel. Production is the deployment serving `thecadpillar.com`.
DKIM	An email signature proving a message really came from your domain.
DMARC	A DNS policy telling receivers what to do with email that fails SPF/DKIM for your domain.

Term	Meaning
DNS	The internet's address book, turning names into server addresses. Managed in Vercel for TCP.
E2E (end-to-end) test	A test that drives a real browser through a whole journey (for example brief → quote → pay → design → QC → approve).
Enum	A database type with a fixed list of allowed values (for example the 17 order statuses).
Environment variable	A setting provided to the app at run time (keys, URLs, switches), stored in Vercel, not in code.
Escrow	Holding a client's payment until the work is approved, then releasing it to the payees.
Feature flag	A switch that turns a feature on or off without a code change (`feature_flags`).
Fingerprint (SHA-256)	A unique code computed from a file's bytes. Changing one byte changes it. Used to prove delivered files are unchanged.
HMAC	A signature made with a shared secret; proves a message (for example a Razorpay webhook) is genuine and untampered.
HSTS	A header telling browsers to always use HTTPS for the site.
Idempotent	Safe to repeat: doing it twice has the same effect as once (for example processing the same webhook twice).
JWT	JSON Web Token: a signed, tamper-proof digital pass that carries who you are (`sub`).
JWKS	The public keys used to check a JWT's signature, published by Clerk.
Magic bytes	The first bytes of a file that reveal its real type, regardless of its name.
Metadata	Hidden information inside a file (author, software, GPS location, dates). Stripped on upload.
Middleware	Code that runs before every request (`middleware.ts`), deciding whether sign-in is needed.
Monolith	One application containing all features, deployed as one unit.
Next.js	The web framework TCP is built with (React-based, by Vercel).
npm	The package manager for JavaScript; `package.json` lists the libraries used.
OAuth	The standard behind "Continue with Google".
pg_cron	A Postgres extension that runs SQL on a schedule; used for the SLA sweep.
PITR	Point-in-time recovery: restoring a database to any moment within a window.
Pod	A team led by a pod lead (QC), containing designers.
Policy (RLS policy)	A rule on a table saying which rows a role may read or write.
Postgres	The database engine Supabase runs.
PostgREST	Supabase's Data API: turns database tables and functions into web endpoints, applying RLS.
Preview deployment	A temporary Vercel deployment of a branch, for review before merging.

Term	Meaning
Pull request (PR)	A proposal to merge a branch into `main`, with checks and review.
PWA	Progressive Web App: a website that installs to a phone's home screen and can work offline and receive push notifications.
Realtime	Supabase's live channels; TCP uses them to tell an open chat "something changed".
Revalidate	Telling Next.js to refresh cached pages after data changes.
RLS (row-level security)	Database rules that filter every query row by row, by who is asking. The core of TCP's security.
Role	A person's function in the system: Client, Designer, QC (pod lead), Sales, Ops, Finance, Admin.
Rollback	Returning to a previous working version (instant for the app on Vercel; a new migration for the database).
Route handler	A server endpoint under `app/api/…` (downloads, webhook, cron).
RPO / RTO	Recovery point objective (how much data may be lost) and recovery time objective (how long recovery may take).
Search path	Where Postgres looks for unqualified names. Definer functions pin it so callers cannot substitute look-alike tables.
Secret	A value that grants access (passwords, private keys, API secrets). Never in code, chat or screenshots.
SECURITY DEFINER	A database function that runs with its owner's rights, so it can do privileged work. It must check the caller itself.
Server Action	A server function called by a form in the portal (Next.js).
Server Component	A React component rendered on the server; sends HTML, never secrets.
Service role	Supabase's all-powerful key that bypasses RLS. Server-only, used only after the app's own checks.
Signed URL	A file link that works only for a short time (60 seconds at TCP).
SLA	Service level agreement: TCP's promise of a first look within 6 hours of confirmation (standard tier).
SPF	A DNS record listing which servers may send email for your domain.
SQL	The language used to query and change the database.
STL	A mesh 3D file format used for printing and casting; delivered with the 3DM.
Supabase	The managed platform providing TCP's Postgres database, file storage and realtime.
Trigger	Database code that runs automatically when rows change (for example to enforce money conservation).
TypeScript	JavaScript with types; catches many mistakes before the code runs.
Vercel	TCP's hosting provider for the Next.js app, and its DNS host.
VAPID	The key pair that identifies TCP to browser push services.
Webhook	A message a provider sends to TCP when something happens (for example Razorpay: payment captured).
Zod	A library that validates the shape of data coming into the server.

CHAPTER 19

Audit method, assumptions and missing access

This chapter records how the findings in this handbook were produced, what was verified and how, what could not be verified, and why. It exists so a future reviewer can repeat the audit and knows exactly where its conclusions stop.

19.1 Scope

- **Code:** the whole repository `simran313/GetCAD` at the audited revision (`main` after pull request #137; the short SHA is printed on the cover and in every generated appendix).
- **Database:** a database built from `db/migrations` (67 files) and `db/policies` (46 files) in a throwaway local Postgres 16. The production database was **not** connected to or queried. Production facts come from the owner's read-only health checks on 13 November 2026.
- **Live site:** public, read-only HTTP requests to `https://thecadpillar.com` (pages, headers, `/api/health`, robots, sitemap) and public DNS lookups.
- **Out of scope:** penetration testing against production, load testing, third-party providers' internal security, and legal review.

19.2 Method

Step	How	Output
Inventory	Walked `app/` for pages and route handlers; found every `"use server"` file; read `core/auth/session.ts` for guards	Appendices C and D
Database catalogue	Applied all migrations and policies to a scratch database and read `pg_class`, `pg_policies`, `pg_proc`, grants and triggers	Appendices A and B
RLS coverage	`relrowsecurity` for all 69 tables	69/69 on
Definer hardening	`prosecdef` with `proconfig` containing `search_path`	254/254 pinned
Grants	`has_function_privilege` / `has_schema_privilege` for `anon` and `authenticated`	SEC-06, SEC-11
Caller checks	Read the source of sampled definer functions (brief, references, files, money, identity, admin)	Checks present in all sampled
Service-role usage	Found all 36 files that create the service-role client; read the guard before each use	Inventory table, ch. 9
Webhook and cron	Read the signature and bearer checks (constant time) and their tests	Controls verified
Files	Read the sanitisation gate, metadata stripper, 3DM rebuild, PDF cleaner and download gate	Ch. 8
Headers	`curl -I https://thecadpillar.com/`	HSTS and the other headers present; CSP framing only

Step	How	Output
Secrets	`npm run secret-scan`; a pattern scan of the full git history for Clerk, Razorpay, Supabase JWT, Postgres URL, Resend and private-key formats; a check of tracked env files	Clean
Dependencies	`npm audit --omit=dev` and `npm audit` (13 Nov 2026)	SEC-01, 04, 05, 12
Email domain	Public DNS (TXT at apex and `_dmarc`, DKIM selectors)	SEC-07
Remediation test	The drafted grant-hardening migration was applied as a temporary migration, and the full database test suite run (58 files, 607 tests, all passing); then removed	Restricted annex
Tests	`npm run ci` on the audited branch	See the verification results in the pull request

19.3 Verified facts (with evidence)

- All 69 tables have RLS enabled; 43 policies exist (Appendix A).
- 285 functions, 254 SECURITY DEFINER, all with a pinned `search_path` (Appendix B).
- `anon` and `authenticated` have no `USAGE` on schema `audit`.
- 73 pages, 20 route handlers, 92 server actions in 31 files (Appendices C and D).
- Signed URL lifetime 60 s; preview tokens 192-bit and hashed; webhook HMAC in constant time; cron bearer in constant time.
- Live headers include HSTS `max-age=63072000`.
- No secrets in the working tree or the git history (patterns listed above).
- Dependency advisories as listed in SEC-01, 04, 05, 12 on 13 November 2026.
- 34 configuration variable names are read by the code (Appendix E).

19.4 Assumptions

1. The production database was built from exactly the migrations and policies in the repository, in order, with nothing changed by hand. `scripts/apply-migrations.mjs` records applied files in `public.schema_migrations`; the owner can confirm with `npm run db:status`.
2. Production uses the `tcp` Vercel project, deploying `main`.
3. Supabase Storage buckets were created private, as the code and migrations assume.
4. The owner's screenshots on 13 November 2026 show the actual current settings (Supabase plan, Vercel variables, Clerk configuration).
5. Business rules follow `docs/DECISIONS.md` v1.28 where a document and the code disagree. Where the code differs from DECISIONS, the code describes what *runs* and DECISIONS what is *intended*.

19.5 Not verified (missing infrastructure access)

Item	Why it matters	How the owner can verify
Supabase exposed Data API schemas	SEC-06 exploitability	Supabase → Project Settings → Data API → Exposed schemas = `public`, `graphql_public`

Item	Why it matters	How the owner can verify
`pg_cron` enabled with the SLA sweep scheduled	SLA promise	Supabase → Database → Extensions; `select * from cron.job;`
Supabase backups and PITR configuration	Recovery	Supabase → Database → Backups
Vercel environment scoping for Preview	SEC-08	Vercel → `tcp` → Settings → Environment Variables (Preview column)
Vercel function region	Performance	Vercel → `tcp` → Settings → Functions
GitHub branch protection and Actions secrets	Process safety	GitHub → Settings → Branches / Secrets
Team membership across providers	Access control	Each provider's members page
Razorpay live configuration and Route activation	Payments	Razorpay dashboard; `npm run verify:*`
Transactional email sender domain (`EMAIL_FROM`)	SEC-07	Vercel variables; Resend → Domains
Live behaviour of authenticated pages under load	Performance	Not tested in this audit

19.6 Repeating this audit

1. Check out the revision. Run `npm ci` and `npm run test:db:up`.
2. Run `node docs/document-generation/extract-reference.mjs` and compare the summary JSON (`docs/document-generation/generated-summary.json`) with the previous run: new tables without RLS, new definer functions without `search_path`, new `anon`-executable functions.
3. Grep for `createAdminSupabaseClient` and review any new uses.
4. Run `npm audit --omit=dev`.
5. Re-check headers, DNS and the "Not verified" table with the owner.
6. Update chapter 9's register and the restricted annex. Rebuild the handbook.

APPENDIX

Appendix A · Database schema reference

> Generated from the repository at commit `1c08b52`. Every row below is read from the code or from a database built from `db/migrations` and `db/policies`, not written by hand.

Summary

Measure	Count
Tables (public, app, audit)	69
Tables with row-level security ON	69
Tables with RLS forced on the owner too	69
Policies	43
Triggers	39
Views	0
Enum types	19
Migration files applied	113

Tables with RLS **off**: none.

Enum types

Type	Values
`cad_purpose`	CASTING, DIRECT_PRINT, RENDER_ONLY
`file_kind`	CLIENT_REFERENCE, RENDER, WEIGHT_CHART, DIAMOND_DETAILS, TECHNICAL_SHEET, STL, RHINO_3DM, SUMMARY_SHEET, OTHER
`finish_type`	HIGH_POLISH, MATTE, BRUSHED, HAMMERED, MIXED
`intake_mode`	full, quick
`job_tier`	standard, complex
`metal_colour`	YELLOW, WHITE, ROSE, TWO_TONE, TRI_COLOUR, PLATINUM, SILVER
`order_source`	portal, whatsapp, email, phone
`order_status`	DRAFT, SUBMITTED, QUOTED, PAYMENT_HELD, ASSIGNED, IN_PROGRESS, DESIGNER_SUBMITTED, QC_REVIEW, REVISION_REQUESTED, CLIENT_PREVIEW, APPROVED, DELIVERED, CLOSED, PAYOUT_RELEASED, CANCELLED, DISPUTED, REFUNDED
`output_format`	THREE_DM, STL, BOTH, STEP
`payment_method`	razorpay, manual, wise, paypal, bank
`priority_tier`	STANDARD, EXPRESS, RUSH

Type	Values
`product_kind`	RING, BAND, PENDANT, EARRING, BRACELET, BANGLE, NECKLACE, BROOCH, OTHER
`role`	CLIENT, DESIGNER, QC, SALES, OPS, FINANCE, ADMIN, SUPER_ADMIN
`service_kind`	cad, render, cad_render
`setting_type`	PRONG_4, PRONG_6, BEZEL, HALO, PAVE, CHANNEL, TENSION, FLUSH, OTHER, AS_PER_REFERENCE
`sla_status`	pending, running, met, breached, breached_credited
`stone_shape`	ROUND, OVAL, CUSHION, PRINCESS, EMERALD, PEAR, MARQUISE, RADIANT, ASSCHER, HEART, TRILLION, BAGUETTE, OTHER, AS_PER_REFERENCE
`stone_supply`	CLIENT, DESIGNER, PLATFORM, NONE
`user_status`	PENDING, ACTIVE, SUSPENDED

Views

View	security_invoker

Tables

audit.audit_log

RLS: **on** (forced). Table grants: none to anon/authenticated.

Column	Type	Null	Default
`id`	text	no	(gen_random_uuid())::text
`seq`	bigint	no	nextval('audit.audit_log_seq'::regclass)
`actor_id`	text	yes	
`actor_role`	role	yes	
`action`	text	no	
`entity_type`	text	no	
`entity_id`	text	yes	
`payload`	jsonb	no	'{}'::jsonb
`is_identity_read`	boolean	no	false
`created_at`	timestamp with time zone	no	now()
`prev_hash`	text	yes	
`hash`	text	no	

Constraint	Kind	Definition
`audit_log_pkey`	primary key	`PRIMARY KEY (id)`
`audit_log_seq_key`	unique	`UNIQUE (seq)`

Policies: none (with RLS on, this table is closed to anon and authenticated; only SECURITY DEFINER functions and the service role reach it).

Triggers:

- `audit_log_chain`: `BEFORE INSERT ON audit.audit_log FOR EACH ROW EXECUTE FUNCTION audit.chain_before_insert()`
- `audit_log_fanout`: `AFTER INSERT ON audit.audit_log FOR EACH ROW EXECUTE FUNCTION app.fanout_notifications()`
- `audit_log_no_delete`: `BEFORE DELETE ON audit.audit_log FOR EACH ROW EXECUTE FUNCTION audit.prevent_mutation()`
- `audit_log_no_truncate`: `BEFORE TRUNCATE ON audit.audit_log FOR EACH STATEMENT EXECUTE FUNCTION audit.prevent_mutation()`
- `audit_log_no_update`: `BEFORE UPDATE ON audit.audit_log FOR EACH ROW EXECUTE FUNCTION audit.prevent_mutation()`

public.agreement_acceptances

RLS: **on** (forced). Table grants: authenticated: SELECT. Column-level grants: authenticated SELECT on 5 column(s).

Column	Type	Null	Default
`id`	text	no	(gen_random_uuid())::text
`agreement_id`	text	no	
`user_id`	text	no	
`content_sha256`	text	no	
`accepted_at`	timestamp with time zone	no	now()

Constraint	Kind	Definition
`agreement_acceptances_agreement_id_fkey`	foreign key	`FOREIGN KEY (agreement_id) REFERENCES agreement_documents(id) ON DELETE RESTRICT`
`agreement_acceptances_user_id_fkey`	foreign key	`FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE RESTRICT`
`agreement_acceptances_pkey`	primary key	`PRIMARY KEY (id)`
`agreement_acceptances_agreement_id_user_id_key`	unique	`UNIQUE (agreement_id, user_id)`

Policies:

Policy	Command	Roles	USING	WITH CHECK
`agreement_acceptances_read_own`	SELECT	{authenticated}	`(user_id = app.current_clerk_id())`	

Triggers:

- `agreement_acceptances_no_delete`: `BEFORE DELETE ON public.agreement_acceptances FOR EACH ROW EXECUTE FUNCTION app.reject_mutation()`

- `agreement_acceptances_no_update`: `BEFORE UPDATE ON public.agreement_acceptances FOR EACH ROW EXECUTE FUNCTION app.reject_mutation()`

public.agreement_documents

RLS: **on** (forced). Table grants: authenticated: SELECT. Column-level grants: authenticated SELECT on 7 column(s).

Column	Type	Null	Default
`id`	text	no	
`kind`	text	no	
`version`	text	no	
`title`	text	no	
`body`	text	no	
`content_sha256`	text	no	
`published_at`	timestamp with time zone	no	now()

Constraint	Kind	Definition
`agreement_documents_pkey`	primary key	`PRIMARY KEY (id)`
`agreement_documents_kind_version_key`	unique	`UNIQUE (kind, version)`

Policies:

Policy	Command	Roles	USING	WITH CHECK
`agreement_documents_read`	SELECT	{authenticated}	`true`	

Triggers:

- `agreement_documents_no_delete`: `BEFORE DELETE ON public.agreement_documents FOR EACH ROW EXECUTE FUNCTION app.reject_mutation()`
- `agreement_documents_no_update`: `BEFORE UPDATE ON public.agreement_documents FOR EACH ROW EXECUTE FUNCTION app.reject_mutation()`

public.app_settings

RLS: **on** (forced). Table grants: authenticated: SELECT. Column-level grants: authenticated SELECT on 5 column(s).

Column	Type	Null	Default
`key`	text	no	
`value`	integer	yes	
`description`	text	no	
`updated_by`	text	yes	
`updated_at`	timestamp with time zone	no	now()

Constraint	Kind	Definition
`app_settings_key_check`	check	`CHECK ((key ~ '^[a-z][a-z0-9_]*$'::text))`

Constraint	Kind	Definition
`app_settings_updated_by_fkey`	foreign key	`FOREIGN KEY (updated_by) REFERENCES users(id) ON DELETE RESTRICT`
`app_settings_pkey`	primary key	`PRIMARY KEY (key)`

Policies:

Policy	Command	Roles	USING	WITH CHECK
`app_settings_staff_read`	SELECT	{authenticated}	`app.is_staff()`	

public.certificates

RLS: **on** (forced). Table grants: none to anon/authenticated.

Column	Type	Null	Default
`order_id`	text	no	
`version`	integer	no	
`object_key`	text	no	
`sha256`	text	no	
`size_bytes`	integer	no	
`created_at`	timestamp with time zone	no	now()

Constraint	Kind	Definition
`certificates_sha256_check`	check	`CHECK ((sha256 ~ '^[0-9a-f]{64}$'::text))`
`certificates_size_bytes_check`	check	`CHECK ((size_bytes > 0))`
`certificates_version_check`	check	`CHECK ((version >= 1))`
`certificates_order_id_fkey`	foreign key	`FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE RESTRICT`
`certificates_pkey`	primary key	`PRIMARY KEY (order_id, version)`
`certificates_object_key_key`	unique	`UNIQUE (object_key)`

Policies: none (with RLS on, this table is closed to anon and authenticated; only SECURITY DEFINER functions and the service role reach it).

Triggers:

- `certificates_immutable`: `BEFORE DELETE OR UPDATE ON public.certificates FOR EACH ROW EXECUTE FUNCTION app.certificates_immutable()`

public.checklist_items

RLS: **on** (forced). Table grants: authenticated: SELECT. Column-level grants: authenticated SELECT on 9 column(s).

Column	Type	Null	Default
`id`	smallint	no	

Column	Type	Null	Default
group_name	text	no	
label	text	no	
flag	text	yes	
is_signoff	boolean	no	false
stage	text	no	'LEGACY'::text
standard	text	yes	
position	smallint	no	
active	boolean	no	true

Constraint	Kind	Definition
checklist_items_flag_check	check	CHECK ((flag = ANY (ARRAY['MUST'::text, 'STOP_IF_UNSURE'::text, 'CRITICAL'::text])))
checklist_items_id_check	check	CHECK (((id >= 1) AND (id <= 999)))
checklist_items_stage_check	check	CHECK ((stage = ANY (ARRAY['LEGACY'::text, 'DESIGNER'::text, 'MANAGER'::text])))
checklist_items_pkey	primary key	PRIMARY KEY (id)

Policies:

Policy	Command	Roles	USING	WITH CHECK
checklist_items_read	SELECT	{authenticated}	true	

public.client_accounts

RLS: **on** (forced). Table grants: none to anon/authenticated. Column-level grants: authenticated SELECT on 4 column(s).

Column	Type	Null	Default
user_id	text	no	
code_prefix	text	no	
code_n	integer	no	
client_code	text (generated)	yes	
first_design_order_id	text	yes	
created_at	timestamp with time zone	no	now()

Constraint	Kind	Definition
client_accounts_code_n_check	check	CHECK ((code_n >= 1))
client_accounts_code_prefix_check	check	CHECK ((code_prefix ~ '^[A-Z]{2}$'::text))

Constraint	Kind	Definition
client_accounts_first_design_order_id_fkey	foreign key	FOREIGN KEY (first_design_order_id) REFERENCES orders(id) ON DELETE RESTRICT
client_accounts_user_id_fkey	foreign key	FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE RESTRICT
client_accounts_pkey	primary key	PRIMARY KEY (user_id)
client_accounts_client_code_key	unique	UNIQUE (client_code)
client_accounts_code_prefix_code_n_key	unique	UNIQUE (code_prefix, code_n)

Policies:

Policy	Command	Roles	USING	WITH CHECK
client_accounts_read	SELECT	{authenticated}	((user_id = app.current_clerk_id()) OR (app.current_roles() && ARRAY['SALES'::role, 'OPS'::role, 'FINANCE'::role]) OR app.client_in_my_pods(user_id))	

public.client_credits

RLS: **on** (forced). Table grants: none to anon/authenticated. Column-level grants: authenticated SELECT on 11 column(s).

Column	Type	Null	Default
id	text	no	(gen_random_uuid())::text
client_id	text	no	
order_id	text	no	
kind	text	no	
amount_cents	integer	no	
currency	text	no	
status	text	no	'pending'::text
decided_amount_cents	integer	yes	
decision_reason	text	yes	
decided_by	text	yes	
decided_at	timestamp with time zone	yes	
applied_to_order_id	text	yes	
created_at	timestamp with time zone	no	now()

Constraint	Kind	Definition
client_credits_amount_cents_check	check	CHECK ((amount_cents > 0))

Constraint	Kind	Definition
client_credits_check	check	CHECK (((status = 'pending'::text) OR (decided_at IS NOT NULL)))
client_credits_check1	check	CHECK (((status <> ALL (ARRAY['partial'::text, 'rejected'::text])) OR (decision_reason IS NOT NULL)))
client_credits_check2	check	CHECK (((status <> 'partial'::text) OR (decided_amount_cents < amount_cents)))
client_credits_currency_check	check	CHECK ((char_length(currency) = 3))
client_credits_decided_amount_cents_check	check	CHECK (((decided_amount_cents IS NULL) OR (decided_amount_cents >= 0)))
client_credits_decision_reason_check	check	CHECK (((decision_reason IS NULL) OR ((char_length(btrim(decision_reason)) >= 1) AND (char_length(btrim(decision_reason)) <= 1000))))
client_credits_kind_check	check	CHECK ((kind = ANY (ARRAY['sla_credit'::text, 'claim_credit'::text])))
client_credits_status_check	check	CHECK ((status = ANY (ARRAY['pending'::text, 'apply_next'::text, 'refunded'::text, 'partial'::text, 'rejected'::text, 'applied'::text])))
client_credits_applied_to_order_id_fkey	foreign key	FOREIGN KEY (applied_to_order_id) REFERENCES orders(id) ON DELETE RESTRICT
client_credits_client_id_fkey	foreign key	FOREIGN KEY (client_id) REFERENCES users(id) ON DELETE RESTRICT
client_credits_decided_by_fkey	foreign key	FOREIGN KEY (decided_by) REFERENCES users(id) ON DELETE RESTRICT
client_credits_order_id_fkey	foreign key	FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE RESTRICT
client_credits_pkey	primary key	PRIMARY KEY (id)
client_credits_order_id_kind_key	unique	UNIQUE (order_id, kind)

Policies:

Policy	Command	Roles	USING	WITH CHECK
client_credits_read	SELECT	{authenticated}	(((client_id = app.current_clerk_id()) AND (status = ANY (ARRAY['apply_next'::text, 'refunded'::text, 'partial'::text, 'applied'::text]))) OR app.is_admin() OR app.has_role('FINANCE'::role))	

Triggers:

- `client_credits_emails`: `AFTER UPDATE OF status ON public.client_credits FOR EACH ROW EXECUTE FUNCTION app.credit_emails()`

public.client_managers

RLS: **on** (forced). Table grants: none to anon/authenticated.

Column	Type	Null	Default
`client_id`	text	no	
`manager_id`	text	no	
`set_by`	text	yes	
`set_at`	timestamp with time zone	no	now()

Constraint	Kind	Definition
`client_managers_client_id_fkey`	foreign key	`FOREIGN KEY (client_id) REFERENCES users(id) ON DELETE RESTRICT`
`client_managers_manager_id_fkey`	foreign key	`FOREIGN KEY (manager_id) REFERENCES users(id) ON DELETE RESTRICT`
`client_managers_set_by_fkey`	foreign key	`FOREIGN KEY (set_by) REFERENCES users(id) ON DELETE RESTRICT`
`client_managers_pkey`	primary key	`PRIMARY KEY (client_id)`

Policies: none (with RLS on, this table is closed to anon and authenticated; only SECURITY DEFINER functions and the service role reach it).

public.client_master_instructions

RLS: **on** (forced). Table grants: none to anon/authenticated.

Column	Type	Null	Default
`client_id`	text	no	
`body`	text	no	
`updated_by`	text	no	
`updated_at`	timestamp with time zone	no	now()

Constraint	Kind	Definition
`client_master_instructions_body_check`	check	`CHECK (((char_length(btrim(body)) >= 1) AND (char_length(btrim(body)) <= 4000)))`
`client_master_instructions_client_id_fkey`	foreign key	`FOREIGN KEY (client_id) REFERENCES users(id) ON DELETE RESTRICT`
`client_master_instructions_updated_by_fkey`	foreign key	`FOREIGN KEY (updated_by) REFERENCES users(id) ON DELETE RESTRICT`
`client_master_instructions_pkey`	primary key	`PRIMARY KEY (client_id)`

Policies: none (with RLS on, this table is closed to anon and authenticated; only SECURITY DEFINER functions and the service role reach it).

public.client_profiles

RLS: **on** (forced). Table grants: anon: SELECT; authenticated: SELECT. Column-level grants: anon SELECT on 10 column(s); authenticated SELECT on 10 column(s).

Column	Type	Null	Default
`id`	text	no	
`user_id`	text	no	
`legal_name`	text	no	
`email`	text	no	
`phone`	text	yes	
`billing_address`	text	yes	
`created_at`	timestamp with time zone	no	now()
`deleted_at`	timestamp with time zone	yes	
`business_name`	text	yes	
`logo_object_key`	text	yes	

Constraint	Kind	Definition
`client_profiles_business_name_check`	check	`CHECK (((business_name IS NULL) OR ((char_length(btrim(business_name)) >= 1) AND (char_length(btrim(business_name)) <= 120))))`
`client_profiles_user_id_fkey`	foreign key	`FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE RESTRICT`
`client_profiles_pkey`	primary key	`PRIMARY KEY (id)`
`client_profiles_user_id_key`	unique	`UNIQUE (user_id)`

Policies:

Policy	Command	Roles	USING	WITH CHECK
`client_profiles_self_select`	SELECT	{authenticated}	`(user_id = app.current_clerk_id())`	

Triggers:

- `client_profiles_welcome`: `AFTER INSERT ON public.client_profiles FOR EACH ROW EXECUTE FUNCTION app.welcome_email()`

public.deliverable_files

RLS: **on** (forced). Table grants: none to anon/authenticated.

Column	Type	Null	Default
file_version_id	text	no	
order_id	text	no	
version	integer	no	
slot	text	no	
sha256	text	no	
size_bytes	integer	no	
recorded_at	timestamp with time zone	no	now()
archived_at	timestamp with time zone	yes	

Constraint	Kind	Definition
deliverable_files_sha256_check	check	CHECK ((sha256 ~ '^[0-9a-f]{64}$'::text))
deliverable_files_size_bytes_check	check	CHECK ((size_bytes > 0))
deliverable_files_file_version_id_fkey	foreign key	FOREIGN KEY (file_version_id) REFERENCES file_versions(id) ON DELETE RESTRICT
deliverable_files_order_id_fkey	foreign key	FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE RESTRICT
deliverable_files_pkey	primary key	PRIMARY KEY (file_version_id)

Policies: none (with RLS on, this table is closed to anon and authenticated; only SECURITY DEFINER functions and the service role reach it).

public.deliverable_slots

RLS: **on** (forced). Table grants: authenticated: SELECT. Column-level grants: authenticated SELECT on 4 column(s).

Column	Type	Null	Default
code	text	no	
label	text	no	
required	boolean	no	
position	smallint	no	

Constraint	Kind	Definition
deliverable_slots_code_check	check	CHECK ((code ~ '^[A-Z0-9_]{1,24}$'::text))
deliverable_slots_pkey	primary key	PRIMARY KEY (code)
deliverable_slots_position_key	unique	UNIQUE ("position")

Policies:

Policy	Command	Roles	USING	WITH CHECK
deliverable_slots_read	SELECT	{authenticated}	true	

public.deliverables

RLS: **on** (forced). Table grants: none to anon/authenticated. Column-level grants: authenticated SELECT on 6 column(s).

Column	Type	Null	Default
order_id	text	no	
slot	text	no	
version	integer	no	
file_version_id	text	no	
file_name	text	no	
created_at	timestamp with time zone	no	now()

Constraint	Kind	Definition
deliverables_file_name_check	check	CHECK (((char_length(file_name) >= 1) AND (char_length(file_name) <= 120)))
deliverables_version_check	check	CHECK ((version >= 1))
deliverables_file_version_id_fkey	foreign key	FOREIGN KEY (file_version_id) REFERENCES file_versions(id) ON DELETE RESTRICT
deliverables_order_id_fkey	foreign key	FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE RESTRICT
deliverables_slot_fkey	foreign key	FOREIGN KEY (slot) REFERENCES deliverable_slots(code)
deliverables_pkey	primary key	PRIMARY KEY (order_id, slot, version)
deliverables_file_version_id_key	unique	UNIQUE (file_version_id)

Policies:

Policy	Command	Roles	USING	WITH CHECK
deliverables_read	SELECT	{authenticated}	(app.is_assigned_designer(order_id) OR (EXISTS (SELECT 1 FROM orders o WHERE (o.id = deliverables.order_id))))	

public.designer_applications

RLS: **on** (forced). Table grants: none to anon/authenticated.

Column	Type	Null	Default
id	text	no	(gen_random_uuid())::text
full_name	text	no	
email	text	no	
phone	text	no	

Column	Type	Null	Default
country	text	no	
years_experience	integer	no	
primary_software	text	no	
categories	_text	no	
portfolio_url	text	yes	
portfolio_file_keys	_text	yes	
status	text	no	'PENDING_REVIEW'::text
created_at	timestamp with time zone	no	now()
reviewed_at	timestamp with time zone	yes	
reviewed_by	text	yes	
review_notes	text	yes	

Constraint	Kind	Definition
designer_applications_categories_check	check	CHECK (((cardinality(categories) >= 1) AND (categories <@ ARRAY['RINGS'::text, 'PENDANTS'::text, 'EARRINGS'::text, 'BRACELETS'::text, 'BANGLES'::text])))
designer_applications_country_check	check	CHECK (((char_length(country) >= 1) AND (char_length(country) <= 100)))
designer_applications_email_check	check	CHECK ((email ~ '^[^@\s]+@[^@\s]+\.[^@\s]+$'::text))
designer_applications_full_name_check	check	CHECK (((char_length(full_name) >= 1) AND (char_length(full_name) <= 200)))
designer_applications_phone_check	check	CHECK (((char_length(phone) >= 3) AND (char_length(phone) <= 40)))
designer_applications_portfolio_file_keys_check	check	CHECK (((portfolio_file_keys IS NULL) OR ((cardinality(portfolio_file_keys) >= 2) AND (cardinality(portfolio_file_keys) <= 3))))
designer_applications_portfolio_url_check	check	CHECK (((portfolio_url IS NULL) OR (char_length(portfolio_url) <= 2000)))
designer_applications_portfolio_xor	check	CHECK ((((portfolio_url IS NOT NULL) AND (portfolio_file_keys IS NULL)) OR ((portfolio_url IS NULL) AND (portfolio_file_keys IS NOT NULL))))
designer_applications_primary_software_check	check	CHECK ((primary_software = ANY (ARRAY['RHINO'::text, 'MATRIX'::text, '3DESIGN'::text, 'OTHER'::text])))
designer_applications_review_notes_check	check	CHECK (((review_notes IS NULL) OR (char_length(review_notes) <= 2000)))
designer_applications_status_check	check	CHECK ((status = ANY (ARRAY['PENDING_REVIEW'::text, 'ACCEPTED'::text, 'REJECTED'::text])))

Constraint	Kind	Definition
designer_applications_years_experience_check	check	CHECK (((years_experience >= 0) AND (years_experience <= 60)))
designer_applications_reviewed_by_fkey	foreign key	FOREIGN KEY (reviewed_by) REFERENCES users(id) ON DELETE RESTRICT
designer_applications_pkey	primary key	PRIMARY KEY (id)

Policies: none (with RLS on, this table is closed to anon and authenticated; only SECURITY DEFINER functions and the service role reach it).

public.designer_documents

RLS: **on** (forced). Table grants: none to anon/authenticated.

Column	Type	Null	Default
id	text	no	(gen_random_uuid())::text
user_id	text	no	
kind	text	no	
object_key	text	no	
file_name	text	no	
mime	text	no	
size_bytes	integer	no	
uploaded_by	text	no	
created_at	timestamp with time zone	no	now()
deleted_at	timestamp with time zone	yes	

Constraint	Kind	Definition
designer_documents_file_name_check	check	CHECK (((char_length(file_name) >= 1) AND (char_length(file_name) <= 200)))
designer_documents_kind_check	check	CHECK ((kind = ANY (ARRAY['PAN_CARD'::text, 'AADHAAR'::text, 'BANK_PROOF'::text, 'PHOTO'::text, 'OTHER'::text])))
designer_documents_mime_check	check	CHECK ((mime = ANY (ARRAY['image/jpeg'::text, 'image/png'::text, 'image/webp'::text, 'application/pdf'::text])))
designer_documents_size_bytes_check	check	CHECK (((size_bytes > 0) AND (size_bytes <= 10485760)))
designer_documents_uploaded_by_fkey	foreign key	FOREIGN KEY (uploaded_by) REFERENCES users(id) ON DELETE RESTRICT
designer_documents_user_id_fkey	foreign key	FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE RESTRICT
designer_documents_pkey	primary key	PRIMARY KEY (id)
designer_documents_object_key_key	unique	UNIQUE (object_key)

Policies: none (with RLS on, this table is closed to anon and authenticated; only SECURITY DEFINER functions and the service role reach it).

public.designer_profiles

RLS: **on** (forced). Table grants: anon: SELECT; authenticated: SELECT. Column-level grants: anon SELECT on 11 column(s); authenticated SELECT on 11 column(s).

Column	Type	Null	Default
`id`	text	no	
`user_id`	text	no	
`legal_name`	text	no	
`email`	text	no	
`country`	text	yes	
`created_at`	timestamp with time zone	no	now()
`deleted_at`	timestamp with time zone	yes	
`agreement_accepted_at`	timestamp with time zone	yes	
`agreement_version`	text	yes	
`designer_ref`	text	no	
`phone`	text	yes	

Constraint	Kind	Definition
`designer_profiles_phone_check`	check	`CHECK (((phone IS NULL) OR ((char_length(btrim(phone)) >= 6) AND (char_length(btrim(phone)) <= 30))))`
`designer_profiles_user_id_fkey`	foreign key	`FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE RESTRICT`
`designer_profiles_pkey`	primary key	`PRIMARY KEY (id)`
`designer_profiles_designer_ref_key`	unique	`UNIQUE (designer_ref)`
`designer_profiles_user_id_key`	unique	`UNIQUE (user_id)`

Policies:

Policy	Command	Roles	USING	WITH CHECK
`designer_profiles_self_select`	SELECT	{authenticated}	`(user_id = app.current_clerk_id())`	

Triggers:

- `designer_profiles_ref`: `BEFORE INSERT ON public.designer_profiles FOR EACH ROW EXECUTE FUNCTION app.set_designer_ref()`

public.designer_rates

RLS: **on** (forced). Table grants: none to anon/authenticated.

Column	Type	Null	Default
designer_id	text	no	
paise_per_minute	integer	no	
updated_by	text	yes	
updated_at	timestamp with time zone	no	now()

Constraint	Kind	Definition
designer_rates_paise_per_minute_check	check	CHECK (((paise_per_minute >= 1) AND (paise_per_minute <= 100000)))
designer_rates_designer_id_fkey	foreign key	FOREIGN KEY (designer_id) REFERENCES users(id) ON DELETE RESTRICT
designer_rates_updated_by_fkey	foreign key	FOREIGN KEY (updated_by) REFERENCES users(id) ON DELETE RESTRICT
designer_rates_pkey	primary key	PRIMARY KEY (designer_id)

Policies: none (with RLS on, this table is closed to anon and authenticated; only SECURITY DEFINER functions and the service role reach it).

public.disputes

RLS: **on** (forced). Table grants: none to anon/authenticated. Column-level grants: authenticated SELECT on 8 column(s).

Column	Type	Null	Default
id	text	no	(gen_random_uuid())::text
order_id	text	no	
raised_by	text	no	
reason	text	no	
status	text	no	'OPEN'::text
resolution	text	yes	
resolved_by	text	yes	
resolution_notes	text	yes	
created_at	timestamp with time zone	no	now()
resolved_at	timestamp with time zone	yes	

Constraint	Kind	Definition
disputes_reason_check	check	CHECK (((char_length(reason) >= 1) AND (char_length(reason) <= 5000)))
disputes_resolution_check	check	CHECK ((resolution = ANY (ARRAY['REWORK'::text, 'REFUND'::text])))

Constraint	Kind	Definition
disputes_resolution_shape	check	CHECK ((((status = 'OPEN'::text) AND (resolution IS NULL) AND (resolved_by IS NULL) AND (resolved_at IS NULL)) OR ((status = 'RESOLVED'::text) AND (resolution IS NOT NULL) AND (resolved_by IS NOT NULL) AND (resolved_at IS NOT NULL))))
disputes_status_check	check	CHECK ((status = ANY (ARRAY['OPEN'::text, 'RESOLVED'::text])))
disputes_order_id_fkey	foreign key	FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE RESTRICT
disputes_raised_by_fkey	foreign key	FOREIGN KEY (raised_by) REFERENCES users(id) ON DELETE RESTRICT
disputes_resolved_by_fkey	foreign key	FOREIGN KEY (resolved_by) REFERENCES users(id) ON DELETE RESTRICT
disputes_pkey	primary key	PRIMARY KEY (id)

Policies:

Policy	Command	Roles	USING	WITH CHECK
disputes_read	SELECT	{authenticated}	(EXISTS (SELECT 1 FROM orders o WHERE (o.id = disputes.order_id)))	

public.email_outbox

RLS: **on** (forced). Table grants: anon: SELECT; authenticated: SELECT. Column-level grants: anon SELECT on 12 column(s); authenticated SELECT on 12 column(s).

Column	Type	Null	Default
id	text	no	(gen_random_uuid())::text
template	text	no	
recipient_email	text	no	
payload	jsonb	no	'{}'::jsonb
status	text	no	'PENDING'::text
attempts	integer	no	0
idempotency_key	text	no	
provider_message_ref	text	yes	
failure_reason	text	yes	
created_at	timestamp with time zone	no	now()
updated_at	timestamp with time zone	no	now()
sent_at	timestamp with time zone	yes	

Constraint	Kind	Definition
email_outbox_attempts_check	check	CHECK ((attempts >= 0))
email_outbox_failed_has_reason	check	CHECK (((status <> 'FAILED'::text) OR (failure_reason IS NOT NULL)))
email_outbox_failure_reason_check	check	CHECK (((failure_reason IS NULL) OR (char_length(failure_reason) <= 500)))
email_outbox_recipient_email_check	check	CHECK ((recipient_email ~ '^[^@\s]+@[^@\s]+\.[^@\s]+$'::text))
email_outbox_sent_has_ref	check	CHECK (((status <> 'SENT'::text) OR ((provider_message_ref IS NOT NULL) AND (sent_at IS NOT NULL))))
email_outbox_status_check	check	CHECK ((status = ANY (ARRAY['PENDING'::text, 'SENDING'::text, 'SENT'::text, 'FAILED'::text])))
email_outbox_template_check	check	CHECK ((template = ANY (ARRAY['DESIGNER_APPLICATION_RECEIVED'::text, 'CONTACT_RECEIVED'::text, 'DESIGNER_APPLICATION_ACCEPTED'::text, 'DESIGNER_APPLICATION_REJECTED'::text, 'PAYOUT_SENT'::text, 'ORDER_PLACED_FOR_YOU'::text, 'WELCOME'::text, 'ORDER_RECEIVED'::text, 'QUOTE_READY'::text, 'QUOTE_REMINDER'::text, 'PAYMENT_RECEIVED'::text, 'ORDER_CONFIRMED'::text, 'FIRST_LOOK_READY'::text, 'REVISION_RECEIVED'::text, 'REVISED_DESIGN_READY'::text, 'PREVIEW_REMINDER'::text, 'DESIGN_APPROVED'::text, 'FILES_DELIVERED'::text, 'PROBLEM_RECEIVED'::text, 'ORDER_REFUNDED'::text, 'ORDER_CANCELLED'::text, 'ORDER_DELAYED'::text, 'CREDIT_DECIDED'::text, 'NEW_MESSAGE'::text, 'FEEDBACK_REQUEST'::text])))
email_outbox_pkey	primary key	PRIMARY KEY (id)
email_outbox_idempotency_key_key	unique	UNIQUE (idempotency_key)

Policies: none (with RLS on, this table is closed to anon and authenticated; only SECURITY DEFINER functions and the service role reach it).

public.email_preferences

RLS: **on** (forced). Table grants: none to anon/authenticated.

Column	Type	Null	Default
user_id	text	no	
messages	boolean	no	true

Column	Type	Null	Default
reminders	boolean	no	true
marketing	boolean	no	false
updated_at	timestamp with time zone	no	now()

Constraint	Kind	Definition
email_preferences_user_id_fkey	foreign key	FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE RESTRICT
email_preferences_pkey	primary key	PRIMARY KEY (user_id)

Policies: none (with RLS on, this table is closed to anon and authenticated; only SECURITY DEFINER functions and the service role reach it).

public.escrow_ledger

RLS: **on** (forced). Table grants: none to anon/authenticated. Column-level grants: authenticated SELECT on 10 column(s).

Column	Type	Null	Default
id	text	no	(gen_random_uuid())::text
order_id	text	no	
kind	text	no	
party	text	no	
amount	integer	no	
currency	text	no	
created_by	text	yes	
created_at	timestamp with time zone	no	now()
payee_id	text	yes	
external_ref	text	yes	
idempotency_key	text	yes	
fx_rate	numeric	yes	

Constraint	Kind	Definition
escrow_ledger_amount_check	check	CHECK ((amount > 0))
escrow_ledger_fx_rate_check	check	CHECK (((fx_rate IS NULL) OR (fx_rate > (0)::numeric)))
escrow_ledger_kind_check	check	CHECK ((kind = ANY (ARRAY['HOLD'::text, 'RELEASE'::text, 'REFUND'::text, 'PROCESSOR_FEE'::text, 'CHARGEBACK'::text, 'REVERSAL'::text])))
escrow_ledger_party_check	check	CHECK ((party = ANY (ARRAY['CLIENT'::text, 'DESIGNER'::text, 'QC'::text, 'PLATFORM'::text, 'PROCESSOR'::text])))

Constraint	Kind	Definition
escrow_ledger_order_id_fkey	foreign key	FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE RESTRICT
escrow_ledger_payee_id_fkey	foreign key	FOREIGN KEY (payee_id) REFERENCES users(id) ON DELETE RESTRICT
escrow_ledger_pkey	primary key	PRIMARY KEY (id)

Policies:

Policy	Command	Roles	USING	WITH CHECK
escrow_ledger_read	SELECT	{authenticated}	((EXISTS (SELECT 1 FROM orders o WHERE (o.id = escrow_ledger.order_id))) AND app.sees_money_of(order_id))	

Triggers:

- escrow_ledger_conservation: BEFORE INSERT ON public.escrow_ledger FOR EACH ROW EXECUTE FUNCTION app.enforce_escrow_conservation()
- escrow_ledger_hold_paid: AFTER INSERT ON public.escrow_ledger FOR EACH ROW WHEN ((new.kind = 'HOLD'::text)) EXECUTE FUNCTION app.on_hold_recorded()
- escrow_ledger_no_delete: BEFORE DELETE ON public.escrow_ledger FOR EACH ROW EXECUTE FUNCTION app.reject_mutation()
- escrow_ledger_no_update: BEFORE UPDATE ON public.escrow_ledger FOR EACH ROW EXECUTE FUNCTION app.reject_mutation()
- escrow_marketing_payment: AFTER INSERT ON public.escrow_ledger FOR EACH ROW EXECUTE FUNCTION app.marketing_payment_event()

public.feature_flags

RLS: **on** (forced). Table grants: authenticated: SELECT. Column-level grants: authenticated SELECT on 5 column(s).

Column	Type	Null	Default
key	text	no	
enabled	boolean	no	
description	text	no	
updated_by	text	yes	
updated_at	timestamp with time zone	no	now()

Constraint	Kind	Definition
feature_flags_key_check	check	CHECK ((key ~ '^[a-z][a-z0-9_]*$'::text))
feature_flags_updated_by_fkey	foreign key	FOREIGN KEY (updated_by) REFERENCES users(id) ON DELETE RESTRICT
feature_flags_pkey	primary key	PRIMARY KEY (key)

Policies:

Policy	Command	Roles	USING	WITH CHECK
feature_flags_read	SELECT	{authenticated}	true	

public.file_versions

RLS: **on** (forced). Table grants: none to anon/authenticated. Column-level grants: authenticated SELECT on 10 column(s).

Column	Type	Null	Default
id	text	no	
order_id	text	no	
version_no	integer	no	
object_key	text	no	
content_type	text	no	
size_bytes	integer	no	
uploaded_by	text	no	
created_at	timestamp with time zone	no	now()
deleted_at	timestamp with time zone	yes	
kind	file_kind	no	'OTHER'::file_kind
uploader_party	text	no	

Constraint	Kind	Definition
file_versions_size_bytes_check	check	CHECK ((size_bytes >= 0))
file_versions_uploader_party_check	check	CHECK ((uploader_party = ANY (ARRAY['CLIENT'::text, 'DESIGNER'::text, 'TEAM'::text])))
file_versions_order_id_fkey	foreign key	FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE RESTRICT
file_versions_uploaded_by_fkey	foreign key	FOREIGN KEY (uploaded_by) REFERENCES users(id) ON DELETE RESTRICT
file_versions_pkey	primary key	PRIMARY KEY (id)
file_versions_object_key_key	unique	UNIQUE (object_key)
file_versions_order_id_version_no_key	unique	UNIQUE (order_id, version_no)

Policies:

Policy	Command	Roles	USING	WITH CHECK
file_versions_designer_read	SELECT	{authenticated}	app.is_assigned_designer(order_id)	
file_versions_read	SELECT	{authenticated}	(EXISTS (SELECT 1 FROM orders o WHERE (o.id = file_versions.order_id)))	

Triggers:

- `file_versions_uploader_party`: `BEFORE INSERT ON public.file_versions FOR EACH ROW EXECUTE FUNCTION app.set_uploader_party()`

public.job_alerts

RLS: **on** (forced). Table grants: none to anon/authenticated.

Column	Type	Null	Default
`order_id`	text	no	
`version`	integer	no	
`kind`	text	no	
`sent_at`	timestamp with time zone	no	now()

Constraint	Kind	Definition
`job_alerts_kind_check`	check	`CHECK ((kind = ANY (ARRAY['DUE_SOON'::text, 'OVERDUE'::text])))`
`job_alerts_order_id_fkey`	foreign key	`FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE RESTRICT`
`job_alerts_pkey`	primary key	`PRIMARY KEY (order_id, version, kind)`

Policies: none (with RLS on, this table is closed to anon and authenticated; only SECURITY DEFINER functions and the service role reach it).

public.job_assignments

RLS: **on** (forced). Table grants: none to anon/authenticated. Column-level grants: authenticated SELECT on 6 column(s).

Column	Type	Null	Default
`id`	text	no	(gen_random_uuid())::text
`order_id`	text	no	
`designer_id`	text	no	
`assigned_by`	text	no	
`assigned_at`	timestamp with time zone	no	now()
`override_reason`	text	yes	
`ended_at`	timestamp with time zone	yes	

Constraint	Kind	Definition
`job_assignments_override_reason_check`	check	`CHECK (((override_reason IS NULL) OR ((char_length(btrim(override_reason)) >= 1) AND (char_length(btrim(override_reason)) <= 500))))`
`job_assignments_assigned_by_fkey`	foreign key	`FOREIGN KEY (assigned_by) REFERENCES users(id) ON DELETE RESTRICT`

Constraint	Kind	Definition
job_assignments_designer_id_fkey	foreign key	FOREIGN KEY (designer_id) REFERENCES users(id) ON DELETE RESTRICT
job_assignments_order_id_fkey	foreign key	FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE RESTRICT
job_assignments_pkey	primary key	PRIMARY KEY (id)

Policies:

Policy	Command	Roles	USING	WITH CHECK
job_assignments_read	SELECT	{authenticated}	(((designer_id = app.current_clerk_id()) AND (app.current_user_role() = 'DESIGNER'::role)) OR app.is_admin() OR app.order_in_my_pods(order_id))	

public.job_categories

RLS: **on** (forced). Table grants: authenticated: SELECT. Column-level grants: authenticated SELECT on 5 column(s).

Column	Type	Null	Default
letter	character	no	
minutes	integer	no	
tier	job_tier	no	
designer_rate_paise	integer	no	
updated_at	timestamp with time zone	no	now()

Constraint	Kind	Definition
job_categories_designer_rate_paise_check	check	CHECK ((designer_rate_paise >= 0))
job_categories_letter_check	check	CHECK (((letter >= 'A'::bpchar) AND (letter <= 'L'::bpchar)))
job_categories_minutes_check	check	CHECK ((minutes > 0))
job_categories_pkey	primary key	PRIMARY KEY (letter)

Policies:

Policy	Command	Roles	USING	WITH CHECK
job_categories_staff_read	SELECT	{authenticated}	app.is_staff()	

public.job_escalations

RLS: **on** (forced). Table grants: none to anon/authenticated.

Column	Type	Null	Default
id	text	no	(gen_random_uuid())::text
order_id	text	no	
raised_by	text	no	
reason	text	no	
created_at	timestamp with time zone	no	now()
resolved_by	text	yes	
resolved_at	timestamp with time zone	yes	
resolution	text	yes	

Constraint	Kind	Definition
job_escalations_check	check	CHECK (((resolved_at IS NULL) = (resolved_by IS NULL)))
job_escalations_reason_check	check	CHECK (((char_length(btrim(reason)) >= 1) AND (char_length(btrim(reason)) <= 1000)))
job_escalations_resolution_check	check	CHECK (((resolution IS NULL) OR (char_length(resolution) <= 1000)))
job_escalations_order_id_fkey	foreign key	FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE RESTRICT
job_escalations_raised_by_fkey	foreign key	FOREIGN KEY (raised_by) REFERENCES users(id) ON DELETE RESTRICT
job_escalations_resolved_by_fkey	foreign key	FOREIGN KEY (resolved_by) REFERENCES users(id) ON DELETE RESTRICT
job_escalations_pkey	primary key	PRIMARY KEY (id)

Policies: none (with RLS on, this table is closed to anon and authenticated; only SECURITY DEFINER functions and the service role reach it).

public.job_pauses

RLS: **on** (forced). Table grants: none to anon/authenticated.

Column	Type	Null	Default
id	text	no	(gen_random_uuid())::text
order_id	text	no	
version	integer	no	
designer_id	text	no	
paused_at	timestamp with time zone	no	now()
resumed_at	timestamp with time zone	yes	

Constraint	Kind	Definition
job_pauses_check	check	CHECK (((resumed_at IS NULL) OR (resumed_at >= paused_at)))
job_pauses_version_check	check	CHECK ((version >= 1))

Constraint	Kind	Definition
job_pauses_designer_id_fkey	foreign key	FOREIGN KEY (designer_id) REFERENCES users(id) ON DELETE RESTRICT
job_pauses_order_id_fkey	foreign key	FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE RESTRICT
job_pauses_pkey	primary key	PRIMARY KEY (id)

Policies: none (with RLS on, this table is closed to anon and authenticated; only SECURITY DEFINER functions and the service role reach it).

public.job_volumes

RLS: **on** (forced). Table grants: none to anon/authenticated.

Column	Type	Null	Default
order_id	text	no	
version	integer	no	
volume_mm3	numeric	no	
entered_by	text	no	
entered_at	timestamp with time zone	no	now()

Constraint	Kind	Definition
job_volumes_version_check	check	CHECK ((version >= 1))
job_volumes_volume_mm3_check	check	CHECK (((volume_mm3 > (0)::numeric) AND (volume_mm3 < (10000000)::numeric)))
job_volumes_entered_by_fkey	foreign key	FOREIGN KEY (entered_by) REFERENCES users(id) ON DELETE RESTRICT
job_volumes_order_id_fkey	foreign key	FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE RESTRICT
job_volumes_pkey	primary key	PRIMARY KEY (order_id, version)

Policies: none (with RLS on, this table is closed to anon and authenticated; only SECURITY DEFINER functions and the service role reach it).

public.job_weights

RLS: **on** (forced). Table grants: none to anon/authenticated.

Column	Type	Null	Default
order_id	text	no	
version	integer	no	
gold_18k_g	numeric	yes	
casting_allowance_g	numeric	no	0.1
post_allowance_g	numeric	no	1
stones	jsonb	no	'[]'::jsonb

Column	Type	Null	Default
tech_source_id	text	yes	
entered_by	text	yes	
updated_at	timestamp with time zone	no	now()

Constraint	Kind	Definition
job_weights_casting_allowance_g_check	check	CHECK (((casting_allowance_g >= (0)::numeric) AND (casting_allowance_g <= (50)::numeric)))
job_weights_gold_18k_g_check	check	CHECK (((gold_18k_g > (0)::numeric) AND (gold_18k_g <= (5000)::numeric)))
job_weights_post_allowance_g_check	check	CHECK (((post_allowance_g >= (0)::numeric) AND (post_allowance_g <= (50)::numeric)))
job_weights_stones_check	check	CHECK (((jsonb_typeof(stones) = 'array'::text) AND (jsonb_array_length(stones) <= 60)))
job_weights_version_check	check	CHECK ((version >= 1))
job_weights_entered_by_fkey	foreign key	FOREIGN KEY (entered_by) REFERENCES users(id) ON DELETE RESTRICT
job_weights_order_id_fkey	foreign key	FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE RESTRICT
job_weights_tech_source_id_fkey	foreign key	FOREIGN KEY (tech_source_id) REFERENCES file_versions(id) ON DELETE SET NULL
job_weights_pkey	primary key	PRIMARY KEY (order_id, version)

Policies: none (with RLS on, this table is closed to anon and authenticated; only SECURITY DEFINER functions and the service role reach it).

public.marketing_attribution

RLS: **on** (forced). Table grants: none to anon/authenticated.

Column	Type	Null	Default
user_id	text	no	
heard_about	text	no	
first_touch	jsonb	no	'{}'::jsonb
created_at	timestamp with time zone	no	now()

Constraint	Kind	Definition
marketing_attribution_heard_about_check	check	CHECK ((heard_about = ANY (ARRAY['google'::text, 'instagram'::text, 'facebook'::text, 'linkedin'::text, 'referral'::text, 'trade_show'::text, 'email'::text, 'other'::text])))

Constraint	Kind	Definition
marketing_attribution_user_id_fkey	foreign key	FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE RESTRICT
marketing_attribution_pkey	primary key	PRIMARY KEY (user_id)

Policies: none (with RLS on, this table is closed to anon and authenticated; only SECURITY DEFINER functions and the service role reach it).

public.marketing_events

RLS: **on** (forced). Table grants: none to anon/authenticated.

Column	Type	Null	Default
id	text	no	(gen_random_uuid())::text
event	text	no	
user_id	text	yes	
order_id	text	yes	
visitor_id	text	yes	
props	jsonb	no	'{}'::jsonb
created_at	timestamp with time zone	no	now()

Constraint	Kind	Definition
marketing_events_event_check	check	CHECK ((event = ANY (ARRAY['brief_started'::text, 'brief_submitted'::text, 'payment_completed'::text, 'first_look_viewed'::text, 'approved'::text, 'preview_link_created'::text, 'spec_claim_opened'::text])))
marketing_events_props_check	check	CHECK ((pg_column_size(props) <= 2048))
marketing_events_visitor_id_check	check	CHECK (((visitor_id IS NULL) OR (char_length(visitor_id) <= 64)))
marketing_events_order_id_fkey	foreign key	FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE RESTRICT
marketing_events_user_id_fkey	foreign key	FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE RESTRICT
marketing_events_pkey	primary key	PRIMARY KEY (id)

Policies: none (with RLS on, this table is closed to anon and authenticated; only SECURITY DEFINER functions and the service role reach it).

public.marketing_leads

RLS: **on** (forced). Table grants: none to anon/authenticated.

Column	Type	Null	Default
id	text	no	(gen_random_uuid())::text
name	text	no	
company	text	yes	
email	text	no	
role	text	no	'BUSINESS'::text
message	text	no	
created_at	timestamp with time zone	no	now()
status	text	no	'NEW'::text
handled_at	timestamp with time zone	yes	
handled_by	text	yes	

Constraint	Kind	Definition
marketing_leads_company_check	check	CHECK (((company IS NULL) OR (char_length(company) <= 200)))
marketing_leads_email_check	check	CHECK ((email ~ '^[^@\s]+@[^@\s]+\.[^@\s]+$'::text))
marketing_leads_message_check	check	CHECK (((char_length(message) >= 1) AND (char_length(message) <= 5000)))
marketing_leads_name_check	check	CHECK (((char_length(name) >= 1) AND (char_length(name) <= 200)))
marketing_leads_role_check	check	CHECK ((role = ANY (ARRAY['BUSINESS'::text, 'DESIGNER'::text, 'OTHER'::text])))
marketing_leads_status_check	check	CHECK ((status = ANY (ARRAY['NEW'::text, 'HANDLED'::text])))
marketing_leads_handled_by_fkey	foreign key	FOREIGN KEY (handled_by) REFERENCES users(id) ON DELETE RESTRICT
marketing_leads_pkey	primary key	PRIMARY KEY (id)

Policies: none (with RLS on, this table is closed to anon and authenticated; only SECURITY DEFINER functions and the service role reach it).

public.message_threads

RLS: **on** (forced). Table grants: authenticated: SELECT. Column-level grants: authenticated SELECT on 4 column(s).

Column	Type	Null	Default
id	text	no	(gen_random_uuid())::text
order_id	text	no	
kind	text	no	
created_at	timestamp with time zone	no	now()

Constraint	Kind	Definition
`message_threads_kind_check`	check	`CHECK ((kind = ANY (ARRAY['client_team'::text, 'internal'::text, 'legacy'::text])))`
`message_threads_order_id_fkey`	foreign key	`FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE RESTRICT`
`message_threads_pkey`	primary key	`PRIMARY KEY (id)`
`message_threads_order_id_kind_key`	unique	`UNIQUE (order_id, kind)`

Policies:

Policy	Command	Roles	USING	WITH CHECK
`message_threads_read`	SELECT	{authenticated}	`app.can_read_thread(id)`	

public.messages

RLS: **on** (forced). Table grants: none to anon/authenticated. Column-level grants: authenticated SELECT on 7 column(s).

Column	Type	Null	Default
`id`	text	no	(gen_random_uuid())::text
`order_id`	text	no	
`sender_id`	text	no	
`sender_party`	text	no	
`body`	text	no	
`created_at`	timestamp with time zone	no	now()
`thread_id`	text	no	
`sender_alias`	text	no	
`client_ref`	text	yes	

Constraint	Kind	Definition
`messages_body_check`	check	`CHECK (((char_length(body) >= 1) AND (char_length(body) <= 5000)))`
`messages_sender_alias_check`	check	`CHECK (((sender_alias IS NULL) OR ((char_length(sender_alias) >= 1) AND (char_length(sender_alias) <= 60))))`
`messages_sender_party_check`	check	`CHECK ((sender_party = ANY (ARRAY['CLIENT'::text, 'DESIGNER'::text, 'TEAM'::text])))`
`messages_order_id_fkey`	foreign key	`FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE RESTRICT`
`messages_sender_id_fkey`	foreign key	`FOREIGN KEY (sender_id) REFERENCES users(id) ON DELETE RESTRICT`
`messages_thread_id_fkey`	foreign key	`FOREIGN KEY (thread_id) REFERENCES message_threads(id) ON DELETE RESTRICT`

Constraint	Kind	Definition
`messages_pkey`	primary key	`PRIMARY KEY (id)`

Policies:

Policy	Command	Roles	USING	WITH CHECK
`messages_read`	SELECT	{authenticated}	`app.can_read_thread(thread_id)`	

Triggers:

- `messages_client_emails`: `AFTER INSERT ON public.messages FOR EACH ROW EXECUTE FUNCTION app.message_emails()`
- `messages_no_delete`: `BEFORE DELETE ON public.messages FOR EACH ROW EXECUTE FUNCTION app.reject_mutation()`
- `messages_no_update`: `BEFORE UPDATE ON public.messages FOR EACH ROW EXECUTE FUNCTION app.reject_mutation()`
- `messages_signal_thread`: `AFTER INSERT ON public.messages FOR EACH ROW EXECUTE FUNCTION app.signal_thread_change()`
- `messages_take_client_ref`: `BEFORE INSERT ON public.messages FOR EACH ROW EXECUTE FUNCTION app.messages_take_client_ref()`

public.notifications

RLS: **on** (forced). Table grants: authenticated: SELECT. Column-level grants: authenticated SELECT on 9 column(s).

Column	Type	Null	Default
`id`	text	no	(gen_random_uuid())::text
`user_id`	text	no	
`kind`	text	no	
`order_id`	text	yes	
`summary`	text	no	
`read_at`	timestamp with time zone	yes	
`created_at`	timestamp with time zone	no	now()
`pushed_at`	timestamp with time zone	yes	
`push_attempts`	integer	no	0

Constraint	Kind	Definition
`notifications_push_attempts_check`	check	`CHECK ((push_attempts >= 0))`
`notifications_order_id_fkey`	foreign key	`FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE RESTRICT`
`notifications_user_id_fkey`	foreign key	`FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE RESTRICT`
`notifications_pkey`	primary key	`PRIMARY KEY (id)`

Policies:

Policy	Command	Roles	USING	WITH CHECK
`notifications_read_own`	SELECT	{authenticated}	`(user_id = app.current_clerk_id())`	

public.order_checks

RLS: **on** (forced). Table grants: none to anon/authenticated. Column-level grants: authenticated SELECT on 5 column(s).

Column	Type	Null	Default
`order_id`	text	no	
`version`	integer	no	
`item_id`	smallint	no	
`designer_by`	text	yes	
`designer_at`	timestamp with time zone	yes	
`qc_by`	text	yes	
`qc_at`	timestamp with time zone	yes	
`outcome`	text	yes	
`finding`	text	yes	
`remark`	text	yes	

Constraint	Kind	Definition
`order_checks_check`	check	`CHECK (((designer_by IS NULL) = (designer_at IS NULL)))`
`order_checks_check1`	check	`CHECK (((qc_by IS NULL) = (qc_at IS NULL)))`
`order_checks_finding_check`	check	`CHECK ((char_length(finding) <= 300))`
`order_checks_outcome_check`	check	`CHECK ((outcome = ANY (ARRAY['PASS'::text, 'FAIL'::text])))`
`order_checks_remark_check`	check	`CHECK ((char_length(remark) <= 500))`
`order_checks_version_check`	check	`CHECK ((version >= 1))`
`order_checks_designer_by_fkey`	foreign key	`FOREIGN KEY (designer_by) REFERENCES users(id) ON DELETE RESTRICT`
`order_checks_item_id_fkey`	foreign key	`FOREIGN KEY (item_id) REFERENCES checklist_items(id)`
`order_checks_order_id_fkey`	foreign key	`FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE RESTRICT`
`order_checks_qc_by_fkey`	foreign key	`FOREIGN KEY (qc_by) REFERENCES users(id) ON DELETE RESTRICT`
`order_checks_pkey`	primary key	`PRIMARY KEY (order_id, version, item_id)`

Policies:

Policy	Command	Roles	USING	WITH CHECK
order_checks_read	SELECT	{authenticated}	(app.is_assigned_designer(order_id) OR app.is_admin() OR app.order_in_my_pods(order_id))	

public.order_reference_images

RLS: **on** (forced). Table grants: authenticated: SELECT. Column-level grants: authenticated SELECT on 8 column(s).

Column	Type	Null	Default
id	text	no	(gen_random_uuid())::text
order_id	text	no	
object_key	text	no	
content_type	text	no	
size_bytes	integer	no	
position	integer	no	
is_primary	boolean	no	false
created_at	timestamp with time zone	no	now()

Constraint	Kind	Definition
order_reference_images_content_type_check	check	CHECK ((content_type = ANY (ARRAY['image/jpeg'::text, 'image/png'::text, 'image/webp'::text])))
order_reference_images_position_check	check	CHECK ((("position" >= 1) AND ("position" <= 20)))
order_reference_images_size_bytes_check	check	CHECK ((size_bytes > 0))
order_reference_images_order_id_fkey	foreign key	FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE RESTRICT
order_reference_images_pkey	primary key	PRIMARY KEY (id)
order_reference_images_object_key_key	unique	UNIQUE (object_key)
order_reference_images_order_id_position_key	unique	UNIQUE (order_id, "position")

Policies:

Policy	Command	Roles	USING	WITH CHECK
order_reference_images_designer_read	SELECT	{authenticated}	app.is_assigned_designer(order_id)	
order_reference_images_visible_with_order	SELECT	{authenticated}	(EXISTS (SELECT 1 FROM orders o WHERE (o.id = order_reference_images.order_id)))	

public.order_reference_pins

RLS: **on** (forced). Table grants: authenticated: SELECT. Column-level grants: authenticated SELECT on 6 column(s).

Column	Type	Null	Default
id	text	no	(gen_random_uuid())::text
image_id	text	no	
x_bp	integer	no	
y_bp	integer	no	
label	text	no	
position	integer	no	

Constraint	Kind	Definition
order_reference_pins_label_check	check	CHECK (((char_length(label) >= 1) AND (char_length(label) <= 120)))
order_reference_pins_position_check	check	CHECK ((("position" >= 1) AND ("position" <= 30)))
order_reference_pins_x_bp_check	check	CHECK (((x_bp >= 0) AND (x_bp <= 10000)))
order_reference_pins_y_bp_check	check	CHECK (((y_bp >= 0) AND (y_bp <= 10000)))
order_reference_pins_image_id_fkey	foreign key	FOREIGN KEY (image_id) REFERENCES order_reference_images(id) ON DELETE CASCADE
order_reference_pins_pkey	primary key	PRIMARY KEY (id)
order_reference_pins_image_id_position_key	unique	UNIQUE (image_id, "position")

Policies:

Policy	Command	Roles	USING	WITH CHECK
order_reference_pins_visible_with_image	SELECT	{authenticated}	(EXISTS (SELECT 1 FROM order_reference_images i WHERE (i.id = order_reference_pins.image_id)))	

public.order_screenshots

RLS: **on** (forced). Table grants: none to anon/authenticated. Column-level grants: authenticated SELECT on 7 column(s).

Column	Type	Null	Default
id	text	no	(gen_random_uuid())::text
order_id	text	no	
object_key	text	no	
content_type	text	no	

Column	Type	Null	Default
size_bytes	integer	no	
position	smallint	no	
added_by	text	no	
created_at	timestamp with time zone	no	now()

Constraint	Kind	Definition
order_screenshots_content_type_check	check	CHECK ((content_type = ANY (ARRAY['image/jpeg'::text, 'image/png'::text, 'image/webp'::text])))
order_screenshots_position_check	check	CHECK ((("position" >= 1) AND ("position" <= 20)))
order_screenshots_size_bytes_check	check	CHECK ((size_bytes > 0))
order_screenshots_added_by_fkey	foreign key	FOREIGN KEY (added_by) REFERENCES users(id) ON DELETE RESTRICT
order_screenshots_order_id_fkey	foreign key	FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE RESTRICT
order_screenshots_pkey	primary key	PRIMARY KEY (id)
order_screenshots_object_key_key	unique	UNIQUE (object_key)
order_screenshots_order_id_position_key	unique	UNIQUE (order_id, "position")

Policies:

Policy	Command	Roles	USING	WITH CHECK
order_screenshots_read	SELECT	{authenticated}	app.can_see_screenshots(order_id)	

public.order_spec_accents

RLS: **on** (forced). Table grants: authenticated: SELECT. Column-level grants: authenticated SELECT on 7 column(s).

Column	Type	Null	Default
id	text	no	(gen_random_uuid())::text
order_id	text	no	
position	integer	no	
shape	stone_shape	no	
width_um	integer	yes	
quantity	integer	yes	
setting	setting_type	no	

Constraint	Kind	Definition
order_spec_accents_position_check	check	CHECK ((("position" >= 1) AND ("position" <= 40)))

Constraint	Kind	Definition
`order_spec_accents_quantity_check`	check	`CHECK (((quantity >= 1) AND (quantity <= 2000)))`
`order_spec_accents_width_um_check`	check	`CHECK (((width_um >= 200) AND (width_um <= 20000)))`
`order_spec_accents_order_id_fkey`	foreign key	`FOREIGN KEY (order_id) REFERENCES order_specs(order_id) ON DELETE CASCADE`
`order_spec_accents_pkey`	primary key	`PRIMARY KEY (id)`
`order_spec_accents_order_id_position_key`	unique	`UNIQUE (order_id, "position")`

Policies:

Policy	Command	Roles	USING	WITH CHECK
`order_spec_accents_designer_read`	SELECT	{authenticated}	`app.is_assigned_designer(order_id)`	
`order_spec_accents_visible_with_order`	SELECT	{authenticated}	`(EXISTS ( SELECT 1 FROM orders o WHERE (o.id = order_spec_accents.order_id)))`	

public.order_specs

RLS: **on** (forced). Table grants: authenticated: SELECT. Column-level grants: authenticated SELECT on 30 column(s).

Column	Type	Null	Default
`order_id`	text	no	
`reference_name`	text	no	
`based_on_order_id`	text	yes	
`change_summary`	text	yes	
`product`	product_kind	no	
`has_centre_stone`	boolean	no	false
`centre_shape`	stone_shape	yes	
`centre_length_um`	integer	yes	
`centre_width_um`	integer	yes	
`centre_depth_um`	integer	yes	
`centre_carat_mct`	integer	yes	
`centre_certified`	boolean	no	false
`centre_quantity`	integer	no	0
`centre_setting`	setting_type	yes	
`stones_supplied_by`	stone_supply	no	‘NONE’::stone_supply
`metal`	metal_colour	no	
`karatage`	text	yes	

Column	Type	Null	Default
purpose	cad_purpose	yes	
component_count	integer	no	1
format	output_format	yes	
finish	finish_type	yes	
render_views	integer	no	0
priority	priority_tier	no	'STANDARD'::priority_tier
notes	text	yes	
created_at	timestamp with time zone	no	now()
updated_at	timestamp with time zone	no	now()
ring_size	text	yes	
inner_circumference_um	integer	yes	
inner_diameter_um	integer	yes	
length_um	integer	yes	

Constraint	Kind	Definition
order_specs_centre_carat_mct_check	check	CHECK (((centre_carat_mct IS NULL) OR ((centre_carat_mct >= 1) AND (centre_carat_mct <= 100000))))
order_specs_centre_coherent	check	CHECK (((has_centre_stone AND (centre_shape IS NOT NULL) AND (centre_setting IS NOT NULL) AND (centre_quantity >= 1) AND ((centre_carat_mct IS NOT NULL) OR ((centre_length_um IS NOT NULL) AND (centre_width_um IS NOT NULL)))) OR ((NOT has_centre_stone) AND (centre_shape IS NULL) AND (centre_setting IS NULL) AND (centre_length_um IS NULL) AND (centre_width_um IS NULL) AND (centre_depth_um IS NULL) AND (centre_carat_mct IS NULL) AND (centre_quantity = 0))))
order_specs_centre_depth_um_check	check	CHECK (((centre_depth_um IS NULL) OR ((centre_depth_um >= 200) AND (centre_depth_um <= 40000))))
order_specs_centre_length_um_check	check	CHECK (((centre_length_um IS NULL) OR ((centre_length_um >= 500) AND (centre_length_um <= 60000))))
order_specs_centre_quantity_check	check	CHECK (((centre_quantity >= 0) AND (centre_quantity <= 50)))
order_specs_centre_width_um_check	check	CHECK (((centre_width_um IS NULL) OR ((centre_width_um >= 500) AND (centre_width_um <= 60000))))
order_specs_change_summary_check	check	CHECK (((change_summary IS NULL) OR (char_length(change_summary) <= 4000)))

Constraint	Kind	Definition
order_specs_component_count_check	check	CHECK (((component_count >= 1) AND (component_count <= 20)))
order_specs_inner_circumference_um_check	check	CHECK (((inner_circumference_um IS NULL) OR ((inner_circumference_um >= 20000) AND (inner_circumference_um <= 300000))))
order_specs_inner_diameter_um_check	check	CHECK (((inner_diameter_um IS NULL) OR ((inner_diameter_um >= 5000) AND (inner_diameter_um <= 100000))))
order_specs_karatage_check	check	CHECK (((char_length(karatage) >= 2) AND (char_length(karatage) <= 24)))
order_specs_length_um_check	check	CHECK (((length_um IS NULL) OR ((length_um >= 1000) AND (length_um <= 1000000))))
order_specs_lineage_coherent	check	CHECK ((((based_on_order_id IS NULL) AND (change_summary IS NULL)) OR ((based_on_order_id IS NOT NULL) AND (change_summary IS NOT NULL))))
order_specs_no_self_lineage	check	CHECK ((based_on_order_id <> order_id))
order_specs_notes_check	check	CHECK (((notes IS NULL) OR (char_length(notes) <= 4000)))
order_specs_reference_name_check	check	CHECK (((char_length(reference_name) >= 1) AND (char_length(reference_name) <= 120)))
order_specs_render_views_check	check	CHECK (((render_views >= 0) AND (render_views <= 12)))
order_specs_ring_size_check	check	CHECK (((ring_size IS NULL) OR ((char_length(btrim(ring_size)) >= 1) AND (char_length(btrim(ring_size)) <= 12))))
order_specs_based_on_order_id_fkey	foreign key	FOREIGN KEY (based_on_order_id) REFERENCES orders(id) ON DELETE RESTRICT
order_specs_order_id_fkey	foreign key	FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE RESTRICT
order_specs_pkey	primary key	PRIMARY KEY (order_id)

Policies:

Policy	Command	Roles	USING	WITH CHECK
order_specs_visible_with_order	SELECT	{authenticated}	(EXISTS (SELECT 1 FROM orders o WHERE (o.id = order_specs.order_id)))	

public.order_stone_certificates

RLS: **on** (forced). Table grants: none to anon/authenticated.

Column	Type	Null	Default
id	text	no	(gen_random_uuid())::text
order_id	text	no	
object_key	text	no	
content_type	text	no	
size_bytes	integer	no	
created_at	timestamp with time zone	no	now()

Constraint	Kind	Definition
order_stone_certificates_content_type_check	check	CHECK ((content_type = ANY (ARRAY['image/jpeg'::text, 'image/png'::text, 'application/pdf'::text])))
order_stone_certificates_size_bytes_check	check	CHECK ((size_bytes > 0))
order_stone_certificates_order_id_fkey	foreign key	FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE RESTRICT
order_stone_certificates_pkey	primary key	PRIMARY KEY (id)
order_stone_certificates_object_key_key	unique	UNIQUE (object_key)

Policies: none (with RLS on, this table is closed to anon and authenticated; only SECURITY DEFINER functions and the service role reach it).

public.order_transitions

RLS: **on** (forced). Table grants: authenticated: SELECT. Column-level grants: authenticated SELECT on 4 column(s).

Column	Type	Null	Default
from_status	order_status	no	
to_status	order_status	no	
actor_role	role	no	
actor_scope	text	no	

Constraint	Kind	Definition
order_transitions_actor_scope_check	check	CHECK ((actor_scope = ANY (ARRAY['STAFF'::text, 'CLIENT_PARTY'::text, 'DESIGNER_PARTY'::text])))
order_transitions_pkey	primary key	PRIMARY KEY (from_status, to_status, actor_role)

Policies:

Policy	Command	Roles	USING	WITH CHECK
order_transitions_read	SELECT	{authenticated}	true	

public.orders

RLS: **on** (forced). Table grants: anon: SELECT. Column-level grants: anon SELECT on 42 column(s); authenticated SELECT on 27 column(s).

Column	Type	Null	Default
id	text	no	
client_id	text	no	
designer_id	text	yes	
product_type	text	no	
status	order_status	no	'DRAFT'::order_status
currency	text	no	
price_total	integer	no	
designer_payout	integer	no	
qc_payout	integer	no	
platform_commission	integer	no	
deadline_at	timestamp with time zone	yes	
current_version_id	text	yes	
org_id	text	yes	
created_at	timestamp with time zone	no	now()
deleted_at	timestamp with time zone	yes	
qc_reviewer_id	text	yes	
pod_id	text	yes	
pod_lead_id	text	yes	
service	service_kind	no	'cad'::service_kind
order_no	bigint	yes	
order_ref	text	yes	
job_code	text	yes	
category	character	yes	
tier	job_tier	yes	
complex_deadline_hours	integer	yes	
paid_at	timestamp with time zone	yes	
payment_method	payment_method	yes	
sla_started_at	timestamp with time zone	yes	
sla_due_at	timestamp with time zone	yes	
first_look_at	timestamp with time zone	yes	
sla_status	sla_status	no	'pending'::sla_status
source	order_source	no	'portal'::order_source
created_on_behalf_by	text	yes	
intake_mode	intake_mode	no	'full'::intake_mode

Column	Type	Null	Default
client_po	text	yes	
designer_payout_paise	integer	yes	
qc_payout_paise	integer	yes	
confirmed_at	timestamp with time zone	yes	
confirmed_by	text	yes	
payment_reference	text	yes	
work_version	integer	no	1
received_at	timestamp with time zone	yes	

Constraint	Kind	Definition
orders_client_po_check	check	CHECK (((client_po IS NULL) OR (char_length(client_po) <= 60)))
orders_complex_deadline_hours_check	check	CHECK (((complex_deadline_hours IS NULL) OR (complex_deadline_hours > 0)))
orders_complex_has_deadline	check	CHECK (((tier IS DISTINCT FROM 'complex'::job_tier) OR (complex_deadline_hours IS NOT NULL) OR (status = ANY (ARRAY['DRAFT'::order_status, 'SUBMITTED'::order_status]))))
orders_currency_iso	check	CHECK ((char_length(currency) = 3))
orders_designer_payout_check	check	CHECK ((designer_payout >= 0))
orders_designer_payout_paise_check	check	CHECK (((designer_payout_paise IS NULL) OR (designer_payout_paise >= 0)))
orders_payment_reference_check	check	CHECK (((payment_reference IS NULL) OR (char_length(payment_reference) <= 120)))
orders_platform_commission_check	check	CHECK ((platform_commission >= 0))
orders_price_total_check	check	CHECK ((price_total >= 0))
orders_qc_payout_check	check	CHECK ((qc_payout >= 0))
orders_qc_payout_paise_check	check	CHECK (((qc_payout_paise IS NULL) OR (qc_payout_paise >= 0)))
orders_work_version_check	check	CHECK ((work_version >= 1))
orders_category_fkey	foreign key	FOREIGN KEY (category) REFERENCES job_categories(letter)
orders_client_id_fkey	foreign key	FOREIGN KEY (client_id) REFERENCES users(id) ON DELETE RESTRICT
orders_confirmed_by_fkey	foreign key	FOREIGN KEY (confirmed_by) REFERENCES users(id) ON DELETE RESTRICT
orders_created_on_behalf_by_fkey	foreign key	FOREIGN KEY (created_on_behalf_by) REFERENCES users(id) ON DELETE RESTRICT

Constraint	Kind	Definition
`orders_current_version_fk`	foreign key	`FOREIGN KEY (current_version_id) REFERENCES file_versions(id) ON DELETE RESTRICT`
`orders_designer_id_fkey`	foreign key	`FOREIGN KEY (designer_id) REFERENCES users(id) ON DELETE RESTRICT`
`orders_pod_id_fkey`	foreign key	`FOREIGN KEY (pod_id) REFERENCES pods(id) ON DELETE RESTRICT`
`orders_pod_lead_id_fkey`	foreign key	`FOREIGN KEY (pod_lead_id) REFERENCES users(id) ON DELETE RESTRICT`
`orders_qc_reviewer_id_fkey`	foreign key	`FOREIGN KEY (qc_reviewer_id) REFERENCES users(id) ON DELETE RESTRICT`
`orders_pkey`	primary key	`PRIMARY KEY (id)`
`orders_job_code_key`	unique	`UNIQUE (job_code)`
`orders_order_no_key`	unique	`UNIQUE (order_no)`
`orders_order_ref_key`	unique	`UNIQUE (order_ref)`

Policies:

Policy	Command	Roles	USING	WITH CHECK
`orders_admin_select`	SELECT	{authenticated}	`app.is_admin()`	
`orders_client_select`	SELECT	{authenticated}	`(client_id = app.current_clerk_id())`	
`orders_pod_lead_select`	SELECT	{authenticated}	`((pod_id IS NOT NULL) AND (pod_id = ANY (app.current_pod_ids())))`	
`orders_qc_reviewer_select`	SELECT	{authenticated}	`(qc_reviewer_id = app.current_clerk_id())`	
`orders_staff_select`	SELECT	{authenticated}	`(EXISTS ( SELECT 1 FROM order_transitions t WHERE ((t.from_status = orders.status) AND (t.actor_role = ANY (app.current_roles())) AND (t.actor_role <> 'QC'::role) AND (t.actor_scope = 'STAFF'::text))))`	

Triggers:

- `orders_archive_on_delivery`: `AFTER UPDATE OF status ON public.orders FOR EACH ROW EXECUTE FUNCTION app.archive_on_delivery()`
- `orders_assign_ref`: `BEFORE INSERT OR UPDATE OF status ON public.orders FOR EACH ROW EXECUTE FUNCTION app.assign_order_ref()`

- `orders_auto_pod`: `BEFORE UPDATE OF status, confirmed_at ON public.orders FOR EACH ROW WHEN ((new.pod_id IS NULL)) EXECUTE FUNCTION app.orders_auto_pod()`
- `orders_client_emails`: `AFTER UPDATE OF status, sla_started_at, sla_status ON public.orders FOR EACH ROW EXECUTE FUNCTION app.order_status_emails()`
- `orders_close_pause`: `AFTER UPDATE OF status, designer_id ON public.orders FOR EACH ROW EXECUTE FUNCTION app.close_pause_on_move()`
- `orders_create_threads`: `AFTER INSERT ON public.orders FOR EACH ROW EXECUTE FUNCTION app.create_order_threads()`
- `orders_first_look`: `BEFORE UPDATE OF status ON public.orders FOR EACH ROW EXECUTE FUNCTION app.on_first_look()`
- `orders_marketing_events`: `AFTER UPDATE OF status ON public.orders FOR EACH ROW EXECUTE FUNCTION app.marketing_order_events()`
- `orders_payout_on_close`: `AFTER UPDATE OF status ON public.orders FOR EACH ROW WHEN (((new.status = 'CLOSED'::order_status) AND (old.status IS DISTINCT FROM 'CLOSED'::order_status))) EXECUTE FUNCTION app.create_payout_obligations()`
- `orders_quote_in_usd`: `BEFORE UPDATE OF status ON public.orders FOR EACH ROW WHEN (((new.status = 'QUOTED'::order_status) AND (old.status IS DISTINCT FROM 'QUOTED'::order_status))) EXECUTE FUNCTION app.quote_in_usd()`
- `orders_reprice_on_assign`: `BEFORE UPDATE OF designer_id ON public.orders FOR EACH ROW EXECUTE FUNCTION app.reprice_on_assign()`
- `orders_spec_gates`: `BEFORE UPDATE OF status ON public.orders FOR EACH ROW EXECUTE FUNCTION app.spec_gates()`
- `orders_weights_gate`: `BEFORE UPDATE OF status ON public.orders FOR EACH ROW EXECUTE FUNCTION app.weights_gate()`

public.payment_intents

RLS: **on** (forced). Table grants: none to anon/authenticated.

Column	Type	Null	Default
`id`	text	no	(gen_random_uuid())::text
`order_id`	text	no	
`external_ref`	text	no	
`amount`	integer	no	
`currency`	text	no	
`status`	text	no	'PENDING'::text
`created_at`	timestamp with time zone	no	now()
`confirmed_at`	timestamp with time zone	yes	

Constraint	Kind	Definition
`payment_intents_amount_check`	check	`CHECK ((amount > 0))`
`payment_intents_status_check`	check	`CHECK ((status = ANY (ARRAY['PENDING'::text, 'CONFIRMED'::text, 'FAILED'::text])))`
`payment_intents_order_id_fkey`	foreign key	`FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE RESTRICT`

Constraint	Kind	Definition
payment_intents_pkey	primary key	PRIMARY KEY (id)
payment_intents_external_ref_key	unique	UNIQUE (external_ref)

Policies: none (with RLS on, this table is closed to anon and authenticated; only SECURITY DEFINER functions and the service role reach it).

public.payout_accounts

RLS: **on** (forced). Table grants: anon: SELECT; authenticated: SELECT. Column-level grants: anon SELECT on 16 column(s); authenticated SELECT on 16 column(s).

Column	Type	Null	Default
id	text	no	(gen_random_uuid())::text
user_id	text	no	
country	text	no	'IN'::text
beneficiary_name	text	no	
pan	text	no	
account_number	text	no	
ifsc	text	no	
account_type	text	no	
account_last4	text (generated)	yes	
pan_last4	text (generated)	yes	
status	text	no	'PENDING_VERIFICATION'::text
rejection_reason	text	yes	
processor_account_ref	text	yes	
processor_fund_account_ref	text	yes	
created_at	timestamp with time zone	no	now()
updated_at	timestamp with time zone	no	now()

Constraint	Kind	Definition
payout_accounts_account_number_check	check	CHECK ((account_number ~ '^[0-9]{9,18}$'::text))
payout_accounts_account_type_check	check	CHECK ((account_type = ANY (ARRAY['SAVINGS'::text, 'CURRENT'::text])))
payout_accounts_beneficiary_name_check	check	CHECK (((char_length(beneficiary_name) >= 2) AND (char_length(beneficiary_name) <= 120)))
payout_accounts_country_check	check	CHECK ((country = 'IN'::text))
payout_accounts_ifsc_check	check	CHECK ((ifsc ~ '^[A-Z]{4}0[A-Z0-9]{6}$'::text))

Constraint	Kind	Definition
payout_accounts_pan_check	check	CHECK ((pan ~ '^[A-Z]{5}[0-9]{4}[A-Z]$'::text))
payout_accounts_rejection_has_reason	check	CHECK (((status <> 'REJECTED'::text) OR (rejection_reason IS NOT NULL)))
payout_accounts_rejection_reason_check	check	CHECK (((rejection_reason IS NULL) OR (char_length(rejection_reason) <= 500)))
payout_accounts_status_check	check	CHECK ((status = ANY (ARRAY['PENDING_VERIFICATION'::text, 'VERIFIED'::text, 'REJECTED'::text])))
payout_accounts_user_id_fkey	foreign key	FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE RESTRICT
payout_accounts_pkey	primary key	PRIMARY KEY (id)
payout_accounts_user_id_key	unique	UNIQUE (user_id)

Policies: none (with RLS on, this table is closed to anon and authenticated; only SECURITY DEFINER functions and the service role reach it).

public.payout_obligations

RLS: **on** (forced). Table grants: none to anon/authenticated. Column-level grants: authenticated SELECT on 8 column(s).

Column	Type	Null	Default
id	text	no	(gen_random_uuid())::text
order_id	text	no	
payee_id	text	no	
party	text	no	
amount_paise	integer	no	
status	text	no	'owed'::text
export_batch	text	yes	
created_at	timestamp with time zone	no	now()
exported_at	timestamp with time zone	yes	
paid_at	timestamp with time zone	yes	

Constraint	Kind	Definition
payout_obligations_amount_paise_check	check	CHECK ((amount_paise > 0))
payout_obligations_party_check	check	CHECK ((party = ANY (ARRAY['DESIGNER'::text, 'QC'::text])))
payout_obligations_status_check	check	CHECK ((status = ANY (ARRAY['owed'::text, 'exported'::text, 'paid'::text, 'void'::text])))
payout_obligations_order_id_fkey	foreign key	FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE RESTRICT

Constraint	Kind	Definition
payout_obligations_payee_id_fkey	foreign key	FOREIGN KEY (payee_id) REFERENCES users(id) ON DELETE RESTRICT
payout_obligations_pkey	primary key	PRIMARY KEY (id)
payout_obligations_order_id_party_key	unique	UNIQUE (order_id, party)

Policies:

Policy	Command	Roles	USING	WITH CHECK
payout_obligations_read	SELECT	{authenticated}	((payee_id = app.current_clerk_id()) OR app.is_admin() OR app.has_role('FINANCE'::role))	

public.payouts

RLS: **on** (forced). Table grants: anon: SELECT; authenticated: SELECT. Column-level grants: anon SELECT on 17 column(s); authenticated SELECT on 17 column(s).

Column	Type	Null	Default
id	text	no	(gen_random_uuid())::text
order_id	text	no	
ledger_id	text	no	
payee_id	text	no	
party	text	no	
amount	integer	no	
currency	text	no	
status	text	no	'PENDING'::text
attempts	integer	no	0
idempotency_key	text	no	
source_payment_ref	text	yes	
processor_account_ref	text	yes	
processor_transfer_ref	text	yes	
failure_reason	text	yes	
created_at	timestamp with time zone	no	now()
updated_at	timestamp with time zone	no	now()
paid_at	timestamp with time zone	yes	

Constraint	Kind	Definition
payouts_amount_check	check	CHECK ((amount > 0))
payouts_attempts_check	check	CHECK ((attempts >= 0))
payouts_failed_has_reason	check	CHECK (((status <> 'FAILED'::text) OR (failure_reason IS NOT NULL)))

Constraint	Kind	Definition
payouts_failure_reason_check	check	CHECK (((failure_reason IS NULL) OR (char_length(failure_reason) <= 500)))
payouts_paid_has_ref	check	CHECK (((status <> 'PAID'::text) OR ((processor_transfer_ref IS NOT NULL) AND (paid_at IS NOT NULL))))
payouts_party_check	check	CHECK ((party = ANY (ARRAY['DESIGNER'::text, 'QC'::text])))
payouts_status_check	check	CHECK ((status = ANY (ARRAY['PENDING'::text, 'PROCESSING'::text, 'PAID'::text, 'FAILED'::text, 'REVERSED'::text])))
payouts_ledger_id_fkey	foreign key	FOREIGN KEY (ledger_id) REFERENCES escrow_ledger(id) ON DELETE RESTRICT
payouts_order_id_fkey	foreign key	FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE RESTRICT
payouts_payee_id_fkey	foreign key	FOREIGN KEY (payee_id) REFERENCES users(id) ON DELETE RESTRICT
payouts_pkey	primary key	PRIMARY KEY (id)
payouts_idempotency_key_key	unique	UNIQUE (idempotency_key)
payouts_ledger_id_key	unique	UNIQUE (ledger_id)

Policies: none (with RLS on, this table is closed to anon and authenticated; only SECURITY DEFINER functions and the service role reach it).

public.pod_materials

RLS: **on** (forced). Table grants: none to anon/authenticated.

Column	Type	Null	Default
id	text	no	
pod_id	text	no	
title	text	no	
description	text	yes	
object_key	text	no	
file_name	text	no	
content_type	text	no	
size_bytes	integer	no	
uploaded_by	text	no	
created_at	timestamp with time zone	no	now()
removed_at	timestamp with time zone	yes	

Constraint	Kind	Definition
pod_materials_description_check	check	CHECK (((description IS NULL) OR (char_length(description) <= 2000)))
pod_materials_file_name_check	check	CHECK (((char_length(file_name) >= 1) AND (char_length(file_name) <= 200)))
pod_materials_size_bytes_check	check	CHECK ((size_bytes > 0))
pod_materials_title_check	check	CHECK (((char_length(btrim(title)) >= 1) AND (char_length(btrim(title)) <= 120)))
pod_materials_pod_id_fkey	foreign key	FOREIGN KEY (pod_id) REFERENCES pods(id) ON DELETE RESTRICT
pod_materials_uploaded_by_fkey	foreign key	FOREIGN KEY (uploaded_by) REFERENCES users(id) ON DELETE RESTRICT
pod_materials_pkey	primary key	PRIMARY KEY (id)
pod_materials_object_key_key	unique	UNIQUE (object_key)

Policies: none (with RLS on, this table is closed to anon and authenticated; only SECURITY DEFINER functions and the service role reach it).

public.pod_members

RLS: **on** (forced). Table grants: authenticated: SELECT. Column-level grants: authenticated SELECT on 5 column(s).

Column	Type	Null	Default
pod_id	text	no	
user_id	text	no	
member_role	text	no	
added_by	text	yes	
added_at	timestamp with time zone	no	now()

Constraint	Kind	Definition
pod_members_member_role_check	check	CHECK ((member_role = ANY (ARRAY['POD_LEAD'::text, 'DESIGNER'::text])))
pod_members_added_by_fkey	foreign key	FOREIGN KEY (added_by) REFERENCES users(id) ON DELETE RESTRICT
pod_members_pod_id_fkey	foreign key	FOREIGN KEY (pod_id) REFERENCES pods(id) ON DELETE RESTRICT
pod_members_user_id_fkey	foreign key	FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE RESTRICT
pod_members_pkey	primary key	PRIMARY KEY (pod_id, user_id)
pod_members_user_id_key	unique	UNIQUE (user_id)

Policies:

Policy	Command	Roles	USING	WITH CHECK
pod_members_read	SELECT	{authenticated}	(app.is_admin() OR (user_id = app.current_clerk_id()) OR (pod_id = ANY (app.current_pod_ids())))	

public.pods

RLS: **on** (forced). Table grants: authenticated: SELECT. Column-level grants: authenticated SELECT on 3 column(s).

Column	Type	Null	Default
id	text	no	(gen_random_uuid())::text
name	text	no	
created_at	timestamp with time zone	no	now()

Constraint	Kind	Definition
pods_name_check	check	CHECK (((char_length(btrim(name)) >= 1) AND (char_length(btrim(name)) <= 60)))
pods_pkey	primary key	PRIMARY KEY (id)
pods_name_key	unique	UNIQUE (name)

Policies:

Policy	Command	Roles	USING	WITH CHECK
pods_read	SELECT	{authenticated}	(app.is_admin() OR (EXISTS (SELECT 1 FROM pod_members pm WHERE ((pm.pod_id = pods.id) AND (pm.user_id = app.current_clerk_id())))))	

public.preview_links

RLS: **on** (forced). Table grants: none to anon/authenticated.

Column	Type	Null	Default
id	text	no	(gen_random_uuid())::text
order_id	text	no	
token_hash	text	no	
created_by	text	no	
created_at	timestamp with time zone	no	now()
expires_at	timestamp with time zone	no	
revoked_at	timestamp with time zone	yes	
view_count	integer	no	0

Column	Type	Null	Default
last_viewed_at	timestamp with time zone	yes	

Constraint	Kind	Definition
preview_links_token_hash_check	check	CHECK ((token_hash ~ '^[0-9a-f]{64}$'::text))
preview_links_created_by_fkey	foreign key	FOREIGN KEY (created_by) REFERENCES users(id) ON DELETE RESTRICT
preview_links_order_id_fkey	foreign key	FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE RESTRICT
preview_links_pkey	primary key	PRIMARY KEY (id)
preview_links_token_hash_key	unique	UNIQUE (token_hash)

Policies: none (with RLS on, this table is closed to anon and authenticated; only SECURITY DEFINER functions and the service role reach it).

public.price_bands

RLS: **on** (forced). Table grants: authenticated: SELECT. Column-level grants: authenticated SELECT on 8 column(s).

Column	Type	Null	Default
code	text	no	
label	text	no	
first_category	character	no	
last_category	character	no	
price_cents	integer	no	
is_from	boolean	no	false
first_order_only	boolean	no	false
updated_at	timestamp with time zone	no	now()

Constraint	Kind	Definition
price_bands_check	check	CHECK ((first_category <= last_category))
price_bands_price_cents_check	check	CHECK ((price_cents > 0))
price_bands_first_category_fkey	foreign key	FOREIGN KEY (first_category) REFERENCES job_categories(letter)
price_bands_last_category_fkey	foreign key	FOREIGN KEY (last_category) REFERENCES job_categories(letter)
price_bands_pkey	primary key	PRIMARY KEY (code)

Policies:

Policy	Command	Roles	USING	WITH CHECK
price_bands_read	SELECT	{authenticated}	true	

public.public_stats

RLS: **on** (forced). Table grants: none to anon/authenticated.

Column	Type	Null	Default
week_start	date	no	
computed_at	timestamp with time zone	no	now()
designs_delivered	integer	no	
first_look_on_time_pct	integer	yes	
first_pass_approval_pct	integer	yes	
late_credits_issued	integer	no	
claims_upheld	integer	no	

Constraint	Kind	Definition
public_stats_pkey	primary key	PRIMARY KEY (week_start)

Policies: none (with RLS on, this table is closed to anon and authenticated; only SECURITY DEFINER functions and the service role reach it).

public.push_subscriptions

RLS: **on** (forced). Table grants: authenticated: SELECT. Column-level grants: authenticated SELECT on 7 column(s).

Column	Type	Null	Default
id	text	no	(gen_random_uuid())::text
user_id	text	no	
endpoint	text	no	
p256dh	text	no	
auth	text	no	
created_at	timestamp with time zone	no	now()
last_used_at	timestamp with time zone	yes	

Constraint	Kind	Definition
push_subscriptions_auth_check	check	CHECK (((auth ~ '^[A-Za-z0-9_-]+$'::text) AND ((char_length(auth) >= 8) AND (char_length(auth) <= 64))))
push_subscriptions_p256dh_check	check	CHECK (((p256dh ~ '^[A-Za-z0-9_-]+$'::text) AND ((char_length(p256dh) >= 20) AND (char_length(p256dh) <= 255))))
push_subscriptions_user_id_fkey	foreign key	FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE RESTRICT
push_subscriptions_pkey	primary key	PRIMARY KEY (id)
push_subscriptions_endpoint_key	unique	UNIQUE (endpoint)

Policies:

Policy	Command	Roles	USING	WITH CHECK
`push_subscriptions_read_own`	SELECT	{authenticated}	`(user_id = app.current_clerk_id())`	

public.qc_measurements

RLS: **on** (forced). Table grants: none to anon/authenticated.

Column	Type	Null	Default
`order_id`	text	no	
`version`	integer	no	
`min_wall_um`	integer	no	
`min_prong_um`	integer	yes	
`stone_seats`	jsonb	no	'[]'::jsonb
`ring_size`	text	yes	
`inner_diameter_um`	integer	yes	
`mesh_tolerance_um`	integer	no	5
`open_edges`	integer	no	0
`recorded_by`	text	no	
`recorded_at`	timestamp with time zone	no	now()

Constraint	Kind	Definition
`qc_measurements_inner_diameter_um_check`	check	`CHECK (((inner_diameter_um IS NULL) OR ((inner_diameter_um >= 1) AND (inner_diameter_um <= 300000))))`
`qc_measurements_mesh_tolerance_um_check`	check	`CHECK (((mesh_tolerance_um >= 1) AND (mesh_tolerance_um <= 1000)))`
`qc_measurements_min_prong_um_check`	check	`CHECK (((min_prong_um IS NULL) OR ((min_prong_um >= 1) AND (min_prong_um <= 20000))))`
`qc_measurements_min_wall_um_check`	check	`CHECK (((min_wall_um >= 1) AND (min_wall_um <= 20000)))`
`qc_measurements_open_edges_check`	check	`CHECK ((open_edges >= 0))`
`qc_measurements_ring_size_check`	check	`CHECK (((ring_size IS NULL) OR (char_length(ring_size) <= 40)))`
`qc_measurements_stone_seats_check`	check	`CHECK ((jsonb_typeof(stone_seats) = 'array'::text))`
`qc_measurements_version_check`	check	`CHECK ((version >= 1))`
`qc_measurements_order_id_fkey`	foreign key	`FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE RESTRICT`
`qc_measurements_recorded_by_fkey`	foreign key	`FOREIGN KEY (recorded_by) REFERENCES users(id) ON DELETE RESTRICT`
`qc_measurements_pkey`	primary key	`PRIMARY KEY (order_id, version)`

Policies: none (with RLS on, this table is closed to anon and authenticated; only SECURITY DEFINER functions and the service role reach it).

public.rate_limit_events

RLS: **on** (forced). Table grants: none to anon/authenticated.

Column	Type	Null	Default
`id`	bigint	no	nextval('rate_limit_events_id_seq'::regclass)
`bucket`	text	no	
`hit_at`	timestamp with time zone	no	now()

Constraint	Kind	Definition
`rate_limit_events_pkey`	primary key	`PRIMARY KEY (id)`

Policies: none (with RLS on, this table is closed to anon and authenticated; only SECURITY DEFINER functions and the service role reach it).

public.revision_requests

RLS: **on** (forced). Table grants: none to anon/authenticated.

Column	Type	Null	Default
`id`	text	no	(gen_random_uuid())::text
`order_id`	text	no	
`round`	integer	no	
`note`	text	no	
`requested_by`	text	no	
`requested_at`	timestamp with time zone	no	now()
`work_version`	integer	no	

Constraint	Kind	Definition
`revision_requests_note_check`	check	`CHECK (((char_length(btrim(note)) >= 1) AND (char_length(btrim(note)) <= 4000)))`
`revision_requests_round_check`	check	`CHECK ((round >= 1))`
`revision_requests_order_id_fkey`	foreign key	`FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE RESTRICT`
`revision_requests_requested_by_fkey`	foreign key	`FOREIGN KEY (requested_by) REFERENCES users(id) ON DELETE RESTRICT`
`revision_requests_pkey`	primary key	`PRIMARY KEY (id)`
`revision_requests_order_id_round_key`	unique	`UNIQUE (order_id, round)`

Policies: none (with RLS on, this table is closed to anon and authenticated; only SECURITY DEFINER functions and the service role reach it).

public.sla_events

RLS: **on** (forced). Table grants: authenticated: SELECT. Column-level grants: authenticated SELECT on 5 column(s).

Column	Type	Null	Default
id	text	no	(gen_random_uuid())::text
order_id	text	no	
kind	text	no	
at	timestamp with time zone	no	now()
detail	jsonb	no	'{}'::jsonb

Constraint	Kind	Definition
sla_events_kind_check	check	CHECK ((kind = ANY (ARRAY['started'::text, 'met'::text, 'breached'::text, 'credit_pending'::text, 'credit_decided'::text, 'designer_reminder'::text])))
sla_events_order_id_fkey	foreign key	FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE RESTRICT
sla_events_pkey	primary key	PRIMARY KEY (id)

Policies:

Policy	Command	Roles	USING	WITH CHECK
sla_events_read	SELECT	{authenticated}	(EXISTS (SELECT 1 FROM orders o WHERE (o.id = sla_events.order_id)))	

public.spec_claims

RLS: **on** (forced). Table grants: none to anon/authenticated.

Column	Type	Null	Default
id	text	no	(gen_random_uuid())::text
order_id	text	no	
client_id	text	no	
designer_id	text	yes	
spec	text	no	
description	text	no	
casting_details	text	no	
uploaded_sha256	text	no	
file_object_key	text	no	
photo_keys	_text	no	
hash_match	boolean	no	

Column	Type	Null	Default
matched_version	integer	yes	
status	text	no	
not_covered_reason	text	yes	
measured	jsonb	yes	
decision_note	text	yes	
decided_by	text	yes	
decided_at	timestamp with time zone	yes	
fix_order_id	text	yes	
created_at	timestamp with time zone	no	now()

Constraint	Kind	Definition
spec_claims_casting_details_check	check	CHECK (((char_length(btrim(casting_details)) >= 1) AND (char_length(btrim(casting_details)) <= 2000)))
spec_claims_decision_note_check	check	CHECK (((decision_note IS NULL) OR (char_length(decision_note) <= 4000)))
spec_claims_description_check	check	CHECK (((char_length(btrim(description)) >= 1) AND (char_length(btrim(description)) <= 4000)))
spec_claims_photo_keys_check	check	CHECK (((cardinality(photo_keys) >= 1) AND (cardinality(photo_keys) <= 10)))
spec_claims_spec_check	check	CHECK ((spec = ANY (ARRAY['wall'::text, 'prong'::text, 'stone_seat'::text, 'size'::text, 'mesh'::text, 'open_edges'::text, 'other'::text])))
spec_claims_status_check	check	CHECK ((status = ANY (ARRAY['not_covered'::text, 'open'::text, 'admin_review'::text, 'upheld'::text, 'rejected'::text])))
spec_claims_uploaded_sha256_check	check	CHECK ((uploaded_sha256 ~ '^[0-9a-f]{64}$'::text))
spec_claims_client_id_fkey	foreign key	FOREIGN KEY (client_id) REFERENCES users(id) ON DELETE RESTRICT
spec_claims_decided_by_fkey	foreign key	FOREIGN KEY (decided_by) REFERENCES users(id) ON DELETE RESTRICT
spec_claims_designer_id_fkey	foreign key	FOREIGN KEY (designer_id) REFERENCES users(id) ON DELETE RESTRICT
spec_claims_fix_order_id_fkey	foreign key	FOREIGN KEY (fix_order_id) REFERENCES orders(id) ON DELETE RESTRICT
spec_claims_order_id_fkey	foreign key	FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE RESTRICT

Constraint	Kind	Definition
spec_claims_pkey	primary key	PRIMARY KEY (id)

Policies: none (with RLS on, this table is closed to anon and authenticated; only SECURITY DEFINER functions and the service role reach it).

public.spec_waivers

RLS: **on** (forced). Table grants: none to anon/authenticated.

Column	Type	Null	Default
id	text	no	(gen_random_uuid())::text
order_id	text	no	
spec	text	no	
requested	text	no	
note	text	yes	
recorded_by	text	no	
created_at	timestamp with time zone	no	now()

Constraint	Kind	Definition
spec_waivers_note_check	check	CHECK (((note IS NULL) OR (char_length(note) <= 1000)))
spec_waivers_requested_check	check	CHECK (((char_length(btrim(requested)) >= 1) AND (char_length(btrim(requested)) <= 200)))
spec_waivers_spec_check	check	CHECK ((spec = ANY (ARRAY['wall'::text, 'prong'::text, 'stone_seat'::text, 'size'::text, 'mesh'::text, 'other'::text])))
spec_waivers_order_id_fkey	foreign key	FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE RESTRICT
spec_waivers_recorded_by_fkey	foreign key	FOREIGN KEY (recorded_by) REFERENCES users(id) ON DELETE RESTRICT
spec_waivers_pkey	primary key	PRIMARY KEY (id)

Policies: none (with RLS on, this table is closed to anon and authenticated; only SECURITY DEFINER functions and the service role reach it).

public.staff_profiles

RLS: **on** (forced). Table grants: authenticated: SELECT. Column-level grants: authenticated SELECT on 3 column(s).

Column	Type	Null	Default
user_id	text	no	
first_name	text	no	
updated_at	timestamp with time zone	no	now()

Constraint	Kind	Definition
staff_profiles_first_name_check	check	CHECK (((char_length(btrim(first_name)) >= 1) AND (char_length(btrim(first_name)) <= 40)))
staff_profiles_user_id_fkey	foreign key	FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE RESTRICT
staff_profiles_pkey	primary key	PRIMARY KEY (user_id)

Policies:

Policy	Command	Roles	USING	WITH CHECK
staff_profiles_read	SELECT	{authenticated}	((user_id = app.current_clerk_id()) OR app.has_role('OPS'::role))	

public.thread_files

RLS: **on** (forced). Table grants: none to anon/authenticated.

Column	Type	Null	Default
id	text	no	(gen_random_uuid())::text
thread_id	text	no	
message_id	text	no	
order_id	text	no	
object_key	text	no	
content_type	text	no	
size_bytes	integer	no	
file_name	text	no	
added_by	text	no	
created_at	timestamp with time zone	no	now()

Constraint	Kind	Definition
thread_files_file_name_check	check	CHECK (((char_length(file_name) >= 1) AND (char_length(file_name) <= 120)))
thread_files_size_bytes_check	check	CHECK ((size_bytes > 0))
thread_files_added_by_fkey	foreign key	FOREIGN KEY (added_by) REFERENCES users(id) ON DELETE RESTRICT
thread_files_message_id_fkey	foreign key	FOREIGN KEY (message_id) REFERENCES messages(id) ON DELETE RESTRICT
thread_files_order_id_fkey	foreign key	FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE RESTRICT
thread_files_thread_id_fkey	foreign key	FOREIGN KEY (thread_id) REFERENCES message_threads(id) ON DELETE RESTRICT

Constraint	Kind	Definition
`thread_files_pkey`	primary key	`PRIMARY KEY (id)`
`thread_files_object_key_key`	unique	`UNIQUE (object_key)`

Policies: none (with RLS on, this table is closed to anon and authenticated; only SECURITY DEFINER functions and the service role reach it).

Triggers:

- `thread_files_signal_thread`: `AFTER INSERT ON public.thread_files FOR EACH ROW EXECUTE FUNCTION app.signal_thread_change()`

public.thread_participants

RLS: **on** (forced). Table grants: none to anon/authenticated.

Column	Type	Null	Default
`thread_id`	text	no	
`user_id`	text	no	
`party`	text	no	
`alias`	text	no	
`joined_at`	timestamp with time zone	no	now()

Constraint	Kind	Definition
`thread_participants_alias_check`	check	`CHECK (((char_length(alias) >= 1) AND (char_length(alias) <= 60)))`
`thread_participants_party_check`	check	`CHECK ((party = ANY (ARRAY['CLIENT'::text, 'DESIGNER'::text, 'TEAM'::text])))`
`thread_participants_thread_id_fkey`	foreign key	`FOREIGN KEY (thread_id) REFERENCES message_threads(id) ON DELETE RESTRICT`
`thread_participants_user_id_fkey`	foreign key	`FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE RESTRICT`
`thread_participants_pkey`	primary key	`PRIMARY KEY (thread_id, user_id)`

Policies: none (with RLS on, this table is closed to anon and authenticated; only SECURITY DEFINER functions and the service role reach it).

public.thread_reads

RLS: **on** (forced). Table grants: none to anon/authenticated.

Column	Type	Null	Default
`thread_id`	text	no	
`user_id`	text	no	
`party`	text	no	
`read_at`	timestamp with time zone	no	

Constraint	Kind	Definition
`thread_reads_party_check`	check	`CHECK ((party = ANY (ARRAY['CLIENT'::text, 'TEAM'::text, 'DESIGNER'::text])))`
`thread_reads_thread_id_fkey`	foreign key	`FOREIGN KEY (thread_id) REFERENCES message_threads(id) ON DELETE RESTRICT`
`thread_reads_pkey`	primary key	`PRIMARY KEY (thread_id, user_id)`

Policies: none (with RLS on, this table is closed to anon and authenticated; only SECURITY DEFINER functions and the service role reach it).

Triggers:

- `thread_reads_signal_thread`: `AFTER INSERT OR UPDATE ON public.thread_reads FOR EACH ROW EXECUTE FUNCTION app.signal_thread_change()`

public.user_roles

RLS: **on** (forced). Table grants: authenticated: SELECT. Column-level grants: authenticated SELECT on 4 column(s).

Column	Type	Null	Default
`user_id`	text	no	
`role`	role	no	
`granted_by`	text	yes	
`granted_at`	timestamp with time zone	no	now()

Constraint	Kind	Definition
`user_roles_staff_only`	check	`CHECK ((role = ANY (ARRAY['SALES'::role, 'OPS'::role, 'FINANCE'::role, 'QC'::role])))`
`user_roles_granted_by_fkey`	foreign key	`FOREIGN KEY (granted_by) REFERENCES users(id) ON DELETE RESTRICT`
`user_roles_user_id_fkey`	foreign key	`FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE RESTRICT`
`user_roles_pkey`	primary key	`PRIMARY KEY (user_id, role)`

Policies:

Policy	Command	Roles	USING	WITH CHECK
`user_roles_read`	SELECT	{authenticated}	`((user_id = app.current_clerk_id()) OR app.has_role('OPS'::role))`	

public.users

RLS: **on** (forced). Table grants: anon: SELECT; authenticated: SELECT. Column-level grants: anon SELECT on 5 column(s); authenticated SELECT on 5 column(s).

Column	Type	Null	Default
`id`	text	no	
`role`	role	no	
`status`	user_status	no	'PENDING'::user_status
`created_at`	timestamp with time zone	no	now()
`deleted_at`	timestamp with time zone	yes	

Constraint	Kind	Definition
`users_pkey`	primary key	`PRIMARY KEY (id)`

Policies:

Policy	Command	Roles	USING	WITH CHECK
`users_self_select`	SELECT	{authenticated}	`(id = app.current_clerk_id())`	

APPENDIX

Appendix B · Database functions reference

> Generated from the repository at commit `1c08b52`. Every row below is read from the code or from a database built from `db/migrations` and `db/policies`, not written by hand.

Summary

Measure	Count
Functions (public, app, audit)	285
SECURITY DEFINER	254
SECURITY DEFINER without a pinned search_path	0
Executable by anon	56
Executable by authenticated	221

A SECURITY DEFINER function runs with its owner's rights, so each one is a door through RLS. Each checks the caller itself (usually with `app.current_clerk_id()` and the role helpers). Pinning `search_path` stops a caller from shadowing the tables it uses.

Schema `public` (186)

Function	Returns	Mode	anon	authenticated
`accept_designer_agreement(p_expected_sha256 text)`	jsonb	DEFINER	—	yes
`add_designer_document(p_user_id text, p_kind text, p_object_key text, p_file_name text, p_mime text, p_size_bytes integer)`	text	DEFINER	—	yes
`add_file_version(p_id text, p_order_id text, p_object_key text, p_content_type text, p_size_bytes integer, p_kind file_kind)`	text	DEFINER	—	yes
`add_order_screenshot(p_order_id text, p_object_key text, p_content_type text, p_size_bytes integer)`	jsonb	DEFINER	—	yes

Function	Returns	Mode	anon	authenticated
`add_pod_material(p_id text, p_pod_id text, p_title text, p_description text, p_object_key text, p_file_name text, p_content_type text, p_size_bytes integer)`	void	DEFINER	—	yes
`add_reference_image(p_order_id text, p_object_key text, p_content_type text, p_size_bytes integer)`	text	DEFINER	yes	yes
`add_slot_file(p_id text, p_order_id text, p_slot text, p_object_key text, p_content_type text, p_size_bytes integer, p_extension text)`	jsonb	DEFINER	—	yes
`add_spec_waiver(p_order_id text, p_spec text, p_requested text, p_note text)`	text	DEFINER	—	yes
`add_stone_certificate(p_order_id text, p_object_key text, p_content_type text, p_size_bytes integer)`	text	DEFINER	—	yes
`admin_clients()`	TABLE(client_id text, client_code text, joined_at timestamp with time zone, orders integer, open_orders integer, paid_cents bigint, last_order_at timestamp with time zone, manager_id text, manager_name text)	DEFINER	—	yes
`admin_designers()`	TABLE(user_id text, legal_name text, email text, phone text, status text, agreement_signed boolean, designer_ref text, pod_id text, pod_name text, payout_status text, account_last4 text, documents integer, created_at timestamp with time zone)	DEFINER	—	yes
`apply_as_designer(p_profile_id text, p_legal_name text, p_email text, p_country text)`	jsonb	DEFINER	—	yes

Function	Returns	Mode	anon	authenticated
`assign_designer(p_order_id text, p_designer_id text, p_override_reason text)`	jsonb	DEFINER	—	yes
`assign_order_pod(p_order_id text, p_pod_lead_id text)`	jsonb	DEFINER	—	yes
`attribution_report()`	jsonb	DEFINER	—	yes
`brief_missing(p_order_id text)`	text[]	DEFINER	—	yes
`certificate_data(p_order_id text, p_version integer)`	jsonb	DEFINER	—	yes
`certificate_for_download(p_order_id text, p_version integer)`	TABLE(object_key text, version integer, sha256 text)	DEFINER	—	yes
`check_rate_limit(p_bucket text, p_max_hits integer, p_window_seconds integer)`	boolean	DEFINER	yes	yes
`claim_emails(p_limit integer)`	SETOF email_outbox	DEFINER	—	—
`claim_payouts(p_limit integer)`	SETOF payouts	DEFINER	—	—
`claim_push_notifications(p_limit integer, p_max_age interval, p_attempts integer)`	SETOF notifications	DEFINER	—	—
`claim_window(p_order_id text)`	jsonb	DEFINER	—	yes
`claims_queue()`	TABLE(id text, order_id text, order_ref text, client_code text, designer_ref text, spec text, status text, hash_match boolean, created_at timestamp with time zone, designer_upheld_90d integer)	DEFINER	—	yes
`compute_public_stats()`	void	DEFINER	—	yes
`confirm_order(p_order_id text)`	jsonb	DEFINER	—	yes
`confirm_payment(p_external_ref text, p_amount integer, p_currency text, p_idempotency_key text, p_payment_ref text)`	jsonb	DEFINER	—	—

Function	Returns	Mode	anon	authenticated
`create_change_order(p_parent_id text, p_new_id text, p_note text)`	jsonb	DEFINER	—	yes
`create_order(p_id text, p_product_type text, p_currency text)`	text	DEFINER	—	yes
`create_order_on_behalf(p_id text, p_client_id text, p_source order_source, p_product product_kind, p_service service_kind, p_received_at timestamp with time zone, p_note text, p_pod_id text)`	jsonb	DEFINER	—	yes
`create_pod(p_name text)`	text	DEFINER	—	yes
`create_preview_link(p_order_id text, p_token text, p_days integer)`	text	DEFINER	—	yes
`decide_sla_credit(p_credit_id text, p_decision text, p_amount_cents integer, p_reason text)`	jsonb	DEFINER	—	yes
`decide_spec_claim(p_claim_id text, p_upheld boolean, p_measured jsonb, p_note text, p_fix_order_id text)`	jsonb	DEFINER	—	yes
`delete_push_subscription(p_endpoint text)`	integer	DEFINER	—	yes
`designer_document_open(p_id text)`	TABLE(object_key text, file_name text, mime text)	DEFINER	—	yes
`designer_documents_for(p_user_id text)`	TABLE(id text, kind text, file_name text, mime text, size_bytes integer, created_at timestamp with time zone)	DEFINER	—	yes
`enqueue_email_reminders()`	integer	DEFINER	—	—
`ensure_self()`	jsonb	DEFINER	—	yes
`escalate_job(p_order_id text, p_reason text)`	text	DEFINER	—	yes

Function	Returns	Mode	anon	authenticated
`expire_push_subscription(p_endpoint text)`	integer	DEFINER	—	—
`export_payouts()`	jsonb	DEFINER	—	yes
`fail_payment_intent(p_external_ref text)`	jsonb	DEFINER	—	—
`find_clients(p_query text)`	TABLE(user_id text, client_code text, business_name text, contact_name text, email text, phone text)	DEFINER	—	yes
`grant_user_role(p_user_id text, p_role text)`	jsonb	DEFINER	—	yes
`has_attribution()`	boolean	DEFINER	—	yes
`hold_escrow(p_order_id text)`	jsonb	DEFINER	—	—
`i_am_team()`	boolean	DEFINER	—	yes
`job_brief(p_order_id text)`	jsonb	DEFINER	—	yes
`job_checklist(p_order_id text)`	TABLE(item_id smallint, group_name text, label text, standard text, flag text, is_signoff boolean, stage text, “position” smallint, outcome text, finding text, remark text, designer_at timestamp with time zone, qc_at timestamp with time zone)	DEFINER	—	yes
`job_escalations_list(p_include_resolved boolean)`	TABLE(id text, order_id text, order_ref text, pod_name text, reason text, created_at timestamp with time zone, resolved_at timestamp with time zone, resolution text)	DEFINER	—	yes
`job_slots(p_order_id text)`	TABLE(code text, label text, required boolean, file_version_id text, file_name text, uploaded_at timestamp with time zone)	DEFINER	—	yes

Function	Returns	Mode	anon	authenticated
`job_times(p_order_id text)`	TABLE(order_id text, version integer, status order_status, assigned_at timestamp with time zone, started_at timestamp with time zone, submitted_at timestamp with time zone, paused_at timestamp with time zone, paused_seconds integer, active_seconds integer)	DEFINER	—	yes
`job_weights_get(p_order_id text)`	jsonb	DEFINER	—	yes
`list_designer_applications(p_status text)`	TABLE(id text, full_name text, email text, phone text, country text, years_experience integer, primary_software text, categories text[], portfolio_url text, portfolio_file_keys text[], status text, review_notes text, reviewed_at timestamp with time zone, created_at timestamp with time zone)	DEFINER	—	yes
`list_marketing_leads(p_status text)`	TABLE(id text, name text, company text, email text, role text, message text, status text, handled_at timestamp with time zone, created_at timestamp with time zone)	DEFINER	—	yes
`list_platform_users(p_search text, p_role text, p_limit integer)`	TABLE(id text, role role, status user_status, created_at timestamp with time zone, orders_as_client integer, orders_as_designer integer, reference text, reviews integer)	DEFINER	—	yes
`mark_check(p_order_id text, p_item_id integer, p_outcome text, p_finding text, p_remark text)`	jsonb	DEFINER	—	yes
`mark_notifications_read(p_id text)`	integer	DEFINER	—	yes
`mark_payment_received(p_order_id text, p_method text, p_reference text)`	jsonb	DEFINER	—	yes

Function	Returns	Mode	anon	authenticated
`mark_payout_batch_paid(p_batch text)`	integer	DEFINER	—	yes
`mark_push_sent(p_ids text[])`	integer	DEFINER	—	—
`mark_thread_read(p_thread_id text)`	timestamp with time zone	DEFINER	—	yes
`material_pods()`	TABLE(id text, name text)	DEFINER	—	yes
`min_wall_um(p_purpose cad_purpose)`	integer	invoker	yes	yes
`money_ledger(p_limit integer)`	TABLE(id text, order_id text, order_ref text, client_code text, kind text, amount integer, currency text, created_at timestamp with time zone)	DEFINER	—	yes
`my_conversations()`	TABLE(order_id text, display_ref text, status order_status, thread_id text, kind text, last_body text, last_alias text, last_mine boolean, last_at timestamp with time zone, messages integer)	DEFINER	—	yes
`my_earnings()`	TABLE(obligation_id text, order_id text, display_ref text, product text, category character, minutes integer, party text, amount_paise integer, status text, created_at timestamp with time zone, exported_at timestamp with time zone, paid_at timestamp with time zone)	DEFINER	—	yes
`my_email_preferences()`	jsonb	DEFINER	—	yes
`my_master_instructions()`	TABLE(body text, updated_at timestamp with time zone)	DEFINER	—	yes
`my_order_relation(p_order_id text)`	text	DEFINER	—	yes
`my_payout_account()`	jsonb	DEFINER	—	yes
`my_payouts(p_limit integer)`	TABLE(order_id text, party text, amount integer, currency text, status text, created_at timestamp with time zone, paid_at timestamp with time zone)	DEFINER	—	yes

Function	Returns	Mode	anon	authenticated
`my_preview_links(p_order_id text)`	TABLE(id text, created_at timestamp with time zone, expires_at timestamp with time zone, revoked_at timestamp with time zone, view_count integer, last_viewed_at timestamp with time zone)	DEFINER	—	yes
`my_work(p_order_id text)`	TABLE(order_id text, job_code text, product text, centre_shape text, service service_kind, status order_status, category character, minutes integer, pay_paise integer, work_version integer, assigned_at timestamp with time zone, started_at timestamp with time zone, submitted_at timestamp with time zone, designer_due_at timestamp with time zone)	DEFINER	—	yes
`open_payment_intent(p_order_id text, p_external_ref text)`	jsonb	DEFINER	—	—
`open_payouts_for_order(p_order_id text)`	jsonb	DEFINER	—	—
`open_spec_claim(p_client_id text, p_order_id text, p_spec text, p_description text, p_casting_details text, p_uploaded_sha256 text, p_file_object_key text, p_photo_keys text[])`	jsonb	DEFINER	—	—
`order_activity_stamp(p_order_id text)`	bigint	DEFINER	—	yes

Function	Returns	Mode	anon	authenticated
`order_claims(p_order_id text)`	TABLE(id text, spec text, description text, casting_details text, hash_match boolean, matched_version integer, status text, not_covered_reason text, measured jsonb, decision_note text, decided_at timestamp with time zone, fix_order_id text, created_at timestamp with time zone, file_object_key text, photo_keys text[])	DEFINER	—	yes
`order_client_contact(p_order_id text)`	TABLE(business_name text, contact_name text, email text, phone text)	DEFINER	—	yes
`order_fingerprints(p_order_id text)`	TABLE(file_version_id text, slot text, version integer, sha256 text, size_bytes integer, archived_at timestamp with time zone)	DEFINER	—	yes
`order_lineage(p_order_id text)`	TABLE(direction text, order_id text, order_ref text, status order_status)	DEFINER	—	yes
`order_master_instructions(p_order_id text)`	TABLE(body text, updated_at timestamp with time zone)	DEFINER	—	yes
`order_messages(p_order_id text)`	TABLE(id text, thread_id text, thread_kind text, sender_party text, sender_alias text, is_mine boolean, body text, created_at timestamp with time zone, client_ref text)	DEFINER	—	yes
`order_revisions(p_order_id text)`	TABLE(round integer, note text, requested_at timestamp with time zone, work_version integer)	DEFINER	—	yes
`order_stone_certificates_for(p_order_id text)`	TABLE(id text, object_key text, content_type text, created_at timestamp with time zone)	DEFINER	—	yes
`order_threads(p_order_id text)`	TABLE(id text, kind text, can_post boolean)	DEFINER	—	yes

Function	Returns	Mode	anon	authenticated
`order_timeline(p_order_id text)`	TABLE(seq bigint, created_at timestamp with time zone, action text, actor_role role, from_status text, to_status text, amount integer, detail text)	DEFINER	—	yes
`pause_job(p_order_id text)`	timestamp with time zone	DEFINER	—	yes
`payout_account_full(p_user_id text)`	TABLE(beneficiary_name text, pan text, account_number text, ifsc text, account_type text, status text, updated_at timestamp with time zone)	DEFINER	—	yes
`payout_batch_rows(p_batch text)`	TABLE(obligation_id text, order_ref text, party text, beneficiary_name text, account_number text, ifsc text, account_type text, bank_status text, amount_paise integer)	DEFINER	—	yes
`payout_list()`	TABLE(obligation_id text, order_id text, order_ref text, party text, payee_id text, payee_name text, bank_status text, amount_paise integer, status text, export_batch text, created_at timestamp with time zone, exported_at timestamp with time zone, paid_at timestamp with time zone)	DEFINER	—	yes
`payout_state(p_order_id text)`	jsonb	DEFINER	—	—
`pending_sla_credits()`	TABLE(credit_id text, order_id text, order_ref text, client_code text, amount_cents integer, currency text, due_at timestamp with time zone, created_at timestamp with time zone)	DEFINER	—	yes
`pod_activity(p_since timestamp with time zone)`	TABLE(order_id text, designer_id text, at timestamp with time zone, kind text, from_status text, to_status text)	DEFINER	—	yes

Function	Returns	Mode	anon	authenticated
`pod_designers(p_order_id text)`	TABLE(designer_id text, designer_ref text, name text, open_minutes integer, shift_minutes integer, assignable boolean)	DEFINER	—	yes
`pod_jobs(p_order_id text)`	TABLE(order_id text, order_ref text, title text, product text, service service_kind, status order_status, category character, minutes integer, work_version integer, pod_id text, pod_name text, pod_lead_name text, designer_id text, designer_ref text, designer_name text, client_code text, started_at timestamp with time zone, submitted_at timestamp with time zone, qc_ticks integer, designer_ticks integer, sla_status sla_status, sla_due_at timestamp with time zone, designer_due_at timestamp with time zone, first_look_at timestamp with time zone, confirmed_at timestamp with time zone, paid_at timestamp with time zone, source order_source, dispute_reason text, currency text, price_total integer, qc_payout_paise integer, designer_payout_paise integer, created_at timestamp with time zone)	DEFINER	—	yes
`pod_load()`	TABLE(designer_id text, designer_ref text, name text, pod_id text, pod_name text, open_minutes integer, shift_minutes integer, assignable boolean)	DEFINER	—	yes
`pod_material_file(p_id text)`	jsonb	DEFINER	—	yes

Function	Returns	Mode	anon	authenticated
`pod_materials()`	TABLE(id text, pod_id text, pod_name text, title text, description text, file_name text, content_type text, size_bytes integer, created_at timestamp with time zone, can_manage boolean)	DEFINER	—	yes
`pods_overview()`	TABLE(pod_id text, name text, lead_id text, lead_name text, designers integer, open_orders integer)	DEFINER	—	yes
`post_message(p_order_id text, p_body text)`	jsonb	DEFINER	—	yes
`post_thread_file(p_thread_id text, p_object_key text, p_content_type text, p_size_bytes integer, p_extension text, p_note text)`	jsonb	DEFINER	—	yes
`post_thread_message(p_thread_id text, p_body text)`	jsonb	DEFINER	—	yes
`preview_by_token(p_token text)`	jsonb	DEFINER	—	—
`public_stats_latest()`	TABLE(week_start date, computed_at timestamp with time zone, designs_delivered integer, first_look_on_time_pct integer, first_pass_approval_pct integer, late_credits_issued integer, claims_upheld integer)	DEFINER	yes	yes
`qc_decide(p_order_id text, p_outcome text, p_notes text)`	jsonb	DEFINER	—	yes
`quote_order_v2(p_order_id text, p_category text, p_price_cents integer, p_complex_deadline_hours integer)`	jsonb	DEFINER	—	yes
`raise_dispute(p_order_id text, p_reason text)`	jsonb	DEFINER	—	yes
`reassign_job(p_order_id text, p_designer_id text, p_reason text)`	jsonb	DEFINER	—	yes

Function	Returns	Mode	anon	authenticated
`record_certificate(p_order_id text, p_version integer, p_object_key text, p_sha256 text, p_size_bytes integer)`	void	DEFINER	—	—
`record_email_result(p_idempotency_key text, p_status text, p_provider_ref text, p_failure_reason text)`	jsonb	DEFINER	—	—
`record_file_fingerprint(p_file_version_id text, p_sha256 text, p_size_bytes integer)`	void	DEFINER	—	—
`record_identity_lookup(p_user_ids text[], p_purpose text)`	integer	DEFINER	—	yes
`record_marketing_event(p_event text, p_order_id text, p_visitor_id text, p_props jsonb)`	void	DEFINER	yes	yes
`record_payout_result(p_idempotency_key text, p_status text, p_transfer_ref text, p_failure_reason text)`	jsonb	DEFINER	—	—
`record_qc_decision(p_order_id text, p_outcome text, p_notes text)`	jsonb	DEFINER	—	yes
`record_qc_measurements(p_order_id text, p_min_wall_um integer, p_min_prong_um integer, p_stone_seats jsonb, p_ring_size text, p_inner_diameter_um integer, p_mesh_tolerance_um integer, p_open_edges integer)`	void	DEFINER	—	yes
`record_settlement_event(p_order_id text, p_kind text, p_amount integer, p_idempotency_key text, p_external_ref text, p_payee_id text)`	jsonb	DEFINER	—	—
`refund_escrow(p_order_id text, p_amount integer)`	jsonb	DEFINER	—	yes

Function	Returns	Mode	anon	authenticated
`register_client(p_user_id text, p_business_name text, p_contact_name text, p_email text, p_phone text)`	jsonb	DEFINER	—	yes
`register_designer(p_user_id text, p_legal_name text, p_email text, p_phone text)`	jsonb	DEFINER	—	yes
`release_escrow(p_order_id text)`	jsonb	DEFINER	—	yes
`remove_designer_document(p_id text)`	text	DEFINER	—	yes
`remove_pod_material(p_id text)`	void	DEFINER	—	yes
`remove_pod_member(p_user_id text)`	jsonb	DEFINER	—	yes
`remove_reference_image(p_image_id text)`	void	DEFINER	yes	yes
`remove_stone_certificate(p_id text)`	text	DEFINER	—	yes
`request_revision(p_order_id text, p_note text)`	jsonb	DEFINER	—	yes
`resolve_dispute(p_order_id text, p_resolution text, p_notes text)`	jsonb	DEFINER	—	yes
`resolve_escalation(p_id text, p_note text)`	void	DEFINER	—	yes
`resume_job(p_order_id text)`	timestamp with time zone	DEFINER	—	yes
`review_designer_application(p_id text, p_decision text, p_notes text)`	jsonb	DEFINER	—	yes
`revision_allowance(p_order_id text)`	jsonb	DEFINER	—	yes
`revoke_user_role(p_user_id text, p_role text)`	jsonb	DEFINER	—	yes
`save_attribution(p_heard_about text, p_first_touch jsonb)`	void	DEFINER	—	yes
`save_client_profile(p_business_name text, p_legal_name text, p_email text, p_phone text)`	jsonb	DEFINER	—	yes

Function	Returns	Mode	anon	authenticated
`save_push_subscription(p_endpoint text, p_p256dh text, p_auth text)`	text	DEFINER	—	yes
`send_chat_message(p_thread_id text, p_body text, p_client_ref text)`	jsonb	DEFINER	—	yes
`set_account_manager(p_client_id text, p_manager_id text)`	jsonb	DEFINER	—	yes
`set_app_setting(p_key text, p_value integer)`	void	DEFINER	—	yes
`set_client_logo(p_object_key text)`	void	DEFINER	—	yes
`set_designer_rate(p_user_id text, p_paise_per_minute integer)`	void	DEFINER	—	yes
`set_email_preferences(p_messages boolean, p_reminders boolean, p_marketing boolean)`	void	DEFINER	—	yes
`set_feature_flag(p_key text, p_enabled boolean)`	void	DEFINER	—	yes
`set_job_volume(p_order_id text, p_volume_mm3 numeric)`	void	DEFINER	—	yes
`set_job_weights(p_order_id text, p_gold_18k_g numeric, p_casting_allowance_g numeric, p_post_allowance_g numeric, p_stones jsonb)`	jsonb	DEFINER	—	yes
`set_lead_status(p_id text, p_status text)`	jsonb	DEFINER	—	yes
`set_master_instructions(p_body text, p_client_id text)`	text	DEFINER	—	yes
`set_order_accents(p_order_id text, p_shapes stone_shape[], p_widths_um integer[], p_quantities integer[], p_settings setting_type[])`	integer	DEFINER	yes	yes
`set_order_category(p_order_id text, p_letter text)`	jsonb	DEFINER	—	yes

Function	Returns	Mode	anon	authenticated
`set_order_master_instructions(p_order_id text, p_body text)`	text	DEFINER	—	yes
`set_order_service(p_order_id text, p_service service_kind)`	void	DEFINER	—	yes
`set_payout_account_for(p_user_id text, p_beneficiary_name text, p_pan text, p_account_number text, p_ifsc text, p_account_type text, p_verified boolean)`	jsonb	DEFINER	—	yes
`set_payout_account_status(p_user_id text, p_status text, p_reason text, p_processor_account_ref text, p_processor_fund_account_ref text)`	jsonb	DEFINER	—	—
`set_pod_member(p_pod_id text, p_user_id text, p_member_role text)`	jsonb	DEFINER	—	yes
`set_primary_reference(p_image_id text)`	void	DEFINER	yes	yes
`set_reference_pins(p_image_id text, p_xs integer[], p_ys integer[], p_labels text[])`	integer	DEFINER	yes	yes
`set_staff_first_name(p_user_id text, p_first_name text)`	void	DEFINER	—	yes
`set_tech_source(p_order_id text, p_file_version_id text)`	void	DEFINER	—	yes
`set_user_role(p_user_id text, p_role text)`	jsonb	DEFINER	—	yes
`set_user_status(p_user_id text, p_status text)`	jsonb	DEFINER	—	yes
`settlement_state(p_order_id text)`	jsonb	DEFINER	—	—
`sla_sweep(p_now timestamp with time zone)`	integer	DEFINER	—	—
`stale_payouts(p_minutes integer)`	SETOF payouts	DEFINER	—	—

Function	Returns	Mode	anon	authenticated
`start_brief_lead(p_email text, p_product text, p_note text)`	text	DEFINER	yes	yes
`submit_brief(p_order_id text)`	jsonb	DEFINER	—	yes
`submit_designer_application(p_id text, p_full_name text, p_email text, p_phone text, p_country text, p_years_experience integer, p_primary_software text, p_categories text[], p_portfolio_url text, p_portfolio_file_keys text[])`	text	DEFINER	yes	yes
`submit_marketing_lead(p_name text, p_email text, p_message text, p_company text, p_role text)`	text	DEFINER	yes	yes
`submit_order_on_behalf(p_order_id text, p_notify boolean)`	jsonb	DEFINER	—	yes
`summary_sheet_data(p_order_id text)`	jsonb	DEFINER	—	yes
`sweep_job_alerts()`	integer	DEFINER	—	—
`team_designer_rates()`	TABLE(designer_id text, paise_per_minute integer, is_custom boolean, standard_paise_per_minute integer)	DEFINER	—	yes
`thread_attachments(p_order_id text)`	TABLE(id text, message_id text, thread_id text, file_name text, content_type text, size_bytes integer, created_at timestamp with time zone)	DEFINER	—	yes
`thread_file_for_download(p_file_id text)`	TABLE(object_key text, content_type text, file_name text)	DEFINER	—	yes
`thread_read_state(p_order_id text)`	TABLE(thread_id text, others_read_at timestamp with time zone)	DEFINER	—	yes
`tick_check(p_order_id text, p_item_id integer, p_column text, p_ticked boolean)`	jsonb	DEFINER	—	yes

Function	Returns	Mode	anon	authenticated
`transition_order(p_order_id text, p_new_status order_status, p_payload jsonb)`	jsonb	DEFINER	—	yes
`update_preview_link(p_link_id text, p_revoke boolean, p_days integer)`	void	DEFINER	—	yes
`upsert_order_spec(p_order_id text, p_reference_name text, p_product product_kind, p_metal metal_colour, p_karatage text, p_purpose cad_purpose, p_format output_format, p_finish finish_type, p_has_centre_stone boolean, p_centre_shape stone_shape, p_centre_length_um integer, p_centre_width_um integer, p_centre_depth_um integer, p_centre_carat_mct integer, p_centre_certified boolean, p_centre_quantity integer, p_centre_setting setting_type, p_stones_supplied_by stone_supply, p_component_count integer, p_render_views integer, p_priority priority_tier, p_based_on_order_id text, p_change_summary text, p_notes text, p_ring_size text, p_inner_circumference_um integer, p_inner_diameter_um integer, p_length_um integer)`	void	DEFINER	—	yes
`upsert_payout_account(p_beneficiary_name text, p_pan text, p_account_number text, p_ifsc text, p_account_type text)`	jsonb	DEFINER	—	yes

Function	Returns	Mode	anon	authenticated
`version_spec(p_order_id text)`	jsonb	DEFINER	—	yes
`visible_orders(p_order_id text)`	TABLE(id text, relation text, display_ref text, product_type text, status order_status, service service_kind, category character, tier job_tier, currency text, price_total integer, designer_payout integer, qc_payout integer, platform_commission integer, designer_payout_paise integer, qc_payout_paise integer, sla_status sla_status, sla_due_at timestamp with time zone, designer_due_at timestamp with time zone, first_look_at timestamp with time zone, source order_source, created_at timestamp with time zone, confirmed_at timestamp with time zone, paid_at timestamp with time zone, complex_deadline_hours integer)	DEFINER	—	yes
`weight_sheet_data(p_order_id text)`	jsonb	DEFINER	—	yes
`withdraw_job(p_order_id text, p_reason text)`	jsonb	DEFINER	—	yes

Schema `app` (95)

Function	Returns	Mode	anon	authenticated
`acting_role(p_role role)`	role	invoker	yes	yes
`archive_on_delivery()`	trigger	DEFINER	—	—
`assign_order_ref()`	trigger	DEFINER	yes	yes
`band_price(p_letter character, p_first_design boolean)`	TABLE(price_cents integer, band text, is_from boolean)	invoker	yes	yes
`can_manage_materials(p_pod_id text)`	boolean	DEFINER	—	yes

Function	Returns	Mode	anon	authenticated
`can_read_certificate(p_order_id text)`	boolean	DEFINER	—	yes
`can_read_thread(p_thread_id text)`	boolean	DEFINER	yes	yes
`can_receive_thread_signal(p_topic text)`	boolean	DEFINER	—	yes
`can_see_screenshots(p_order_id text)`	boolean	DEFINER	—	yes
`certificates_immutable()`	trigger	invoker	yes	yes
`claim_window_for(p_order_id text)`	jsonb	DEFINER	—	—
`clean_first_touch(p jsonb)`	jsonb	invoker	yes	yes
`cleanable_type(p_content_type text)`	boolean	invoker	—	yes
`client_in_my_pods(p_client_id text)`	boolean	DEFINER	yes	yes
`close_pause_on_move()`	trigger	DEFINER	—	—
`code_prefix(p_name text)`	text	invoker	yes	yes
`create_order_threads()`	trigger	DEFINER	yes	yes
`create_payout_obligations()`	trigger	DEFINER	—	—
`credit_emails()`	trigger	DEFINER	yes	yes
`current_agreement(p_kind text)`	agreement_documents	DEFINER	yes	yes
`current_clerk_id()`	text	invoker	yes	yes
`current_pod_ids()`	text[]	DEFINER	yes	yes
`current_roles()`	role[]	DEFINER	yes	yes
`current_user_role()`	role	DEFINER	yes	yes
`delivered_at(p_order_id text)`	timestamp with time zone	DEFINER	—	—
`designer_is_assignable(p_designer_id text)`	boolean	DEFINER	yes	yes
`designer_open_minutes(p_designer_id text, p_excluding text)`	integer	DEFINER	—	—
`designer_pay_paise(p_letter character, p_designer_id text)`	integer	DEFINER	—	—
`designer_rate_paise(p_designer_id text)`	integer	DEFINER	—	—

Function	Returns	Mode	anon	authenticated
`email_order_client(p_order_id text, p_template text, p_key text, p_extra jsonb, p_kind text)`	void	DEFINER	—	—
`enforce_escrow_conservation()`	trigger	invoker	yes	yes
`enqueue_email(p_template text, p_recipient text, p_payload jsonb, p_idempotency_key text)`	void	DEFINER	yes	yes
`ensure_client_code(p_user_id text)`	text	DEFINER	—	—
`escrow_held(p_order_id text)`	integer	DEFINER	—	—
`escrow_sign(p_kind text)`	integer	invoker	yes	yes
`fanout_notifications()`	trigger	DEFINER	yes	yes
`first_design_available(p_client_id text, p_order_id text)`	boolean	DEFINER	—	—
`flag(p_key text)`	boolean	DEFINER	yes	yes
`has_role(p_role role)`	boolean	invoker	yes	yes
`is_admin()`	boolean	invoker	yes	yes
`is_assigned_designer(p_order_id text)`	boolean	DEFINER	yes	yes
`is_payee_account(p_user_id text)`	boolean	DEFINER	—	yes
`is_pod_lead_of(p_pod_id text)`	boolean	invoker	yes	yes
`is_staff()`	boolean	invoker	yes	yes
`is_team()`	boolean	DEFINER	—	yes
`last_moved_to(p_order_id text, p_status text)`	timestamp with time zone	DEFINER	—	—
`marketing_order_events()`	trigger	DEFINER	—	—
`marketing_payment_event()`	trigger	DEFINER	—	—
`maybe_start_sla(p_order_id text)`	boolean	DEFINER	—	—
`message_emails()`	trigger	DEFINER	yes	yes
`messages_take_client_ref()`	trigger	invoker	—	—
`missing_slots(p_order_id text)`	text[]	DEFINER	—	—

Function	Returns	Mode	anon	authenticated
`my_approved_order(p_order_id text)`	boolean	DEFINER	—	yes
`my_member_pod_ids()`	text[]	DEFINER	—	yes
`notify(p_user_id text, p_actor_id text, p_kind text, p_order_id text, p_summary text)`	void	DEFINER	yes	yes
`on_first_look()`	trigger	DEFINER	yes	yes
`on_hold_recorded()`	trigger	DEFINER	yes	yes
`only_pod()`	TABLE(pod_id text, lead_id text)	DEFINER	—	—
`order_email_payload(p_order_id text)`	jsonb	DEFINER	—	—
`order_in_my_pods(p_order_id text)`	boolean	DEFINER	yes	yes
`order_relation(p_order_id text)`	text	DEFINER	—	—
`order_status_emails()`	trigger	DEFINER	yes	yes
`orders_auto_pod()`	trigger	DEFINER	—	—
`other_active_ops(p_excluding text)`	integer	invoker	—	—
`payout_account_status(p_user_id text)`	text	DEFINER	yes	yes
`pod_pay_paise(p_designer_paise integer)`	integer	DEFINER	—	—
`product_code(p_product text)`	text	invoker	yes	yes
`qc_pay_paise(p_price_cents integer)`	integer	invoker	yes	yes
`quote_in_usd()`	trigger	invoker	—	—
`reject_mutation()`	trigger	invoker	yes	yes
`reprice_on_assign()`	trigger	DEFINER	—	—
`require_admin()`	void	invoker	—	—
`require_finance()`	void	DEFINER	—	yes
`require_ops()`	void	invoker	—	—
`require_triage_staff()`	void	invoker	yes	yes
`revision_allowance(p_order_id text)`	jsonb	DEFINER	—	—
`sees_money_of(p_order_id text)`	boolean	DEFINER	—	yes
`sees_order_money()`	boolean	DEFINER	—	yes
`service_code(p_service service_kind)`	text	invoker	yes	yes

Function	Returns	Mode	anon	authenticated
`set_designer_ref()`	trigger	invoker	yes	yes
`set_uploader_party()`	trigger	DEFINER	yes	yes
`setting(p_key text)`	integer	DEFINER	—	—
`signal_thread_change()`	trigger	DEFINER	—	—
`slot_kind(p_slot text)`	file_kind	invoker	yes	yes
`spec_gates()`	trigger	DEFINER	—	—
`team_on_order(p_order_id text)`	boolean	DEFINER	—	yes
`team_writes_brief(p_order_id text)`	boolean	DEFINER	—	yes
`thread_party(p_thread_id text)`	text	DEFINER	—	yes
`ticks_complete(p_order_id text, p_column text)`	boolean	DEFINER	—	—
`token_hash(p_token text)`	text	invoker	yes	yes
`user_has_role(p_user_id text, p_role role)`	boolean	DEFINER	—	—
`valid_stones(p_stones jsonb)`	boolean	invoker	—	—
`wants_email(p_user_id text, p_kind text)`	boolean	DEFINER	—	—
`weights_gate()`	trigger	DEFINER	yes	yes
`welcome_email()`	trigger	DEFINER	yes	yes

Schema `audit` (4)

Function	Returns	Mode	anon	authenticated
`chain_before_insert()`	trigger	invoker	—	—
`log_event(p_action text, p_entity_type text, p_entity_id text, p_actor_id text, p_actor_role role, p_payload jsonb, p_is_identity_read boolean)`	text	DEFINER	—	—
`prevent_mutation()`	trigger	invoker	—	—
`verify_chain()`	jsonb	invoker	—	—

APPENDIX

Appendix C · Routes reference (pages and API)

> Generated from the repository at commit `1c08b52`. Every row below is read from the code or from a database built from `db/migrations` and `db/policies`, not written by hand.

Guards are read from `core/auth/session.ts` (the middleware's own functions) and from each file. "Uses service role" marks code that bypasses RLS and must do its own authorisation first; see the security chapter.

Website pages (12)

Path	Methods	Guard	File
`/`	GET	public website page (Clerk middleware skipped on GET/HEAD)	`app/(marketing)/page.tsx`
`/apply-designer`	GET	public website page (Clerk middleware skipped on GET/HEAD)	`app/(marketing)/apply-designer/page.tsx`
`/contact`	GET	public website page (Clerk middleware skipped on GET/HEAD)	`app/(marketing)/contact/page.tsx`
`/faq`	GET	public website page (Clerk middleware skipped on GET/HEAD)	`app/(marketing)/faq/page.tsx`
`/guarantee`	GET	public website page (Clerk middleware skipped on GET/HEAD)	`app/(marketing)/guarantee/page.tsx`
`/how-it-works`	GET	public website page (Clerk middleware skipped on GET/HEAD)	`app/(marketing)/how-it-works/page.tsx`
`/jewelry-cad`	GET	public website page (Clerk middleware skipped on GET/HEAD)	`app/(marketing)/jewelry-cad/page.tsx`
`/pricing`	GET	public website page (Clerk middleware skipped on GET/HEAD)	`app/(marketing)/pricing/page.tsx`
`/privacy`	GET	public website page (Clerk middleware skipped on GET/HEAD)	`app/(marketing)/privacy/page.tsx`
`/quality`	GET	public website page (Clerk middleware skipped on GET/HEAD)	`app/(marketing)/quality/page.tsx`
`/results`	GET	public website page (Clerk middleware skipped on GET/HEAD); uses service role (after own checks)	`app/(marketing)/results/page.tsx`

Path	Methods	Guard	File
`/terms`	GET	public website page (Clerk middleware skipped on GET/HEAD)	`app/(marketing)/terms/page.tsx`

Sign-in pages (2)

Path	Methods	Guard	File
`/sign-in/[[...sign-in]]`	GET	middleware: Clerk session read, not required; handler: Clerk auth()	`app/(auth)/sign-in/[[...sign-in]]/page.tsx`
`/sign-up/[[...sign-up]]`	GET	middleware: Clerk session read, not required; handler: Clerk auth()	`app/(auth)/sign-up/[[...sign-up]]/page.tsx`

Portal pages (55)

Path	Methods	Guard	File
`/account`	GET	middleware: signed-in required; handler: Clerk auth(); uses service role (after own checks)	`app/(app)/account/page.tsx`
`/account/kit`	GET	middleware: signed-in required; handler: Clerk auth()	`app/(app)/account/kit/page.tsx`
`/account/security`	GET	middleware: signed-in required; handler: Clerk auth()	`app/(app)/account/security/page.tsx`
`/admin`	GET	middleware: signed-in required; page: requireTeam	`app/(app)/admin/page.tsx`
`/admin/access`	GET	middleware: signed-in required; handler: Clerk auth()	`app/(app)/admin/access/page.tsx`
`/admin/access/pods`	GET	middleware: signed-in required; handler: Clerk auth()	`app/(app)/admin/access/pods/page.tsx`
`/admin/applications`	GET	middleware: signed-in required; handler: Clerk auth(); uses service role (after own checks)	`app/(app)/admin/applications/page.tsx`
`/admin/billing`	GET	middleware: signed-in required; handler: Clerk auth()	`app/(app)/admin/billing/page.tsx`
`/admin/clients`	GET	middleware: signed-in required; handler: Clerk auth()	`app/(app)/admin/clients/page.tsx`
`/admin/credits`	GET	middleware: signed-in required; handler: Clerk auth()	`app/(app)/admin/credits/page.tsx`
`/admin/designers`	GET	middleware: signed-in required; handler: Clerk auth()	`app/(app)/admin/designers/page.tsx`
`/admin/designers/[id]`	GET	middleware: signed-in required; handler: Clerk auth()	`app/(app)/admin/designers/[id]/page.tsx`
`/admin/emails`	GET	middleware: signed-in required; page: requireTeam	`app/(app)/admin/emails/page.tsx`
`/admin/leads`	GET	middleware: signed-in required; handler: Clerk auth()	`app/(app)/admin/leads/page.tsx`

Path	Methods	Guard	File
`/admin/new-order`	GET	middleware: signed-in required; handler: Clerk auth()	`app/(app)/admin/new-order/page.tsx`
`/admin/orders`	GET	middleware: signed-in required; handler: Clerk auth()	`app/(app)/admin/orders/page.tsx`
`/admin/orders/export`	GET	middleware: signed-in required; handler: Clerk auth()	`app/(app)/admin/orders/export/route.ts`
`/admin/payouts`	GET	middleware: signed-in required; handler: Clerk auth()	`app/(app)/admin/payouts/page.tsx`
`/admin/payouts/[batch]`	GET	middleware: signed-in required; handler: Clerk auth()	`app/(app)/admin/payouts/[batch]/route.ts`
`/admin/settings`	GET	middleware: signed-in required; page: requireTeam	`app/(app)/admin/settings/page.tsx`
`/admin/users`	GET	middleware: signed-in required	`app/(app)/admin/users/page.tsx`
`/dashboard`	GET	middleware: signed-in required	`app/(app)/dashboard/page.tsx`
`/files`	GET	middleware: signed-in required; handler: Clerk auth()	`app/(app)/files/page.tsx`
`/home`	GET	middleware: signed-in required; handler: Clerk auth()	`app/(app)/home/page.tsx`
`/materials`	GET	middleware: Clerk session read, not required; handler: Clerk auth()	`app/(app)/materials/page.tsx`
`/messages`	GET	middleware: signed-in required; handler: Clerk auth()	`app/(app)/messages/page.tsx`
`/messages/[orderId]`	GET	middleware: signed-in required; handler: Clerk auth()	`app/(app)/messages/[orderId]/page.tsx`
`/onboarding/designer`	GET	middleware: signed-in required; handler: Clerk auth()	`app/(app)/onboarding/designer/page.tsx`
`/orders`	GET	middleware: signed-in required; handler: Clerk auth()	`app/(app)/orders/(list)/page.tsx`
`/orders/[id]`	GET	middleware: signed-in required; handler: Clerk auth(); uses service role (after own checks)	`app/(app)/orders/[id]/page.tsx`
`/orders/[id]/brief`	GET	middleware: signed-in required; handler: Clerk auth(); uses service role (after own checks)	`app/(app)/orders/[id]/brief/page.tsx`
`/orders/[id]/brief/review`	GET	middleware: signed-in required; handler: Clerk auth()	`app/(app)/orders/[id]/brief/review/page.tsx`
`/orders/[id]/claim`	GET	middleware: signed-in required; handler: Clerk auth()	`app/(app)/orders/[id]/claim/page.tsx`
`/orders/[id]/pay`	GET	middleware: signed-in required; handler: Clerk auth()	`app/(app)/orders/[id]/pay/page.tsx`
`/orders/[id]/review`	GET	middleware: signed-in required; handler: Clerk auth()	`app/(app)/orders/[id]/review/page.tsx`
`/orders/[id]/sent`	GET	middleware: signed-in required; handler: Clerk auth()	`app/(app)/orders/[id]/sent/page.tsx`

Path	Methods	Guard	File
`/orders/new`	GET	middleware: signed-in required; handler: Clerk auth()	`app/(app)/orders/new/page.tsx`
`/pod/board`	GET	middleware: signed-in required; page: requireTeam	`app/(app)/pod/board/page.tsx`
`/pod/designers/[id]`	GET	middleware: signed-in required; page: requireTeam	`app/(app)/pod/designers/[id]/page.tsx`
`/pod/performance`	GET	middleware: signed-in required; page: requireTeam	`app/(app)/pod/performance/page.tsx`
`/qc`	GET	middleware: signed-in required; page: requireTeam	`app/(app)/qc/page.tsx`
`/qc/[id]`	GET	middleware: signed-in required; page: requireTeam	`app/(app)/qc/[id]/page.tsx`
`/qc/earnings`	GET	middleware: signed-in required; handler: Clerk auth()	`app/(app)/qc/earnings/page.tsx`
`/qc/orders`	GET	middleware: signed-in required; page: requireTeam	`app/(app)/qc/orders/page.tsx`
`/settings/payouts`	GET	middleware: signed-in required; handler: Clerk auth()	`app/(app)/settings/payouts/page.tsx`
`/tools`	GET	middleware: Clerk session read, not required; handler: Clerk auth()	`app/(app)/tools/page.tsx`
`/wallet`	GET	middleware: signed-in required; handler: Clerk auth()	`app/(app)/wallet/page.tsx`
`/wallet/invoices`	GET	middleware: signed-in required; handler: Clerk auth()	`app/(app)/wallet/invoices/page.tsx`
`/wallet/transactions`	GET	middleware: signed-in required; handler: Clerk auth()	`app/(app)/wallet/transactions/page.tsx`
`/welcome`	GET	middleware: signed-in required; handler: Clerk auth()	`app/(app)/welcome/page.tsx`
`/work`	GET	middleware: signed-in required; handler: Clerk auth()	`app/(app)/work/page.tsx`
`/work/[id]`	GET	middleware: signed-in required; handler: Clerk auth()	`app/(app)/work/[id]/page.tsx`
`/work/[id]/checklist`	GET	middleware: signed-in required; handler: Clerk auth()	`app/(app)/work/[id]/checklist/page.tsx`
`/work/[id]/review`	GET	middleware: signed-in required; handler: Clerk auth(); uses service role (after own checks)	`app/(app)/work/[id]/review/page.tsx`
`/work/earnings`	GET	middleware: signed-in required; handler: Clerk auth()	`app/(app)/work/earnings/page.tsx`

API route handlers (20)

Path	Methods	Guard	File
`/admin/orders/export`	GET	middleware: signed-in required; handler: Clerk auth()	`app/(app)/admin/orders/export/route.ts`

Path	Methods	Guard	File
`/admin/payouts/[batch]`	GET	middleware: signed-in required; handler: Clerk auth()	`app/(app)/admin/payouts/[batch]/route.ts`
`/api/cron/emails`	GET	middleware: Clerk session read, not required; bearer CRON_SECRET; uses service role (after own checks)	`app/api/cron/emails/route.ts`
`/api/cron/push`	GET	middleware: Clerk session read, not required; bearer CRON_SECRET; uses service role (after own checks)	`app/api/cron/push/route.ts`
`/api/designer-documents/[id]`	GET	middleware: Clerk session read, not required; handler: Clerk auth(); uses service role (after own checks)	`app/api/designer-documents/[id]/route.ts`
`/api/files/[versionId]`	GET	middleware: Clerk session read, not required; handler: Clerk auth(); uses service role (after own checks)	`app/api/files/[versionId]/route.ts`
`/api/files/[versionId]/stream`	GET	middleware: Clerk session read, not required; handler: Clerk auth(); uses service role (after own checks)	`app/api/files/[versionId]/stream/route.ts`
`/api/health`	GET	middleware: Clerk session read, not required	`app/api/health/route.ts`
`/api/inbox/stamp`	GET	middleware: Clerk session read, not required; handler: Clerk auth()	`app/api/inbox/stamp/route.ts`
`/api/materials/[id]`	GET	middleware: Clerk session read, not required; handler: Clerk auth(); uses service role (after own checks)	`app/api/materials/[id]/route.ts`
`/api/notifications`	GET	middleware: Clerk session read, not required; handler: Clerk auth()	`app/api/notifications/route.ts`
`/api/orders/[id]/certificate`	GET	middleware: Clerk session read, not required; handler: Clerk auth(); uses service role (after own checks)	`app/api/orders/[id]/certificate/route.ts`
`/api/orders/[id]/stamp`	GET	middleware: Clerk session read, not required; handler: Clerk auth()	`app/api/orders/[id]/stamp/route.ts`
`/api/orders/[id]/summary`	GET	middleware: Clerk session read, not required; handler: Clerk auth(); uses service role (after own checks)	`app/api/orders/[id]/summary/route.ts`
`/api/orders/[id]/weight-sheet`	GET	middleware: Clerk session read, not required; handler: Clerk auth()	`app/api/orders/[id]/weight-sheet/route.ts`
`/api/pod/report`	GET	middleware: Clerk session read, not required; page: requireTeam	`app/api/pod/report/route.ts`

Path	Methods	Guard	File
`/api/reference-images/[imageId]`	GET	middleware: Clerk session read, not required; handler: Clerk auth(); uses service role (after own checks)	`app/api/reference-images/[imageId]/route.ts`
`/api/thread-files/[fileId]`	GET	middleware: Clerk session read, not required; handler: Clerk auth(); uses service role (after own checks)	`app/api/thread-files/[fileId]/route.ts`
`/api/threads/[id]/read`	POST	middleware: Clerk session read, not required; handler: Clerk auth()	`app/api/threads/[id]/read/route.ts`
`/api/webhooks/razorpay`	POST	middleware: Clerk session read, not required; HMAC signature; uses service role (after own checks)	`app/api/webhooks/razorpay/route.ts`

Other pages (6)

Path	Methods	Guard	File
`/design-preview`	GET	middleware: Clerk session read, not required	`app/design-preview/page.tsx`
`/design-preview/frame`	GET	middleware: Clerk session read, not required	`app/design-preview/frame/page.tsx`
`/login`	GET	middleware: Clerk session read, not required	`app/login/page.tsx`
`/offline`	GET	public website page (Clerk middleware skipped on GET/HEAD)	`app/offline/page.tsx`
`/p/[token]`	GET	middleware: Clerk session read, not required; uses service role (after own checks)	`app/p/[token]/page.tsx`
`/signup`	GET	middleware: Clerk session read, not required	`app/signup/page.tsx`

APPENDIX

Appendix D · Server actions reference

31 files, 92 exported actions.

> Generated from the repository at commit `1c08b52`. Every row below is read from the code or from a database built from `db/migrations` and `db/policies`, not written by hand.

A server action is a function the browser calls by posting a form; it runs on the server. Each one below is listed with the database functions (RPCs) it calls, which is where the real permission checks live (SECURITY DEFINER functions and RLS), and whether it uses the service role.

`app/(app)/account/actions.ts`

- Actions (2): `saveClientProfileAction`, `saveEmailPreferencesAction`
- Database functions called: `ensure_self`, `save_attribution`, `save_client_profile`, `set_email_preferences`
- Tables/views read or written directly: `client_profiles`
- Service role: no
- Guards in file: via the database functions it calls

`app/(app)/account/pushActions.ts`

- Actions (2): `savePushSubscriptionAction`, `deletePushSubscriptionAction`
- Database functions called: `delete_push_subscription`, `save_push_subscription`
- Tables/views read or written directly: —
- Service role: no
- Guards in file: via the database functions it calls

`app/(app)/admin/access/actions.ts`

- Actions (3): `setUserRoleAction`, `createLeadPodAction`, `setUserStatusAction`
- Database functions called: `create_pod`, `pods_overview`, `record_identity_lookup`, `set_pod_member`, `set_user_role`, `set_user_status`
- Tables/views read or written directly: —
- Service role: no
- Guards in file: via the database functions it calls

`app/(app)/admin/actions.ts`

- Actions (6): `choosePodAction`, `createPodAction`, `setPodMemberAction`, `removePodMemberAction`, `setAccountManagerAction`, `computeStatsAction`
- Database functions called: `assign_order_pod`, `compute_public_stats`, `create_pod`, `remove_pod_member`, `set_account_manager`, `set_pod_member`
- Tables/views read or written directly: —
- Service role: no
- Guards in file: via the database functions it calls

`app/(app)/admin/applications/actions.ts`

- Actions (1): `reviewApplicationAction`
- Database functions called: `review_designer_application`
- Tables/views read or written directly: —
- Service role: no
- Guards in file: via the database functions it calls

`app/(app)/admin/credits/actions.ts`

- Actions (1): `decideCreditAction`
- Database functions called: `decide_sla_credit`
- Tables/views read or written directly: —
- Service role: no
- Guards in file: via the database functions it calls

`app/(app)/admin/designers/actions.ts`

- Actions (7): `addDesignersAction`, `saveBankAction`, `uploadDocumentsAction`, `removeDocumentAction`, `setDesignerPodAction`, `revealBankAction`, `setDesignerRateAction`
- Database functions called: `add_designer_document`, `admin_designers`, `payout_account_full`, `register_designer`, `remove_designer_document`, `remove_pod_member`, `set_designer_rate`, `set_payout_account_for`, `set_pod_member`
- Tables/views read or written directly: —
- Service role: **yes** (storage or a trusted write after the caller's own check)
- Guards in file: requireAdmin

`app/(app)/admin/emails/actions.ts`

- Actions (1): `sendTestEmailsAction`
- Database functions called: —
- Tables/views read or written directly: `email_outbox`, `user_roles`
- Service role: **yes** (storage or a trusted write after the caller's own check)
- Guards in file: Clerk auth(), requireTeam

`app/(app)/admin/leads/actions.ts`

- Actions (1): `setLeadStatusAction`
- Database functions called: `set_lead_status`
- Tables/views read or written directly: —
- Service role: no
- Guards in file: via the database functions it calls

`app/(app)/admin/new-order/actions.ts`

- Actions (1): `newOrderForClientAction`
- Database functions called: `add_order_screenshot`, `create_order_on_behalf`, `i_am_team`, `register_client`, `submit_order_on_behalf`
- Tables/views read or written directly: —
- Service role: **yes** (storage or a trusted write after the caller's own check)
- Guards in file: via the database functions it calls

`app/(app)/admin/payouts/actions.ts`

- Actions (2): `exportPayoutsAction`, `markBatchPaidAction`
- Database functions called: `export_payouts`, `mark_payout_batch_paid`
- Tables/views read or written directly: —
- Service role: no
- Guards in file: via the database functions it calls

`app/(app)/admin/settings/actions.ts`

- Actions (2): `setSettingAction`, `setFlagAction`
- Database functions called: `set_app_setting`, `set_feature_flag`
- Tables/views read or written directly: —
- Service role: no
- Guards in file: via the database functions it calls

`app/(app)/home/actions.ts`

- Actions (1): `markNotificationsReadAction`
- Database functions called: `mark_notifications_read`
- Tables/views read or written directly: —
- Service role: no
- Guards in file: via the database functions it calls

`app/(app)/materials/actions.ts`

- Actions (3): `beginMaterialUploadAction`, `finishMaterialUploadAction`, `removeMaterialAction`
- Database functions called: `add_pod_material`, `material_pods`, `remove_pod_material`
- Tables/views read or written directly: —
- Service role: **yes** (storage or a trusted write after the caller's own check)
- Guards in file: via the database functions it calls

`app/(app)/messages/actions.ts`

- Actions (1): `markConversationReadAction`
- Database functions called: —
- Tables/views read or written directly: —
- Service role: no
- Guards in file: via the database functions it calls

`app/(app)/onboarding/designer/actions.ts`

- Actions (2): `applyAsDesignerAction`, `signAgreementAction`
- Database functions called: `accept_designer_agreement`, `apply_as_designer`
- Tables/views read or written directly: —
- Service role: no
- Guards in file: via the database functions it calls

`app/(app)/orders/[id]/brief/actions.ts`

- Actions (4): `startBriefAction`, `duplicateBriefAction`, `saveBriefAction`, `sendBriefAction`

- Database functions called: `add_reference_image`, `create_order`, `ensure_self`, `record_marketing_event`, `set_order_accents`, `set_order_service`, `set_primary_reference`, `set_reference_pins`, `submit_brief`, `upsert_order_spec`, `visible_orders`
- Tables/views read or written directly: `order_reference_images`, `order_spec_accents`, `order_specs`
- Service role: **yes** (storage or a trusted write after the caller's own check)
- Guards in file: via the database functions it calls

`app/(app)/orders/[id]/brief/referenceActions.ts`

- Actions (4): `addReferenceImageAction`, `setPinsAction`, `setPrimaryReferenceAction`, `removeReferenceImageAction`
- Database functions called: `add_reference_image`, `remove_reference_image`, `set_primary_reference`, `set_reference_pins`
- Tables/views read or written directly: `order_reference_images`
- Service role: **yes** (storage or a trusted write after the caller's own check)
- Guards in file: via the database functions it calls

`app/(app)/orders/actions.ts`

- Actions (19): `createOrderAction`, `transitionAction`, `qcDecisionAction`, `setVolumeAction`, `recordMeasurementsAction`, `addWaiverAction`, `issueCertificateAction`, `quoteAction`, `confirmOrderAction`, `markPaidAction`, `releaseEscrowAction`, `refundEscrowAction`, `postMessageAction`, `raiseDisputeAction`, `resolveDisputeAction`, `assignDesignerAction`, `markCheckAction`, `requestRevisionAction`, `changeOrderAction`
- Database functions called: `add_spec_waiver`, `assign_designer`, `confirm_order`, `create_change_order`, `create_order`, `mark_check`, `mark_payment_received`, `post_message`, `qc_decide`, `quote_order_v2`, `raise_dispute`, `record_qc_measurements`, `request_revision`, `resolve_dispute`, `set_job_volume`, `transition_order`
- Tables/views read or written directly: —
- Service role: no
- Guards in file: via the database functions it calls

`app/(app)/orders/chat/actions.ts`

- Actions (1): `sendChatMessageAction`
- Database functions called: `post_thread_message`, `send_chat_message`
- Tables/views read or written directly: —
- Service role: no
- Guards in file: via the database functions it calls

`app/(app)/orders/fileActions.ts`

- Actions (6): `uploadFileAction`, `beginSlotUploadAction`, `finishSlotUploadAction`, `saveWeightsAction`, `postThreadAction`, `forwardMessageAction`
- Database functions called: `add_file_version`, `add_slot_file`, `job_slots`, `job_weights_get`, `my_work`, `order_messages`, `order_threads`, `post_thread_file`, `post_thread_message`, `set_job_weights`, `set_tech_source`, `thread_attachments`, `thread_file_for_download`
- Tables/views read or written directly: `file_versions`
- Service role: **yes** (storage or a trusted write after the caller's own check)
- Guards in file: via the database functions it calls

`app/(app)/orders/paymentActions.ts`

- Actions (1): `startPaymentAction`
- Database functions called: `open_payment_intent`, `visible_orders`
- Tables/views read or written directly: `users`
- Service role: **yes** (storage or a trusted write after the caller's own check)
- Guards in file: Clerk auth()

`app/(app)/orders/payoutActions.ts`

- Actions (2): `sendPayoutsAction`, `reconcilePayoutsAction`
- Database functions called: `open_payouts_for_order`
- Tables/views read or written directly: `users`
- Service role: **yes** (storage or a trusted write after the caller's own check)
- Guards in file: Clerk auth(), requireFinance

`app/(app)/orders/previewActions.ts`

- Actions (5): `createPreviewLinkAction`, `updatePreviewLinkAction`, `uploadLogoAction`, `openClaimAction`, `decideClaimAction`
- Database functions called: `claim_window`, `create_preview_link`, `decide_spec_claim`, `open_spec_claim`, `set_client_logo`, `update_preview_link`
- Tables/views read or written directly: —
- Service role: **yes** (storage or a trusted write after the caller's own check)
- Guards in file: Clerk auth()

`app/(app)/pod/actions.ts`

- Actions (4): `reassignJobAction`, `withdrawJobAction`, `escalateJobAction`, `resolveEscalationAction`
- Database functions called: `escalate_job`, `reassign_job`, `resolve_escalation`, `withdraw_job`
- Tables/views read or written directly: —
- Service role: no
- Guards in file: via the database functions it calls

`app/(app)/settings/payouts/actions.ts`

- Actions (1): `savePayoutAccountAction`
- Database functions called: `upsert_payout_account`
- Tables/views read or written directly: —
- Service role: no
- Guards in file: via the database functions it calls

`app/(app)/work/actions.ts`

- Actions (2): `pauseJobAction`, `resumeJobAction`
- Database functions called: `pause_job`, `resume_job`
- Tables/views read or written directly: —
- Service role: no
- Guards in file: via the database functions it calls

`app/(marketing)/apply-designer/actions.ts`

- Actions (1): `submitDesignerApplicationAction`

- Database functions called: `submit_designer_application`
- Tables/views read or written directly: —
- Service role: **yes** (storage or a trusted write after the caller's own check)
- Guards in file: rate limit

`app/(marketing)/contact/actions.ts`

- Actions (1): `submitLeadAction`
- Database functions called: `submit_marketing_lead`
- Tables/views read or written directly: —
- Service role: no
- Guards in file: rate limit

`app/(marketing)/jewelry-cad/actions.ts`

- Actions (1): `startBriefLeadAction`
- Database functions called: `record_marketing_event`, `start_brief_lead`
- Tables/views read or written directly: —
- Service role: no
- Guards in file: Clerk auth(), rate limit

`lib/client/standingActions.ts`

- Actions (4): `saveMasterInstructionsAction`, `saveOrderMasterInstructionsAction`, `addCertificateAction`, `removeCertificateAction`
- Database functions called: `add_stone_certificate`, `order_stone_certificates_for`, `remove_stone_certificate`, `set_master_instructions`, `set_order_master_instructions`
- Tables/views read or written directly: —
- Service role: **yes** (storage or a trusted write after the caller's own check)
- Guards in file: via the database functions it calls

APPENDIX

Appendix E · Configuration variables (names only)

> Generated from the repository at commit `1c08b52`. Every row below is read from the code or from a database built from `db/migrations` and `db/policies`, not written by hand.

Values are never written here. Secrets live in Vercel → Project → Settings → Environment Variables (and locally in an untracked `.env.local`). `NEXT_PUBLIC_` variables are built into the pages and visible to anyone; never put a secret in one.

Variable	Public?	Likely secret	In .env.example	Read in code
`APP_URL`	server only	—	—	yes
`CLERK_DEV_SECRET_KEY`	server only	yes	—	yes
`CLERK_JWKS_URL`	server only	—	yes	yes
`CLERK_JWT_ISSUER`	server only	—	yes	yes
`CLERK_PROD_SECRET_KEY`	server only	yes	—	yes
`CLERK_SECRET_KEY`	server only	yes	yes	—
`CRON_SECRET`	server only	yes	yes	yes
`DATABASE_URL`	server only	yes	yes	yes
`EMAIL_BATCH`	server only	—	—	yes
`EMAIL_DELIVERY`	server only	—	—	yes
`EMAIL_FROM`	server only	—	yes	yes
`EMAIL_REPLY_TO`	server only	—	yes	yes
`EMAIL_TEST_TO`	server only	—	—	yes
`NEXT_PUBLIC_CHAT_REALTIME`	yes (in the browser)	—	yes	yes
`NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY`	yes (in the browser)	—	yes	—
`NEXT_PUBLIC_CLERK_SIGN_IN_URL`	yes (in the browser)	—	yes	—
`NEXT_PUBLIC_CLERK_SIGN_UP_URL`	yes (in the browser)	—	yes	—
`NEXT_PUBLIC_GOOGLE_SITE_VERIFICATION`	yes (in the browser)	—	—	yes
`NEXT_PUBLIC_SITE_URL`	yes (in the browser)	—	yes	yes
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	yes (in the browser)	—	yes	yes
`NEXT_PUBLIC_SUPABASE_URL`	yes (in the browser)	—	yes	yes
`NEXT_PUBLIC_VAPID_PUBLIC_KEY`	yes (in the browser)	—	yes	yes

Variable	Public?	Likely secret	In .env.example	Read in code
`NEXT_PUBLIC_VERCEL_ENV`	yes (in the browser)	—	—	yes
`NEXT_SERVER_ACTION_ALLOWED_ORIGINS`	server only	—	yes	—
`NODE_ENV`	server only	—	—	yes
`PUSH_BATCH`	server only	—	—	yes
`RATE_LIMIT_SALT`	server only	yes	yes	yes
`RAZORPAY_KEY_ID`	server only	yes	yes	yes
`RAZORPAY_KEY_SECRET`	server only	yes	yes	yes
`RAZORPAY_WEBHOOK_SECRET`	server only	yes	yes	yes
`RESEND_API_KEY`	server only	yes	yes	yes
`SUPABASE_SERVICE_ROLE_KEY`	server only	yes	yes	yes
`VAPID_PRIVATE_KEY`	server only	yes	yes	yes
`VAPID_SUBJECT`	server only	—	yes	yes